
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 20:44:02
Файлов исходного кода: 82
Суммарный объём кода: 1.15 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/82: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
+env:
+  CI: false
```


- ```
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
 done
 convert -version
```
- name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.density}
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
  - name: Summary  
run: |  
{  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \public/export/full\_code\_all\_pages  
 echo "| \public/export/full\_code\_all\_pages  
 echo "| PNG-страниц | \$(ls tmp/export-pages.  
} >> "\$GITHUB\_STEP\_SUMMARY"
  - name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7

```
 steps:
 - name: Checkout
 uses: actions/checkout@v4
+ uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v4
+ uses: actions/setup-node@v7
 with:
 node-version: "22"
+ node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
 } >> "$GITHUB_STEP_SUMMARY"

 - name: Upload artifacts (PDF + TXT)
 uses: actions/upload-artifact@v4
+ uses: actions/upload-artifact@v7
 with:
 name: full-code-export
 path: |
@@ -107,7 +107,7 @@ jobs:
 retention-days: 30

 - name: Upload artifacts (PNG-страницы)
 uses: actions/upload-artifact@v4
+ uses: actions/upload-artifact@v7
 with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png

`index.html`
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
  <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
```

```
new file mode 100644
```

```
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
```

```
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
```

```
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
```

```
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
```

```
+</svg>
```

````

## `src/App.tsx`

### Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
```

```
import { chapters } from '../data/content';
```

```
import { Navbar } from '../components/Navbar';
```

```
import { Sidebar } from '../components/Sidebar';
```

```
import { MobileToc } from '../components/MobileToc';
```

```
import { ChapterView } from '../components/ChapterView';
```

```
import { ChapterNav } from '../components/ChapterNav';
```



```

<div className="min-h-screen bg-slate-950 text-slate-50" >
  <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

  {activeTab === "guide" ? (
    <>
      <MobileToc
        selectedChapterId={activeChapterId}
        setSelectedChapterId={goTo}
        currentTitle={activeChapter.title}
        index={index}
        total={chapters.length}
      />

      <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
        {/* Сайдбар только на десктопе – на мобильном скрывается */}
        <div className="hidden lg:block lg:w-80 shrink-0" >
          <Sidebar selectedChapterId={activeChapterId} />
        </div>

        <main id="main-content" className="flex-1 min-w-0" >
          {/* Шапка главы с быстрыми переходами */}
          <div className="flex items-center justify-between" >
            <div className="min-w-0" >
              <div className="text-[10px] uppercase" >
                {activeChapter.category || "Универсальное"}
              </div>
              <h2 className="text-base sm:text-xl font-medium" >
                {activeChapter.title}
              </h2>
            </div>
            <ChapterNav prev={prev} next={next} index={index} />
          </div>

          <div className="h-1 w-full bg-slate-800 rounded" >
            <div
              className="h-full bg-gradient-to-r from-slate-800 to-slate-900"
              style={{ width: `${progress}%` }}
            />

```

Patch относительно `main`

```

`diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-   const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+   const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
   const [activeChapterId, setActiveChapterId] = useState<string>("");
   const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
    </>
-   ) : (
+   ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
      </>
    </main>
-   ) : (
+   ) : (
+     <PythonCompiler />
  )}

  <SimulatorModal vizId={modalVizId} onClose={() => setModalVizId("")} />

```

```
    <div className="min-w-0 flex-1">
      <h1 className="text-xl font-extrabold text-slate-800">
        Универсальное пособие
      </h1>
      <p className="text-xs text-indigo-400 font-extrabold">
        Подготовка к экзаменам: Алгоритмы, Структуры
      </p>
    </div>
  </div>

  <div className="flex items-center w-full lg:w-auto" >
    >
    <button
      id="tab-guide-btn"
      onClick={() => setActiveTab('guide')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <BookOpen className="w-4 h-4" /> Учебное пособие
    </button>
    <button
      id="tab-simulator-btn"
      onClick={() => setActiveTab('simulator')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <Cpu className="w-4 h-4" /> Тренажёры
    </button>
    <button
      id="tab-compiler-btn"
      onClick={() => setActiveTab('compiler')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'compiler'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <Code className="w-4 h-4" /> Компилятор
    </button>
  </div>
</div>
```

```
        className="flex items-center justify-center
        er:bg-amber-600/20 border border-amber-500/
        title="Скачать всё пособие одним автономным
      >
        {busy ? <Loader2 className="w-4 h-4 animate
        </button>
      </div>
    </div>
  </header>
);
};
},
```

Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

 export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
 >
 <Cpu className="w-4 h-4" /> Тренажёры
```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/full/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/full/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в переменную
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {
 module as PyodideModule,
 }).then((module) => module.loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}
```

```

 }
 }, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>

 <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
 <section className="rounded-2xl border border-slate-200 p-4">
 <div className="flex flex-wrap items-center">
 <div className="flex items-center gap-2">


```

```

 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 border"
 <label className="block px-4 pt-3 text-slate-500"
 Ввод для input() • по одной строке на клавишу
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.target.value)}
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-transparent border"
 style={{border: '1px solid #333'}}
 />
 </div>
</section>

<section className="rounded-2xl border border-slate-800 border"
 <div className="flex items-center justify-between"
 <div className="flex items-center gap-2 text-slate-500"
 <Terminal className="w-4 h-4 text-emerald-500"
 />
 <span className={`inline-flex items-center gap-2`}
 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500"
 {runtimeReady ? "готов" : "не загружен"}
 />

 </div>
 <pre className="min-h-[360px] max-h-[560px] border border-slate-800 p-4"
 x] leading-6 text-slate-300">
 {output}
 </pre>
 <div className="border-t border-slate-800 border"
 </section>
</div>

```

```

+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) =>
+ setStderr: (options: { batched: (message: string) =>
+ setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE
+ (module as PyodideModule).loadPyodide({ indexURL
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
+ const [code, setCode] = useState(STARTER_CODE);
+ const [stdin, setStdin] = useState("");
+ const [output, setOutput] = useState("Нажмите «Запус
+ const [running, setRunning] = useState(false);
+ const [runtimeReady, setRuntimeReady] = useState(fal

```



```

+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"
+ >
+ Как это работает <ExternalLink className="text-slate-400" />
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex flex-wrap items-center gap-2">
+ <div className="flex items-center gap-2">
+
+
+
+ </div>
+ <div className="flex items-center gap-2">
+ <button
+ type="button"
+ onClick={reset}

```

```

+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-transparent border-slate-800 p-2"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-800 p-4">
+ <div className="flex items-center justify-between">
+ <div className="flex items-center gap-2">
+ <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+ </div>
+
+ <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+ {runtimeReady ? "готов" : "не загружен"}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800 p-2">
+ </div>
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3">
+ <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+ <Info className="w-4 h-4 shrink-0 text-indigo-500 font-mono" />
+ Ctrl или Cmd + Enter запускает код. L
+ </div>
+ <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+ <Info className="w-4 h-4 shrink-0 text-amber-500 font-mono" />

```

```
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
 },
});
\\
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
  },
}
```

Универсальное пособие • полный исходный код

- Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и классы элементов в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/82: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <title> Дискретка для тупых – Интерактивный гид по дискретной математике
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/82: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "description": "Удобная web-читалка для подготовки к экзаменам по дискретной математике",
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

Универсальное пособие • полный исходный код

```
-----  
    "@vitejs/plugin-react": "5.1.1",  
    "esbuild": "^0.28.2",  
    "tailwindcss": "4.1.17",  
    "typescript": "5.9.3",  
    "vite": "7.3.2",  
    "vite-plugin-singlefile": "2.3.0"  
  },  
  "description": "Интерактивное пособие по алгоритмам и  
}
```

ФАЙЛ 7/82: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с
и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash  
npm install
npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---------------------|
| `npm run export:txt` | код → txt | `public/export/ful` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/ful` |

Универсальное пособие • полный исходный код

известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор).
Словарь — `src/data/glossary.ts`, мини-анимации — `src/mini-animations/`
* **Реестр интерактивов** — `src/components/vizRegistry`
и для панели под главой, и для модальки из подсказки.

Структура

...

| | |
|--------------------|---|
| src/ | |
| App.tsx | — оболочка, привязка глав к визуализации |
| components/ | — интерактивные симуляторы (Действия) |
| data/ | — контент пособия (главы/билеты/карты) |
| scripts/export/ | — пайплайн код → txt → png → pdf |
| public/export/ | — сгенерированные TXT и PDF (обновляются) |
| .github/workflows/ | — автоматизация |

...

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и классы `h2` и `h2-1`.
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика: Используют тег `

` и классы `p` и `p-1`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| | | |
|--------------------|-----------|----------------------|
| Глава | id | Что делать |
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| | | |
|--------|--------------------------------|-----------------------------|
| Билет | Тема | Статус |
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. К |
| | ит вхолостую. | |
| **7** | Планарность и раскраска графов | Номерной глав |
| | огии**. | |
| **11** | Мосты в графах | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- * `stack-dfs` → `StackViz.tsx` – **Стек и DFS**
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **Очередь
- * `heap-beam-search` → `HeapViz.tsx` – **Куча и лучевой
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – **Дер
- * `dynamic-programming` → `DPVisualizer.tsx` – **Динами

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

| | | |
|--|--------------|-------------|
| Глава | id | Комментарий |
| --- | --- | --- |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e | |

Универсальное пособие • полный исходный код

| | | | | |
|----|--------------------------------------|---|---|---|
| 17 | 17. Графы. Алгоритм Джонсона | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - | - |
| 22 | 22. Z-функция | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| - | Обзор: Кратчайшие пути и Графы | - | - | - |
| - | Алгоритмы на карте | - | - | - |
| - | Эйлер: Путь ≠ Цикл | - | - | - |
| - | Мосты в графах: код + визуализация | 1 | - | - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)
это 5 готовых компонентов, которые сейчас просто не...
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш...
3. ****Добавить 2D**** к 4 главам «Уровня 4» – минимум inli...
4. ****Решить судьбу `alg-map`**** – раскрыть или удалить.

ФАЙЛ 9/82: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],
```



```
-----
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

=====

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

ФАЙЛ 11/82: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 KB

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate
```

```
    <Navbar activeTab={activeTab} setActiveTab={setAc
```

```
    {activeTab === "guide" ? (
```

```
      <>
```

```
        <MobileToc
```

```
          selectedChapterId={activeChapterId}
```

```
          setSelectedChapterId={goTo}
```

```
          currentTitle={activeChapter.title}
```

```
          index={index}
```

```
          total={chapters.length}
```

```
        />
```

```
      <div className="flex flex-col lg:flex-row max
```

```
        {/* Сайдбар только на десктопе – на мобильном
```

```
        <div className="hidden lg:block lg:w-80 shr
```

```
          <Sidebar selectedChapterId={activeChapter
```

```
        </div>
```

```
      <main id="main-content" className="flex-1 m
```

```
        {/* Шапка главы с быстрыми переходами */}
```

```
        <div className="flex items-center justify
```

```
          <div className="min-w-0">
```

```
            <div className="text-[10px] uppercase
```

```
              {activeChapter.category || "Универс
```

```
            </div>
```

```
            <h2 className="text-base sm:text-xl f
```

```
              {activeChapter.title}
```

```
            </h2>
```

```
          </div>
```

```
          <ChapterNav prev={prev} next={next} ind
```

```
        </div>
```

```
      <div className="h-1 w-full bg-slate-800 r
```

```
        <div
```

ФАЙЛ 12/82: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
  const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
  // границы «не буква/цифра» – \b не работает с кириллицей
  return new RegExp(`(?<![\\p{L}\\p{N}]_)(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * <span class="term-hint" data-term-id="...">, чтобы на
 * повесить всплывающую карточку (как превью ссылок в Википедии)
```

```

    const usedTerm = perTerm.get(id) ?? 0;
    const usedSurface = perSurface.get(surface) ?? 0;
    if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
      perTerm.set(id, usedTerm + 1);
      perSurface.set(surface, usedSurface + 1);
    }

    if (m.index > last) frag.appendChild(document.createElement("div"));
    const span = document.createElement("span");
    span.className = "term-hint";
    span.dataset.termId = id;
    span.tabIndex = 0;
    span.setAttribute("role", "button");
    span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
    span.textContent = m[1];
    frag.appendChild(span);
    last = m.index + m[1].length;
  }

  if (last === 0) continue;
  if (last < text.length) frag.appendChild(document.createElement("div"));
  if (textNode.parentNode?.replaceChild(frag, textNode)) {
    //
  }
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
  const run = () => {
    const el = ref.current;
    if (!el) return;
    try {
      annotateTerms(el);
    }
  };
}

```

```
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
  from {
    transform: translateX(-100%);
  }
  to {
    transform: translateX(0);
  }
}

@keyframes hintIn {
  from {
    opacity: 0;
  }
}
```

```
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
    white-space: pre-wrap;
    word-break: break-word;
}

.chapter-body pre {
    overflow-x: auto;
    max-width: 100%;
    font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
    display: block;
    width: 100%;
    overflow-x: auto;
    border-collapse: collapse;
    font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
    cursor: pointer;
    padding: 0.35rem 0;
}

@media (max-width: 767px) {
    .chapter-body :where(.grid) {
        grid-template-columns: minmax(0, 1fr) !important;
    }
    .chapter-body :where(h2) {
        font-size: 1.25rem !important;
    }
}
```

```
    from {  
      width: 0%;  
    }  
    to {  
      width: 100%;  
    }  
  }  
}
```

ФАЙЛ 14/82: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";  
import { createRoot } from "react-dom/client";  
import "./index.css";  
import App from "./App";  
  
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

ФАЙЛ 15/82: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {  
  id: string;  
  title: string;  
  type: "html" | "markdown";  
  content: string;  
  description?: string;  
  category?: string;
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">Сложность
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">2. Двумерная разреженная матрица (Sparse Table)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-emerald-400">Идея
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Представим себе двумерную матрицу, где каждый элемент — это результат операции над двумя элементами из массива.
        <p class="text-xl font-mono text-white">Например, для операции взятия минимума (min) мы можем использовать двоичную кучу. Два базовых метода: Split и Merge.
      </div>
      <p class="text-slate-300 text-sm mb-4">Пара операций: Split и Merge.
      <div class="text-center my-6">
        <p class="text-slate-400 text-xs uppercase">Аналогия 1
        <p class="my-1 font-bold leading-none">
          <span class="text-2xl text-slate-300">И</span>
          <span class="text-2xl text-slate-300">И</span>
        </p>
        <p class="text-slate-300 font-mono text-sm">
          <div class="grid grid-cols-1 xl:grid-cols-2">
            <!-- Аналогия 1 -->
            <div class="bg-slate-800 p-6 rounded-lg">

```


ение – `<code class="bg-slate-950 px-2 py-0 code">. Худший случай – приоритеты вытянулись</code>
ose-300 border border-slate-800">O(n)</code>
-rose-300 border border-slate-800">O(n²)</code>`

`<p class="text-slate-500 text-xs mt-3 leading">`
и останется набор букв, как treap без у ост
но кричат по «!»: корень самый громкий, род
; равные x – идут влево по уставу.`</p>`

`<details class="bg-slate-900/40 rounded-lg l">`
`<summary class="p-4 font-bold text-slate">`
`-b border-slate-700/50">`

Код: split (Скрыто)

`</summary>`

`<div class="p-5 text-sm text-slate-300"`

`<pre class="bg-slate-950 p-4 rounded">`
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}`</pre>`

`</div>`

`</details>`

`<details class="bg-slate-900/40 rounded-lg l">`
`<summary class="p-4 font-bold text-slate">`
`-b border-slate-700/50">`

Код: вставка, merge, erase (Скрыто)

`</summary>`

`<div class="p-5 text-sm text-slate-300"`

```

<span class="bg-indigo-600 text-white px-4 py-1">
  <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
    <h3 class="text-xl font-bold text-blue-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
        метода: Split и Merge.</p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4
          border border-emerald-500 shadow-md">
          Аналогия 1: Split – таможня
        </div>
        <p class="text-slate-300 text-sm mb-4">
          > х». Спуск от корня: узел, который вмещает
          <b> без вскрытия</b> – дальше досматривается толк
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
            border border-rose-500 shadow-md">
            Аналогия 2: Merge – стыковка
          </div>
          <p class="text-slate-300 text-sm mb-4">
            дый шаг – одно сравнение: чей у у корня больше
            что сохраняет порядок х). Итого O(h) сравнений
          </div>
        </div>
      <details class="bg-slate-900/40 rounded-lg border border-slate-800">
        <summary class="p-4 font-bold text-slate-300">
          -b border-slate-700/50">
            Код (Скрыто)
          </summary>
          <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded">

```

```
<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Точки на клетчатке

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

Каждая пара – точка на бумаге в
рево = правильно соединить: линия всегда па
на каждом шаге спуска от корня.

«!» держит всё слово: убери его
ево поиска, которое на неудачном вводе выро
– идут влево по уставу. В демо-наборе все :

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
```

```
<div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Ты путаешь

```
</div>
```

```
<p class="text-slate-300 text-sm mb
не быстрее  $n \cdot \log n$ .</p>
```

```
<p class="text-slate-300 text-sm mb
/p>
```

```
<p class="text-slate-300 text-sm">.
сшиваются merge за  $O(h)$ .</p>
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-400 text-sm mt-6">Симу.
uicksort против counting → линии в любом по
<b>Миф  $2k/2k+1$ </b> и <b>Поиск ≠ сортировка</b>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
"bg-slate-950 px-2 py-0.5 rounded text-xs f
оение – <code class="bg-slate-950 px-2 py-0
code> (сортировка + n спусков; counting по
2 py-0.5 rounded text-xs font-mono text-ros
x-2 py-0.5 rounded text-xs font-mono text-r
```

```

    id: "splay-tree",
    title: "4. Splay-дерево",
    type: "html",
    description: "Дерево поиска, которое чинит само себя",
    category: "Продвинутые структуры",
    content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">

    <!-- 0. Определение -->
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-lg sm:text-xl font-mono">
        поиска <span class="text-slate-500">без</span>
        : серия поворотов, которая поднимает v в корень
        <p class="text-sm text-slate-400 text-cyan-300">
        т – поворот.</p>
      </div>

      <div class="grid grid-cols-1 lg:grid-cols-3">
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-white font-bold">можн
          <p class="text-slate-400 text-xs mt-2">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-emerald-300 font-bold">
          <p class="text-slate-400 text-xs mt-2">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">

```



```

        <td class="py-3 pr-3"><span
        <td class="py-3">x поднялся
    </tr>
    <tr>
        <td class="py-3 pr-3 font-b
        <td class="py-3 pr-3">x и p
        <td class="py-3 pr-3"><span
        <td class="py-3">p и g стал
    </tr>
</tbody>
</table>
</div>

```

```

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>
</div>

```

```

<!-- 5. Главная ловушка -->
<div class="bg-rose-950/30 p-6 rounded-xl borde
    <h3 class="text-lg font-bold text-rose-300
    <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
    <div class="grid grid-cols-1 lg:grid-cols-2
        <div class="bg-slate-950 rounded-lg p-4
            <p class="text-rose-400 font-bold t
            <pre class="overflow-x-auto text-[1

```

```

      20
     /
    40
   /
  20
 /
20

→

      20
     /
    20
   \
  40
 \

→

      60
     /
    40
   \
  40

```

было: бамбук влево
 стало: бамбук вправо
 глубина никого не сократилась!</pre>

```

<p class="text-slate-400 text-xs mt

```

итог: было 4 узла в линию (высота 4) → стало высота 3,
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра  
инили дерево по пути</span>. Все узлы, мимо  
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">  
  <h3 class="text-lg font-bold text-white mb-3">Сбалансированное  
  <p class="text-slate-300 text-sm mb-3">Строим дерево (или сбалансиро  
оддерева) по всем узлам, и Zig-Zig/Zig-Zag :  
ах идея такая:</p>  
  <div class="grid grid-cols-1 md:grid-cols-3 gap-4">  
    <div class="bg-slate-900 rounded-lg p-4">  
      <p class="text-white font-bold mb-1">Шаг 1: Zig-Zig  
      <p class="text-slate-400 text-xs">сбалансировать  
    </div>  
    <div class="bg-slate-900 rounded-lg p-4">  
      <p class="text-white font-bold mb-1">Шаг 2: Zig-Zag  
      <p class="text-slate-400 text-xs">сбалансировать  
    </div>  
    <div class="bg-slate-900 rounded-lg p-4">  
      <p class="text-white font-bold mb-1">Шаг 3: Zig-Zig  
      <p class="text-slate-400 text-xs">сбалансировать  
    </div>  
  </div>  
  <p class="text-slate-400 text-xs mt-4">Отсюда видно, что  
сти запросов splay-дерево не хуже (с точностью до O(1))  
дерево. Оно как бы само подстраивается под  
</div>
```

```
<!-- 8. Операции -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">  
  <h3 class="text-lg font-bold text-white mb-3">Операции  
  <p class="text-slate-300 text-sm mb-4">Красно-черное дерево  
лается в корне.</p>  
  <div class="space-y-2 text-sm">
```

```

    p.parent = x;                // родитель уехал вниз
    x.parent = g;                // x занял его место
    if (g) { if (g.left === p) g.left = x; else g.right =
}

// поднимаем x в корень
function splay(x) {
    while (x.parent) {
        const p = x.parent, g = p.parent;

        if (!g) {                // ZIG: деда не
            rotate(x);
        } else if ((g.left === p) === (p.left === x)) {
            rotate(p);            // ZIG-ZIG: сн
            rotate(x);
        } else {
            rotate(x);            // ZIG-ZAG: сн
            rotate(x);
        }
    }
    return x;                    // теперь x —
}

// поиск: спустились и подняли найденное наверх
function find(root, key) {
    let cur = root, last = null;
    while (cur) {
        last = cur;
        if (key === cur.key) break;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return last ? splay(last) : null;    // splay делае
}

```

>Вся p
ate(x); против ate(x);
 т свою оценку.</p>
 </div>


```

        <strong class="text-white block mb-6">
        <p class="text-slate-400">Потому что они
        еворачивают бамбук в другую сторону (см. блок
        </div>
        <div>
        <strong class="text-white block mb-6">
        <p class="text-slate-400">Чередование
        вает один из них в корень, превращая дерево
        ступает только для <em>одной</em> конкретной
        </div>
        <div>
        <strong class="text-white block mb-6">
        <p class="text-slate-400">Треар держит
        play не хранит вообще ничего и держит баланс
        <p>
        </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сжатие суммы подматрицы за O(1)",
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-6">

```

```

</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4
order border-emerald-500 shadow-md">
      Аналогия 1: Карта мира
    </div>
    <p class="text-slate-300 text-sm mb-6">
      . Границы не пересекаются в одной точке по-
      ы не сливались цветами, Всегда можно всего `
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        перед.</p>
      </div>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2">
      <div class="bg-slate-800 p-6 rounded-lg">

```

```

        <p class="text-xl font-mono text-white">
нвертированному в порядке top-sort.</p>
</div>
<div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторон
        </div>
        <p class="text-slate-300 text-sm mb
БЫХ направлениях по односторонним улицам. К
у на карте.</p>
    </div>
</div>
</div>
</div>
</section>`,
    },
    {
        id: "graph-bridges",
        title: "11. Графы. Мосты",
        type: "html",
        description: "Рёбра, удаление которых расхреничивае
category: "Графы. Продвинутое",
        content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 r
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-rose-400 m
            <div class="bg-slate-900 p-4 rounded-lg mb-
                <p class="text-lg text-rose-300 italic m
                <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
            </div>

```

```
-----
        Аналогия 1: Центральный роутер
      </div>
      <p class="text-slate-300 text-sm mb-6">
        правильный роутер - и два сегмента офиса б
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрыв
          </div>
          <p class="text-slate-300 text-sm mb-6">
            е сходится нечетное число линий, значит, из
            а везде должно быть четное число (зашел-вышел
```

```

    </div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">18</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Остовное дерево</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Остовное дерево</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Остовное дерево — это подграф, который является деревом и содержит все вершины графа.</p>
        <p class="text-xl font-mono text-white">
          (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Прокладка интернет-кабелей.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Начиная с самых дешевых дорог в стране. Он строит единую сеть.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },

```

```

    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-4">
            изкому племени для объединения. К вечеру пле
            ства шлют гонцов к ближайшему чужому царству.
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "string-kmp",
    title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
    type: "html",
    description: "",
    category: "Строки",
  }

```

```

        if (s[i] == s[j]) j++;
        pi[i] = j;
    }</pre>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-violet-500">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        щегося с i.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Копипаста
          </div>
          <p class="text-slate-300 text-sm mb-4">
            в тексте кричит: "Эй! Начиная с этой буквы
            т "окно" [L, R] самой дальней найденной коп

```

```

        <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
    <!-- Аналогия 1 -->
    <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
        </div>
        <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
    </div>
    <!-- Аналогия 2 -->
    <div class="bg-slate-900 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
            Аналогия 2: Машина-переводчик
        </div>
        <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведенн
аче B, то B – это <b>NP-Полная</b> (сосредо
    </div>
</div>
</div>
</section>`,
    },
];

```

ФАЙЛ 18/82: src/data/content_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМ

```
export interface Chapter {
```



```

        шел-вошёл-вышел).
        <br>* <span class="text-yellow-400 font
        финиш).
    </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4" >
    <div class="bg-slate-800 p-6 rounded-lg border-2 border-
        <div class="absolute -top-3 left-4 bg-slate-800
        order-green-500 shadow-md">
            Путь: Нормальная прогулка
        </div>
        <p class="text-slate-300 text-sm mb-4">
            Ты вышел из дома <strong>(нечётная)</strong>.
            талый пришёл в бар <strong>(вторая нечётная)</strong>.
            Домой вернуться не смог, потому что
        </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2 border-
        <div class="absolute -top-3 left-4 bg-rose-500
        der-rose-500 shadow-md bg-slate-950">
            Цикл: Гамильтонов синдром (болезнь)
        </div>
        <p class="text-slate-300 text-sm mb-4">
            <strong>Гамильтон – это болезнь.</strong>
            ng> ровно раз и вернуться домой.
            Ты обходишь все бары (вершины) ровно раз.
            Главное – зайти и выйти, не заходя повторно.
            Никто не знает, как решать быстро.
            <br><span class="text-xs text-rose-500">
                лноту).</span>
            </p>
        </div>
    </div>

</div>

<details class="bg-slate-900/40 rounded-lg border-2 border-
    <summary class="p-4 font-bold text-slate-400">

```

```
<p><strong class="text-white">low[v
<div class="p-3 bg-slate-950 rounded
  <span class="text-rose-400 font
  <p class="text-xl font-bold tex
  <p class="text-xs text-slate-400
</div>
</div>
</div>

<p class="text-slate-300 text-sm">
  Если из сына и и всех его потомков <strong>
  Единственная ниточка. Порвётся – граф р
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-emerald-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-slate-900 p-2 border-rose-500 shadow-md">
      Мост: Единственная веревка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь, что ты альпинист. Ты спустился в пещеру.
      Из пещеры и нет других выходов – только вверх.
      Если верёвка (ребро v-u) оборвётся – ты погиб.
      Если бы из пещеры и был чёрный ход – ты бы ушёл.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-rose-400 p-2 border-rose-500 shadow-md">
      Аналогия: Кровеносный сосуд
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Граф – это кровеносная система. Ребра (сосуды) – это не мост, живём.
      Если же перерезать единственную артерию, то организм погибнет.
    </p>
  </div>
</div>
```

```

        // ребро дерева DFS
        dfs(to, v); // рекурсивный вызов
        low[v] = min(low[v], low[to]);

        if (low[to] > tin[v]) {
            // ЭТО МОСТ!
            // bridge_list.push_back({v, to});
        }
    }
}
}
}

```

```

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}

```

```

</div>
<p class="text-xs text-slate-400">Запись
</div>
</details>
</div>
</section>`,
},
{
    id: "planarity-euler-formula",
    title: "Планарность: K5, K3,3 и формула Эйлера",
    type: "html",
    content: `<section id="planarity-euler-formula" class="flex items-center mb-6">
        <div class="flex items-center mb-6">
            <span class="bg-blue-600 text-white px-4 py-1 rounded">
                <h2 class="text-3xl font-bold text-white">Планарность
        </div>
    `
}

```

```

<div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
  K3,3: 3 дома и 3 колодца
</div>
<p class="text-slate-300 text-sm mb-4">
  Три дома хотят соединиться с тремя колодцами.
  Нарисовать без пересечений невозможно.
  У него class="text-white">V=6, E=9, F=5</code>
  class="text-white">9 ≤ 12</code> – проходит, но он не планарен.
  Почему? Потому что у двудольного графа должно быть четное число вершин.
  Отсюда более строгое ограничение: class="text-white">9 > 8</code>.
</p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">Доказательство непланарности K5 (Скрыть)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400">Доказательство непланарности K5
    <p>
      Пусть K5 планарен.  $V = 5, E = 10$ .  

       $F = E - V + 2 = 10 - 5 + 2 = 7$   

      Каждая грань ограничена минимумом 3 ребрами.  

      Сумма длин граней  $= 2E = 20$ .  

      Отсюда:  $2E \geq 3F \Rightarrow 20 \geq 21$  – противоречие.  

      Значит, K5 не может быть планарным.
    
  
```

```
<p class="text-slate-300 text-sm mb-4">
    Представь город, где два района сое
    её перекроют (удалят вершину) – районы буд
g>хрупкость</strong> системы.
</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
    <div class="absolute -top-3 left-4 bg-emera
er border-emerald-500 shadow-md">
    Телеком: Центральный свитч
</div>
<p class="text-slate-300 text-sm mb-4">
    У тебя несколько компьютеров в одно
абелем (мостом). Сами свитчи – это точки со
</p>
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
    <summary class="p-4 font-bold text-slate-400
order-slate-700/50">
    Душный мод: Код и корень DFS (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300 space-
    <p class="text-blue-400 font-bold">Особ
    <p>
        Для стартовой вершины обхода правил
        то выше неё прыгать некуда). Поэтому для не
        у него <strong>больше 1 независимого ребенка
    </p>
    <div class="bg-slate-950 p-4 rounded-lg
        <pre class="text-xs font-mono text-
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
```

```
    },  
  ];  
}
```

ФАЙЛ 19/82: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРок: 51 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../types";  
import { graphChapters } from "../graphContent";  
import { additionalTickets } from "../additionalTickets";  
import { tickets14to20 } from "../tickets/ticket14_20";  
import { tickets21to24 } from "../tickets/ticket21_24";  
import { tickets6to13 } from "../tickets/ticket6_13";  
import { chapters as newChapters } from "../content_temp";
```

```
// We want to combine all of them, but overwrite 7, 11,  
const merged = [  
  ...graphChapters,  
  ...additionalTickets,  
  ...tickets6to13,  
  ...tickets14to20,  
  ...tickets21to24  
].filter(c => c && c.id);
```

```
// Remove old 7, 11, 12, 13: graph-planar-colors, graph-bridges  
const toRemove = ["graph-planar-colors", "graph-bridges"];
```

```
const dedup = new Map<string, Chapter>();  
merged.forEach(c => {  
  if (!toRemove.includes(c.id)) {  
    // We prefer the newer versions (like tickets14_20)  
    dedup.set(c.id, c);  
  }  
});
```

```
// Add the new ones
```

```

    id: string;
    /** Варианты написания в тексте (регистр не важен). */
    labels: string[];
    title: string;
    /** Объяснение «на пальцах» – 1–2 фразы. */
    short: string;
    /** Строгая формулировка / что важно не забыть. */
    formal?: string;
    complexity?: string;
    /** Какая мини-анимация показывается в карточке. */
    demo?: DemoKind;
    /** id полного симулятора (см. vizRegistry) – кнопка
    simulator?: string;
}

```

```

export const GLOSSARY: GlossaryEntry[] = [
  {
    id: "bfs",
    labels: ["BFS", "обход в ширину", "поиск в ширину",
    title: "BFS – обход в ширину",
    short:
      "Волна от источника: сначала все соседи, потом со
    formal:
      "Очередь (FIFO). Кладём старт, достаём вершину –
        у рёбер.",
    complexity: "O(V + E)",
    demo: "bfs",
    simulator: "queue-bfs",
  },
  {
    id: "dfs",
    labels: ["DFS", "обход в глубину", "поиск в глубину",
    title: "DFS – обход в глубину",
    short:
      "Прёшь вперёд до упора, упёрся – откатываешься на
    formal:
      "Стек (или рекурсия). Из DFS растут: времена вход
    complexity: "O(V + E)",
  },
]

```

```

    id: "queue",
    labels: ["очередь", "очереди", "очередью", "FIFO"],
    title: "Очередь (FIFO)",
    short: "Очередь в столовой: кто первым встал, тот первый",
    formal: "push_back / pop_front за  $O(1)$ . Основа BFS",
    demo: "queue",
    simulator: "queue-bfs",
},
{
    id: "dsu",
    labels: ["DSU", "CHM", "система непересекающихся множеств"],
    title: "DSU – система непересекающихся множеств",
    short:
        "«Кто твой староста?» Каждый элемент знает своего старосту",
    formal:
        "find со сжатием путей + union по рангу/размеру. Обратная функция Аккермана",
    complexity: "почти  $O(1)$  – обратная функция Аккермана",
    demo: "dsu",
    simulator: "mst-kruskal",
},
{
    id: "trie",
    labels: ["бор", "бора", "боре", "бору", "бором", "trie"],
    title: "Бор (trie, префиксное дерево)",
    short:
        "Дерево, где путь от корня по буквам и есть слово",
    formal: "Из корня по символам; в вершине хранится ф. слово",
    complexity: "поиск слова –  $O(|\text{слова}|)$ ",
    demo: "trie",
    simulator: "aho-corasick",
},
{
    id: "bfs-on-trie",
    labels: [
        "обход дерева",
        "обход бора",
        "обход в ширину по бору",
        "какой-то обход",
    ],

```



```

    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
        ,
    formal: "if  $d[v] > d[u] + w(u,v) \rightarrow d[v] = d[u] + w(u,v)$ 
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "префикс"],
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от начала до i»
    formal: " $P[0]=0, P[i]=P[i-1]+a[i-1]$ . Сумма  $a[l..r] = P[r]-P[l-1]$ .",
    complexity: "препроцессинг  $O(n)$ , запрос  $O(1)$ ",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "sparse-table",
    labels: [
        "разреженная таблица", "разреженная таблица", "sparse-table",
        "разреженная", "двоичные подъёмы", "двоичный подъём",
    ],
    title: "Разреженная таблица (метод двоичных подъёмов)",
    short:
        "Заранее считаем ответ для всех отрезков длины 1, 2, 4, ...,
        M.",
    formal: " $ST[i][j] = op(ST[i][j-1], ST[i+2^{(j-1)}][j-1])$ ",
    complexity: "препроцессинг  $O(n \log n)$ , запрос  $O(1)$ ",
    demo: "sparse-table",

```

```

        "остовное дерево", "остовного дерева", "остовное"
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная
formal: "V-1 ребро, никаких циклов. Краскал (жадно н
        но).",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизир
        "амортизированное", "амортизированного", "амортизир
    ],
    title: "Амортизированная сложность",
    short:
        "Средняя цена операции «по абонементу»: одна опер
formal: "Метод потенциалов: дорогая операция оплачи
        ассив.",
    demo: "amortized",
},
{
    id: "np",
    labels: [
        "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
        "класс P", "полином", "полиномиальное",
    ],
    title: "Класс NP и NP-полнота",
    short:
        "NP — «решение проверить легко, найти трудно» (су
        ые в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полн
    demo: "np",
},
{

```

```

    },
    {
      id: "z-function",
      labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функция"],
      title: "Z-функция",
      short:
        "Для каждой позиции: сколько символов подряд, начинающихся с этой буквы.",
      formal:
        " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i..n]$ ",
      complexity: " $O(n)$ ",
      demo: "prefix-function",
      simulator: "string-z-func",
    },
    {
      id: "dp",
      labels: [
        "динамическое программирование", "динамики", "динамическое",
        "мемоизация", "мемоизацию", "подзадач", "подзадачи"
      ],
      title: "Динамическое программирование",
      short:
        "Решаем задачу через ответы на её меньшие копии и сохраняем их.",
      formal:
        "Нужны: оптимальная подструктура + перекрывающиеся подзадачи.",
      demo: "dp",
      simulator: "dynamic-programming",
    },
    {
      id: "shortest-path",
      labels: ["кратчайший путь", "кратчайшие пути", "кратчайшие"],
      title: "Кратчайший путь",
      short:
        "Самый дешёвый маршрут из A в B. «Дешёвый» – по стоимости.",
      formal:
        "Дейкстра – неотрицательные веса,  $O(E \log V)$ . Форесман – отрицательные веса,  $O(V^3)$ .",
    }
  ]
}

```



```

    demo: "floyd",
    simulator: "floyd",
},
{
    id: "prim",
    labels: ["Прима", "Прим", "Prim"],
    title: "Алгоритм Прима",
    short:
        "Остов растёт как плесень из одной точки: на кажды
    formal:
        "Множество посещённых + куча рёбер на границе. До
        усок.",
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-prim",
},
{
    id: "boruvka",
    labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
    title: "Алгоритм Борувки",
    short:
        "Все компоненты одновременно шлют «гонца» к ближай
    formal:
        "Фаза: для каждой компоненты ищем минимальное исх
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-boruvka",
},
{
    id: "kruskal",
    labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
    title: "Алгоритм Краскала",
    short:
        "Сортируем все рёбра по весу и берём подряд те, ч
    formal:
        "Сортировка O(E log E) + E операций union/find. C
    complexity: "O(E log E)",
    demo: "mst",

```

```

        порядковым номером.",
    formal:
        "sort + unique + binary search: значение → индекс",
    complexity: "O(n log n)",
    demo: "coord-compress",
},
{
    id: "treap",
    labels: ["Treap", "декартово дерево", "декартова де",
    title: "Treap = Tree + Heap",
    short:
        "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
    formal:
        "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить две бинарные деревья через них. Ожидаемая высота O(log n).",
    complexity: "ожидаемо O(log n)",
    demo: "heap",
},
{
    id: "bst",
    labels: ["бинарное дерево поиска", "дерево поиска",
    title: "Бинарное дерево поиска",
    short:
        "Слева всё меньше корня, справа — всё больше. Поиск по ключу.",
    formal:
        "Без балансировки дерево может вырождаться в список.",
    complexity: "O(высоты): от O(log n) до O(n)",
    demo: "segment-tree",
},
{
    id: "inclusion-exclusion",
    labels: [
        "включений-исключений", "включения-исключения", "принцип",
        "вырезание прямоугольников",
    ],
    title: "Включения-исключения на префиксных суммах",

```

```

        (Z-функция) и где встречаются слова словаря (бордер).
formal:
    "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция."
demo: "prefix-function",
simulator: "string-kmp",
},
{
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "повороты"],
    title: "Поворот (rotation) в дереве",
    short:
        "Локальная перевеска двух узлов: ребёнок встаёт на место  

        дившемся месту. Порядок ключей при этом не ломается."
formal:
    "Правый поворот вокруг p:  $x = p.left$ ;  $p.left = x$ .  

    Аналогично для левого поворота и Zig-Zag.",
complexity: "O(1)",
demo: "zig",
simulator: "splay-rotations",
},
{
    id: "zig",
    labels: ["Zig", "Zag", "зиг"],
    title: "Zig — одиночный поворот",
    short:
        "Самый простой случай: родитель x уже сидит в корне.  

        заканчивается, если глубина была нечётной.",
formal: "Применяется ровно один раз за всю операцию",
demo: "zig",
simulator: "splay-rotations",
},
{
    id: "zig-zig",
    labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
    title: "Zig-Zig — x и родитель с одной стороны",
    short:
        "Дерево вытянулось в «бамбук»:  $g \rightarrow p \rightarrow x$  идут в одну  

        сторону (p, x). Именно это вдвое сокращает глубину всего

```



```

</section>`
  },
  {
    id: "bellman-ford",
    title: "Билет 15. Форд-Беллман",
    type: "html",
    category: "Графы",
    content: `
```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
        евле долететь из Москвы в Париж, если мы сд
        <div id="slot-chapter-image-floyd" class="w-full h-40">
        </div>

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4
            border-rose-500 shadow-md">
                Время работы
            </div>
            <p class="text-slate-300 text-sm mb-6">
            три вложенных цикла. Сложность  $O(V^3)$ . Если
            </div>
        </div>

        <div id="slot-floyd-viz" class="mt-8"></div>
    </div>
</div>
</section>`
    },
    {
        id: "aho-corasick",
        title: "Билет 23. Алгоритм Ахо-Корасик",
        type: "html",
        category: "Строки",
        content: `<section id="aho-corasick" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-l">
        <h3 class="text-xl font-bold text-blue-400 mb-4">
        </div>
        <div class="bg-slate-900 p-4 rounded-lg mb-6 text

```

```

        <p>Каждая вершина имеет таблицу переходов <code>
        ссылку <code>term_link</code> (ближайшая мен
        <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
    map<char, int> go;          // Предвычисленные пер
    int link = 0;              // Суффиксная ссылка: куда
    int term_link = 0;         // Терминальная ссылка: матр
    bool is_terminal = false;
    vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {
        nodes[child].link = 0;
        q.push(child);
    }

    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto const& [ch, child] : nodes[v].trie) {
            // Суффиксная ссылка ребёнка – это переход
            nodes[child].link = nodes[nodes[v].link].go

            // Если суффиксная ссылка сама является сло
            // Иначе наследуем terminal_link
            if (nodes[nodes[child].link].is_terminal) {
                nodes[child].term_link = nodes[child].l
            } else {
                nodes[child].term_link = nodes[nodes[ch
            }

            q.push(child);
        }
    }
}

```

```

    question: string;
    options: string[];
    correctIndex: number;
    explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
  "euler-path-vs-cycle": [
    {
      question: "В связном графе ровно 2 вершины с нечётными степенями",
      options: [
        "Есть эйлеров цикл (вернусь домой!)",
        "Есть эйлеров путь (из одной нечётной в другую)",
        "Ни пути, ни цикла нет — это полный провал"
      ],
      correctIndex: 1,
      explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше 2 — эйлерова цикла нет."
    },
    {
      question: "Почему гамильтонов цикл — это NP-полная задача?",
      options: [
        "Потому что Гамильтон — болезнь, и лечить её тяжело",
        "Потому что для эйлерова цикла есть простой критерий",
        "Потому что Эйлер был умнее Гамильтона"
      ],
      correctIndex: 1,
      explanation: "Для эйлерова цикла работает критерий степени вершин. Для гамильтонова цикла нет простого критерия. Это NP-полная задача, которую не известно, как решить за полиномиальное время."
    },
    {
      question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
      options: [
        "Всё ещё можно пройти по всем рёбрам ровно раз",
        "Такого графа не существует",
        "Эйлерова пути нет — нужно удалять лишние рёбра"
      ],
      correctIndex: 2,
    }
  ]

```

```

{
  question: "Планарный граф имеет  $V=6$ ,  $E=12$ . Возможны ли такие графы?",
  options: [
    "Да, если это двудольный граф",
    "Нет, нарушается неравенство  $E \leq 3V - 6$ ",
    "Да, если нет треугольных граней"
  ],
  correctIndex: 1,
  explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$  – на грани возможности. Но если бы было так, то такой граф был бы планарным. Так что такой граф непланарен."
},
{
  question: "Сколько граней у планарного графа с  $V=7$  и  $E=10$ ?",
  options: [
    "3 грани",
    "5 граней",
    "10 граней"
  ],
  correctIndex: 1,
  explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$ . Не 10, а 5."
},
{
  question: "Почему  $K_5$  непланарен?",
  options: [
    "Потому что  $10 > 3*5 - 6 = 9$  – нарушение",
    "Потому что Эйлер был против",
    "Потому что 5 вершин – несчастливое число"
  ],
  correctIndex: 0,
  explanation: "У  $K_5$ :  $V=5$ ,  $E=10$ ,  $3V - 6 = 9$ .  $10 > 9$  – нарушение."
}
]
};

```

```

        <p class="text-slate-300 text-sm mb-4">
            ы рассылать 10 000 писем, он пишет одно письмо
            счетах сотрудников только тогда, когда соотв
            женное обновление).</p>
        </div>

        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 border-rose-500 shadow-md">
                Аналогия 2: Матрешки-коробки
            </div>
            <p class="text-slate-300 text-sm mb-4">
                ше, и так далее. Если нам нужно покрасить по
                Внутри всё синее!" на большую коробку. Опус
            </div>
        </div>

        <details class="bg-slate-900/40 rounded-lg">
            <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
                Строгое доказательство / Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
function push(node) {
    if (lazy[node] !== 0) {
        lazy[2 * node] += lazy[node];
        tree[2 * node] += lazy[node];
        lazy[2 * node + 1] += lazy[node];
        tree[2 * node + 1] += lazy[node];
        lazy[node] = 0;
    }
}</pre>
            </div>
        </details>
    </div>
</div>
</section>

```



```

} </pre>
    </div>
  </details>
</div>
</div>
</section>`
  },
  {
    id: "splay-tree",
    title: "4. Splay-дерево",
    type: "html",
    description: "Дерево поиска, которое самобалансируется",
    category: "Продвинутые структуры",
    content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: VIP-лифт
          </div>
          <p class="text-slate-300 text-sm mb-4">
            вится корнем дерева (VIP-статус). Если ты ч
            ка почти не требуется времени!</p>
          </div>
        <div class="bg-slate-900 p-6 rounded-lg"

```



```

        ска. За счет сдвига этого листа в корень, д
    /p>
    </div>

    <div>
        <strong class="text-white block">
            <p>Если агент постоянно переключо
                находящимися на разных концах дерева, его м
                ащая дерево в вытянутый бамбук для другого.
            . Постоянное свинг-переключение превратит пр
        </div>

    <div>
        <strong class="text-white block">
            <p>Хотя один поиск может стоить
                емах обладают прекрасным свойством: они не м
                по пути. "Палочное" дерево прессуется в сб
                осов.</p>
        </div>
    </div>
</div>
</div>
</div>
</section>`
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сжатие суммы подматрицы за  $O(1)$ ",
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
            <h2 class="text-2xl sm:text-3xl font-bold text-
        </div>
    <div class="space-y-8">

```

```

        content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Позитивное планирование
          </div>
          <p class="text-slate-300 text-sm mb-4">
            (нельзя поехать и заработать). Он всегда в
          </div>
          <!-- Аналогия 2 -->
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
              Ограничения
            </div>
            <p class="text-slate-300 text-sm mb-4">
              дный и не умеет пересматривать уже "зафиксир
            </div>
          </div>
          <div id="slot-dijkstra-viz" class="mt-8"></div>
        </div>
      </div>
    </section>`
  },

```

```

    </div>
</section>`
  },
  {
    id: "floyd",
    title: "16. Графы. Поиск кратчайшего пути. Флойд",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>

      <div class="bg-slate-800 p-6 rounded-lg border">
        <div class="absolute -top-3 left-4 bg-slate-700/50
          border-emerald-500 shadow-md">
          Суть фокуса (По шагам)
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Главный герой – <b>внешний цикл</b>
          шаг <code class="bg-slate-900 text-pink-400">
          шу себе проходить через вершину k?</i>
        </p>
        <ul class="text-sm text-slate-300 space-y-2">
          <li><b>Шаг k=0:</b> Ты знаешь только
          <li><b>Шаг k=1:</b> Разрешаем пересадки
          е> через путь <code class="text-indigo-300">
          <li><b>Шаг k=2:</b> Разрешаем пересадки
          внутри этих путей уже могут быть пересадки ч

```

```

        <li>Добавляем <b>фиктивную верш
        <li>Запускаем от неё <b>медленно
        <b>«потенциалы»</b> <code class="text-pink-400"
        <li>Меняем старые веса: новое р
        <li><b>Магия!</b> Все веса тепер
        ренные потенциалы сокращаются).</li>
        <li>Теперь смело запускаем <b>быстро
    </ol>
  </div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</div>
</section>`
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            rder border-emerald-500 shadow-md">

```

```
        Сеть растёт только из одного связного куска
      </div>
    </div>
  </div>
</section>`
},
{
  id: "mst-boruvka",
  title: "20. Графы. Остовное дерево. Борувка",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-4">
            изкому племени для объединения. К вечеру пле
            ства шлют гонцов к ближайшему чужому царству.
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`
}
```

```

rder border-emerald-500 shadow-md">
    ♂ Аналогия 1: Поиск без отка
</div>
<p class="text-slate-300 text-sm mb
    Мы идём по строке слева направо. К
строки СЕЙЧАС закончился под моим пальцем?»
    Представь, ты ищешь в тексте слова
x</code>. Тупой алгоритм начнет искать заново
</code>, а это начало нашего слова! Не надо
</p>
</div>

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg
    <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
    Аналогия 2: LLM стриминг
</div>
<p class="text-slate-300 text-sm mb
    Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
</p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Пример вычисления (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
    <p>Тот же пример: <b>abcabcd</b></p>
    <ul class="list-disc pl-5 space-y-2
        <li><b>a</b></li>: Префиксов нет. <co
        <li><b>ab</b></li>: Из начала ("a")
>
        <li><b>abc</b></li>: Опять мимо. <co

```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
      <p class="text-xl font-mono text-white">
        са 0) и её суффиксом, начинающимся с i.</p>
    </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

Аналогия 1: Линейка-шаблон

```
</div>
<p class="text-slate-300 text-sm mb-4">
  Тут мы берём всю строку и «прикидываю
  я начну читать строку отсюда, насколько длин
  Это самый быстрый способ найти индекс
  >, считаешь Z-функцию, и там, где <code>Z[i]
  </p>
</div>
```

<!-- Аналогия 2 -->

```
<div class="bg-slate-900 p-6 rounded-lg">
  <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
```

Аналогия 2: LLM Самокопираст

```
</div>
<p class="text-slate-300 text-sm mb-4">
  В LLM Z-функция может применяться
  [i]</code> (где <i>i</i> указывает назад на
  сама себя.
```

```

        </div>
    </div>
</section>`
    },
    {
        id: "aho-corasick",
        title: "23. Алгоритм Ахо-Корасик",
        type: "html",
        category: "Строки",
        content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-l">
      <h3 class="text-xl font-bold text-blue-400 mb-4">

      <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
        <p class="text-sm text-blue-300 italic mb-2">0с
        <p class="text-lg font-mono text-white">Бор (Тр
      </div>
      <p class="text-slate-300 text-sm">Позволяет найти
        го один проход.</p>
    </div>

    <!-- Аналогии -->
    <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg border bo">
        <div class="absolute -top-3 left-4 bg-slate-700
          emerald-500 shadow-md">
          Аналогия 1: Паутина (Бор)
        </div>
        <p class="text-slate-300 text-sm mb-4">Представ
          ет — мы падаем по <b>суффиксной ссылке</b> ("
        </div>

```



```

{
  id: "complexity-classes",
  title: "24. Классы сложности, сведение задач",
  type: "html",
  category: "Теория сложности",
  content: `
<section id="complexity-classes" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-purple-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">
        <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
            order border-emerald-500 shadow-md">
              Аналогия 1: Судoku (P vs NP)
            </div>
            <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
          </div>
          <!-- Аналогия 2 -->
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
              order border-purple-500 shadow-md">
                Аналогия 2: Машина-переводчик
              </div>
              <p class="text-slate-300 text-sm mb-4">
ь отличный переводчик на английский (Сведен

```

```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия: Матрица = Таблица
    </div>
    <p class="text-slate-300 text-sm mb
гупо, но проверка "знает ли Вася Петю" мги
    </div>
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
        Аналогия: Списки = Контакты
      </div>
      <p class="text-slate-300 text-sm mb
т. Оптимально для почти всех задач.</p>
    </div>
  </div>
</div>

```

```

{ /* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-indigo-400"

  <div class="grid grid-cols-1 xl:grid-cols-2"
    <!-- Аналогия DFS -->
    <div class="bg-slate-800 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
        ♂ DFS (Поиск в глубину) = Жадный
      </div>
      <p class="text-slate-300 text-sm mb
        <b>Что делает:</b> Жадный алгоритм
        ую вершину (например, минимальную по номеру
        (рекурсивно) и пробует другой путь.
      </p>
      <ul class="text-xs text-indigo-300"

```



```

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
  <h3 class="text-xl font-bold text-emerald-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          ество компонент.</p>
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
              Аналогия 1: Острова
            </div>
            <p class="text-slate-300 text-sm mb-4">
              карту – остались ли серые (неисследованные)
              ь через океан – столько и компонент.</p>
            </div>
          </div>
        </div>
      </div>
    </section>`,
    },
    {
      id: "top-sort",
      title: "9. Графы. Топологическая сортировка",
      type: "html",
      description: "Разложение задач по порядку выполнения",
      category: "Графы",
      content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
          <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

        visited[v] = true;
        for (int to : adj[v]) {
            if (!visited[to]) {
                dfs(to);
            }
        }
        ans.push_back(v); // Добавляем в момент ВЫХОДА (post-order)
    }
}

```

```

void topological_sort(int n) {
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
    reverse(ans.begin(), ans.end()); // Разворачиваем с
}

```

```

</div>
</div>
</details>
</div>
</div>
</section>`,
},
{
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая.",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">10. Графы. Компоненты сильной связности (SCC)</span>
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)</h2>
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
            <h3 class="text-xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)</h3>

```

```

        <b>SCC (Билет 10)</b> склеивает
    </br><br>
        <b>Точки сочленения (Билет 12)</b>
    ость</strong> монолита. Вырвешь точку сочле
    </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
    -b border-slate-700/50">
        Душный мод: Код Алгоритм Косарайю
    </summary>
    <div class="p-5 text-sm text-slate-300
        <p class="text-emerald-400 font-bol
        <ol class="list-decimal list-inside
            <li>Запускаем DFS на исходном г
li>
            <li>Разворачиваем этот список (
            <li>Переворачиваем все ребра в
            <li>Идем по <code>order</code>:
!</li>
        </ol>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "graph-bridges",
        title: "11. Графы. Мосты",
        type: "html",
        description: "Рёбра, удаление которых расхреничивае
        category: "Графы. Продвинутые",
        content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
      <div class="bg-slate-900 p-4 rounded-lg mb-4"
        <p class="text-lg text-rose-300 italic"
          <div class="text-left font-mono text-sm"
            <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
          <div class="p-3 bg-slate-950 rounded"
            <span class="text-rose-400 font"
              <p class="text-xl font-bold tex
                <p class="text-xs text-slate-400
              </p>
            </div>
          </div>
        </div>
      </div>
    </div>

    <p class="text-slate-300 text-sm">
      <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong>
ег - это тупик со степенью 1).
    </p>
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2 gap"
    <div class="bg-slate-800 p-6 rounded-lg bor"
      <div class="absolute -top-3 left-4 bg-s
rder-rose-500 shadow-md">
        Аналогия: Главный перекресток
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Представь город, где два района сое
её перекроют (удалят вершину) – районы буд
g>хрупкость</strong> системы.
      </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg bor

```

```

        IS_CUTPOINT(v);
        ++children;
    }
}
if (p == -1 && children > 1)
    IS_CUTPOINT(v);
}
        </div>
    </div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">
    <h4 class="text-lg font-bold text-indigo-400">
    <p class="text-slate-300 text-sm mb-4">
        Это глубокий дуальный мост между ориентир
    </p>
    <ul class="list-disc list-inside text-sm text-slate-300">
        <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
        <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
        <li><strong>Как точка сочленения рушит :
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
    </ul>
    </div>
</div>
</section>`,
    },
    {
        id: "graph-euler",
        title: "13. Графы. Эйлеров цикл",
        type: "html",
        description: "Пройти по всем ребрам ровно 1 раз.",
        category: "Графы",
        content: `

```



```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";

interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
}

interface Edge {
  u: number;
  v: number;
}

const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
  { id: 1, x: 100, y: 150, label: "1" },
  { id: 2, x: 250, y: 250, label: "2" },
  { id: 3, x: 400, y: 150, label: "3" },
  { id: 4, x: 550, y: 150, label: "4" },
  { id: 5, x: 700, y: 50, label: "5" },
  { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
  { u: 3, v: 4 }, // 3 and 4 are articulation points
  { u: 4, v: 5 },
  { u: 5, v: 6 },
  { u: 6, v: 4 },
];

// Adjacency list
```

```

        low: [...currentLow],
    });
};

recordStep("Начало DFS (Тарьян) для поиска точек со

const dfs = (v: number, p: number = -1) => {
    visited.add(v);
    currentTin[v] = currentLow[v] = timer++;
    let children = 0;

    recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

    for (const to of adj[v]) {
        if (to === p) continue;

        if (visited.has(to)) {
            currentLow[v] = Math.min(currentLow[v], curren
            recordStep(
                `Обратное ребро ${v}-${to}. Обновляем low[${v}
                v,
            );
        } else {
            children++;
            dfs(to, v);
            currentLow[v] = Math.min(currentLow[v], curren

            recordStep(
                `Возврат в ${v} из ${to}. Обновляем low[${v}
                v,
            );

            if (currentLow[to] >= currentTin[v] && p !==
                ap.add(v);
                recordStep(
                    `Найдена точка сочленения: вершина ${v}
                    v,
                );

```

```

    let interval: NodeJS.Timeout;
    if (isPlaying && currentStepIndex < steps.length - 1) {
      interval = setInterval(() => {
        setCurrentStepIndex((prev) => prev + 1);
      }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearInterval(interval);
  }, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIndex(0);
};

return (
  <div className="bg-slate-900 rounded-xl border border-rose-400" >
    <div className="bg-slate-800 p-4 border-b border-rose-400" >
      <h3 className="font-bold text-white flex items-center" >
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>
      <div className="flex items-center gap-2 bg-slate-900" >
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-2 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
          {isPlaying ? "Пауза " : "Старт "}
        </button>
      </div>
    </div>
  )

```

```

{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id
  const isVisited = currentStep.visited.has(node.id)
  const isAp = currentStep.ap.has(node.id);

  const fill = isCurrent
    ? "#3b82f6"
    : isAp
    ? "#e11d48"
    : isVisited
    ? "#10b981"
    : "#1e293b";
  const stroke = isCurrent
    ? "#60a5fa"
    : isAp
    ? "#f43f5e"
    : isVisited
    ? "#34d399"
    : "#334155";
  const r = isAp ? 24 : 20;

  return (
    <g key={node.id}>
      <circle
        cx={node.x}
        cy={node.y}
        r={r}
        fill={fill}
        stroke={stroke}
        strokeWidth="3"
        className="transition-all duration-300"
      />
      <text
        x={node.x}
        y={node.y}
        dy=".3em"
        textAnchor="middle"
        fill="white"
      />
    </g>
  )
})}

```

```
    </svg>
  </div>

  <div className="bg-slate-950 p-6 border-t lg:bo
    <h4 className="text-sm font-bold text-slate-4
      Статус алгоритма
    </h4>

    <div className="bg-slate-900 border border-sl
      <p className="text-white font-medium min-h-
        {currentStep.desc}
      </p>
    </div>

    <div className="space-y-4 flex-1 overflow-y-a
      <div className="mb-4">
        <h5 className="text-xs text-slate-500 fon
        <div className="flex items-center gap-2 t
          <div className="w-3 h-3 rounded-full bg
            Точка сочленения
          </div>
        <div className="flex items-center gap-2 t
          <div className="w-3 h-3 rounded-full bg
            Посещена
          </div>
        <div className="flex items-center gap-2 t
          <div className="w-3 h-3 rounded-full bg
            Текущая
          </div>
        <div className="flex items-center gap-2 t
          <div className="w-8 h-1 bg-rose-500 bor
            Мост
          </div>
        </div>
      </div>

      <div>
        <h5 className="text-xs text-slate-500 fon
          ЛОГ ДЕЙСТВИЙ:
```

```
import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
  previous: { [key: string]: string | null };
  log: string;
  hasNegativeCycle?: boolean;
  activeCodeLine: number;
}

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
    ];
    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'E', w: -1 }];
    }
    return defaultEdges;
  };
}
```

```
simulationSteps.push({
  iteration: 0, checkingEdge: null,
  distances: { ...dist }, previous: { ...prev },
  log: '    Фура заведена. Начинаем с Базы (S). Расст
  activeCodeLine: 2,
});
```

```
const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
  let anyUpdate = false;
  for (const edge of edges) {
    const uDist = dist[edge.u];
    const vDist = dist[edge.v];

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
      dist = { ...dist, [edge.v]: uDist + edge.w };
      prev = { ...prev, [edge.v]: edge.u };
      updated = true;
      anyUpdate = true;
    }
  }
}
```

```
simulationSteps.push({
  iteration: iter, checkingEdge: { u: edge.u, v
  distances: { ...dist }, previous: { ...prev }
  log: `Итерация ${iter}: Проверяем ${edge.u} →
    updated ? ` Релаксация успешна! dist[${edge
  }`,
  activeCodeLine: updated ? 6 : 5,
});
}
```

```
if (!anyUpdate && iter < vCount - 1) {
  simulationSteps.push({
    iteration: iter, checkingEdge: null, distance
    log: `    Ранний выход после итерации ${iter}.
    activeCodeLine: 10
  });
}
```

```

        timer = setTimeout(() => setCurrentStepIndex((p)
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border-2
    <div className="flex flex-wrap items-center justify-center
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-blue-600 text-white
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-blue-300">Полный
          </div>
        </div>

      <div className="flex items-center gap-3">
        <div className="bg-slate-900 p-1 rounded-lg border
          <button onClick={() => setHasCycle(false)} className="
            bg-blue-600 text-white' : 'text-slate-400'
          <button onClick={() => setHasCycle(true)} className="
            r gap-1 ${hasCycle ? 'bg-rose-600 text-white'
          </div>
        <button onClick={() => { setIsPlaying(false);
          g-slate-600 text-white px-3 py-2 rounded-lg
        <button onClick={() => setIsPlaying(!isPlaying)}
          sition-colors text-white ${isPlaying ? 'bg-
        <button onClick={() => { setIsPlaying(false);
          ; }} className="flex items-center gap-1 bg-l
          s">⏮️ </button>
        </div>

```



```

        let marker = 'url(#arrow-bf-normal)';
        if (isChecking) { strokeColor = '#f59e0b'
        else if (currentStep.hasNegativeCycle &&

        return (
            <g key={idx}>
                <line x1={uNode.x} y1={uNode.y} x2={vNode.x}
                    strokeDash={currentStep.hasNegativeCycle && isCyclePart} ? 4 : 2}
                    strokeColor={isChecking ? '#f59e0b' : '#f43f5e'} />
                <circle cx={({uNode.x + vNode.x} / 2)} cy={
                    (uNode.y + vNode.y) / 2} r={10} strokeDash={
                    isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#7dd3fc'}
                    strokeColor={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#7dd3fc'}
                    strokeWidth={2} />
                <text x={({uNode.x + vNode.x} / 2)} y={
                    (uNode.y + vNode.y) / 2} text={currentStep.hasNegativeCycle &&
                    isCyclePart ? '4,4' : 'none'} />
            </g>
        );
    }
    }
    }

    {nodes.map((node) => {
        const dist = currentStep.distances[node.id];
        const isCheckingNode = currentStep.checkingNode === node.id;

        return (
            <g key={node.id} className="transition-appear">
                <circle cx={node.x} cy={node.y} r={10} strokeDash={
                    isCheckingNode ? '#7dd3fc' : '#64748b'} strokeWidth={2} />
                <text x={node.x} y={node.y + 5} fill={isCheckingNode ? '#f59e0b' :
                    '#64748b'} />
                <text x={node.x} y={node.y - 28} fill={isCheckingNode ? '#f59e0b' :
                    '#64748b'} fontWeight="bold" textAnchor="middle" className="label" />
                <text x={node.x} y={node.y + 35} fill={isCheckingNode ? '#f59e0b' :
                    '#64748b'} />
            </g>
        );
    })
    }
    </svg>

```

```

<div className="w-full bg-slate-950/80 p-4 rounded">
    <p className="font-semibold text-amber-400">

```

```

        : 'border-slate-700/60 bg-slate-800/40'}``},
        <div className="text-xs text-slate
        <div className={`font-mono font-bo
    v>
        </div>
    );
    }}}}
  </div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
  <div>
    <h4 className="text-lg font-bold text-slate
    <div className="bg-slate-950/80 rounded-lg p
      {codeLines.map(item => {
        <div key={item.line} className={`py-1.5
        e === item.line ? 'bg-blue-900/80 text-amber
          <span className="text-slate-600 w-5 s
          <span className="whitespace-pre flex-
        </div>
      })}
    </div>
  </div>
</div>
</div>
</div>
</div>
  );
}

```

ФАЙЛ 29/82: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useCallback } from 'react'
```

```

    queue: [],
    visited: [],
    currentNode: null,
    logs: "Инициализация: начинаем с корня (0)."
```

});

```

queue.push(0);
visited.add(0);
steps.push({
  activeLine: 2,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Добавляем стартовую вершину 0 в очередь и по"
```

});

```

while (queue.length > 0) {
  steps.push({
    activeLine: 4,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: `Очередь не пуста, элементов: ${queue.length}
```

});

```

const curr = queue.shift()!;
steps.push({
  activeLine: 5,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: curr,
  logs: `Достаем из очереди (${curr}).`
```

});

```

const neighbors = ADJ[curr];
if (neighbors.length > 0) {
  for (const n of neighbors) {
    if (!visited.has(n)) {
```

```

    return steps;
  };

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
  }, []);

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);

  useEffect(() => {
    if (!isPlaying) return;
    if (stepIdx >= STEPS.length - 1) {
      setIsPlaying(false);
      return;
    }
    const timer = setTimeout(handleNext, 1200);
    return () => clearTimeout(timer);
  }, [isPlaying, stepIdx, handleNext]);

  return (
    <div className="bg-slate-900 border-2 border-slate-800" >
      <div className="flex flex-col md:flex-row items-center justify-center" >
        <div>
          <h3 className="text-2xl font-bold text-white" >
            <span className="text-blue-400" > </span> Алгоритм Брейна
          </h3>

```

```

        const isVisited = step.visited.includes(n);
        return (
          <line
            key={idx}
            x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
            className={`transition-all duration-500`}
          >
        </line>
      );
    }
  }
}

```

```

/* Вершины */
NODES.map(n => {
  const isCurrent = step.currentNode === n;
  const isInQueue = step.queue.includes(n);
  const isVisited = step.visited.includes(n);

```

```

  let fill = '#1e293b'; // slate-800
  let stroke = '#475569'; // slate-600
  let scale = 1;
  void scale;

```

```

  if (isCurrent) {
    fill = '#3b82f6'; // blue-500
    stroke = '#60a5fa'; // blue-400
    scale = 1.3;
  } else if (isInQueue) {
    fill = '#059669'; // emerald-600
    stroke = '#34d399'; // emerald-400
    scale = 1.1;
  } else if (isVisited) {
    fill = '#0f172a'; // slate-900
    stroke = '#3b82f6'; // blue-500
  }

```

```

  return (
    <g key={n.id} className="transition-transform"
      {isCurrent && {

```

```

        { line: 4, code: 'while not queue.isEmpty'
        { line: 5, code: '    node = queue.pop()'
        { line: 6, code: '    for neighbor in graph'
        { line: 7, code: '        if neighbor not in'
        { line: 8, code: '            visited.add(ne'
        { line: 9, code: '            queue.push(nei'
    ].map((cl) => {
        const isActive = step.activeLine === cl
        return (
            <div key={cl.line} className={`px-2 p-2
d border-l-2 border-blue-400 -ml-0.5`} : 'te
            <span className="opacity-40 w-6 inline-block">
            <span className="whitespace-pre">{code}
        </div>
    );
    })}
</div>
</div>

<div className="bg-slate-800/80 border border-blue-400"
    <div className="text-sm font-bold text-emerald-400">
        <span>Состояние Очереди (Queue):</span>
        <span className="text-xs text-slate-500">{step.queue.length}</span>
    </div>
    <div className="flex items-center gap-2 ml-2">
        {step.queue.length === 0 ? (
            <span className="text-slate-500 text-sm">Очередь пуста</span>
        ) : (
            step.queue.map((qId, i) => (
                <span key={`_${qId}-${i}`} className="text-sm font-medium text-emerald-400"
enter justify-center rounded-md shadow-lg p-2">
                    {qId}
                </span>
            ))
        )}
    </div>
    <div className="mt-2 text-sm text-amber-300">
        ms-center justify-center">

```

```
      borderColor: 'border-emerald-500/30',
      captionColor: 'text-emerald-400',
    },
  ];

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor} ${info.captionColor}`}>
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold ${info.captionColor}`}>{info.caption}</p>
    </div>
  );
}
```

ФАЙЛ 31/82: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";

interface Props {
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  compact?: boolean;
}
```

```

        <button
          onClick={() => onGo(prev.id)}
          className="group text-left p-4 rounded-xl bg-
            transition-all"
        >
          <div className="flex items-center gap-1.5 tex
            400 mb-1">
            <ChevronLeft className="w-3.5 h-3.5" /> Пре
          </div>
          <div className="text-sm font-bold text-slate-1
        </button>
      ) : (
        <div className="hidden sm:block" />
      )}

      {next && (
        <button
          onClick={() => onGo(next.id)}
          className="group text-right p-4 rounded-xl bg-
            transition-all sm:col-start-2"
        >
          <div className="flex items-center justify-end
            text-indigo-400 mb-1">
            Следующая тема <ChevronRight className="w-3
          </div>
          <div className="text-sm font-bold text-slate-1
        </button>
      )}
    </nav>
  );
};

```



```

    hideTimer.current = window.setTimeout(() => setAnchor(
    }, [cancelHide]);

    useEffect(() => {
      setAnchor(null);
      setSaving("idle");
    }, [chapter.id]);

    const saveHtml = useCallback(async () => {
      setSaving("work");
      try {
        await downloadChapterHtml(chapter);
        setSaving("done");
        window.setTimeout(() => setSaving("idle"), 2500);
      } catch {
        setSaving("idle");
      }
    }, [chapter]);

    const open = useCallback(
      (el: HTMLElement) => {
        const termId = el.dataset.termId;
        if (!termId) return;
        cancelHide();
        setAnchor({ termId, rect: el.getBoundingClientRect() });
      },
      [cancelHide]
    );

    const handlePointerOver = (e: React.MouseEvent) => {
      if (isTouch()) return;
      const el = (e.target as HTMLElement).closest<HTMLElement>(".chapter-link");
      if (el) open(el);
    };

    const handlePointerOut = (e: React.MouseEvent) => {
      if (isTouch()) return;
      const el = (e.target as HTMLElement).closest<HTMLElement>(".chapter-link");

```

```

    </button>
    <button
      onClick={() => window.print()}
      title="Распечатать или сохранить в PDF сред
      className="flex items-center gap-1.5 text-x
      300 hover:text-white hover:border-indigo-500
    >
      <Printer className="w-3.5 h-3.5" /> Печать
    </button>
    <span className="text-[11px] text-slate-500">
  </div>

```

```

<div
  ref={contentRef}
  className="prose prose-invert max-w-none"
  onMouseOver={handlePointerOver}
  onMouseOut={handlePointerOut}
  onClick={handleClick}
  onFocus={handleFocus}
  dangerouslySetInnerHTML={{ __html: chapter.con
/>

```

```

<p className="mt-6 flex items-start gap-2 text-
  ed-lg px-3 py-2">
  <MousePointerClick className="w-4 h-4 shrink-1
  <span>
    Подчёркнутые термины – интерактивные: навед
    объяснение на пальцах, мини-анимацию и кноп
  </span>
</p>

```

```

{viz && (
  <section id="chapter-viz" className="mt-8 scr
    <div className="flex items-center justify-b
      <h3 className="flex items-center gap-2 te
        <Sparkles className="w-4 h-4 text-indigo
          Демонстрация: {viz.title}
        </h3>

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
  { line: "    visited[v] = true;", highlight: "normal" },
  { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
  { line: "    ", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "normal" },
  { line: "        if (to == p) continue;", highlight: "normal" },
  { line: "        ", highlight: "normal" },
  { line: "        if (visited[to]) {", highlight: "normal" },
  { line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
  { line: "        } else {", highlight: "normal" },
  { line: "            dfs(to, v);", highlight: "normal" },
  { line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
  { line: "        ", highlight: "normal" },
  { line: "        if (low[to] > tin[v]) {", highlight: "normal" },
  { line: "            // ЭТО МОСТ!", highlight: "normal" },
  { line: "            ", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "    }", highlight: "normal" },
  { line: "}", highlight: "normal" },
];

interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
}

```

```

    },
    {
        currentNode: 6, visitedIndices: [0,1,6],
        tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
        activeEdge: [1,6], backEdge: null,
        log: "Из В идём в G. tin[G]=3, low[G]=3.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 1, visitedIndices: [0,1,6],
        tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
        activeEdge: null, backEdge: null,
        log: "У G нет соседей (кроме В). Возврат. low[G]=3 : ",
        activeCodeLine: [13,14,15]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
        activeEdge: [1,2], backEdge: null,
        log: "Из В идём в С. tin[C]=4, low[C]=4.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
        activeEdge: null, backEdge: [2,0],
        log: "Видим соседа А (уже посещён)! Обратное ребро",
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3],
        tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
        activeEdge: [2,3], backEdge: null,
        log: "Идём С→D. tin[D]=5, low[D]=5.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4],

```

```

        activeCodeLine: [13,14,15,16,17]
    },
    {
        currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:2,2:3,3:4,4:5,5:6},
        activeEdge: null, backEdge: null,
        log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) - мост! DFS завершён! Мосты: (B,G) и (C,D). Алгоритм готов!",
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:2,2:3,3:4,4:5,5:6},
        activeEdge: null, backEdge: null,
        log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм готов!",
        activeCodeLine: [11,13,16,17,18]
    },
    {
        currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:2,2:3,3:4,4:5,5:6},
        activeEdge: null, backEdge: null,
        log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм готов!",
        activeCodeLine: []
    }
  ],
];

```

```

export function DfsBridgesSimulator() {
  const [dfsIdx, setDfsIdx] = useState(0);
  const [dfsAuto, setDfsAuto] = useState(false);
  const dfsTimer = useRef<NodeJS.Timeout | null>(null);

  useEffect(() => {
    if (dfsAuto) {
      dfsTimer.current = setInterval(() => {
        setDfsIdx(prev => {
          if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
            return prev + 1;
          }
        });
      }, 2500);
    }
  }, [dfsAuto]);
}

```



```

const edges: Edge[] = [
  { u: 'A', v: 'B', w: 4 },
  { u: 'A', v: 'C', w: 2 },
  { u: 'C', v: 'B', w: 1 },
  { u: 'C', v: 'G', w: 3 },
  { u: 'C', v: 'E', w: 8 },
  { u: 'B', v: 'G', w: 4 },
  { u: 'B', v: 'D', w: 5 },
  { u: 'G', v: 'D', w: 1 },
  { u: 'G', v: 'E', w: 2 },
  { u: 'D', v: 'F', w: 3 },
  { u: 'E', v: 'F', w: 1 },
];

const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
  adjList[e.u].push({ v: e.v, w: e.w });
});

const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push(start)' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлечение минимума' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbors(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push((dist[v], v))' },
  { line: 11, text: '    return dist // Все кратчайшие пути' },
];

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

```



```

        simulationSteps.push({
            currentNode: u, checkingEdge: { u, v },
            distances: { ...dist }, visited: { ...visited },
            log: `  Смотрим соседа ${v}. Текущее расстояние
                ${dist[u] + edge.w}.`,
            activeCodeLine: 8,
        });

        if (dist[u] + edge.w < dist[v]) {
            dist[v] = dist[u] + edge.w;
            prev[v] = u;
            simulationSteps.push({
                currentNode: u, checkingEdge: { u, v },
                distances: { ...dist }, visited: { ...visited },
                log: `  Улучшение (Релаксация)! Новое кратчайшее
                    расстояние до ${v} равно ${dist[v]}.`,
                activeCodeLine: 9,
            });
        }
    }
}

simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: `  Дейкстра всё доставил! Все возможные кратчайшие
        расстояния найдены!`,
    activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => {
            setCurrentStepIndex((prev) => prev + 1);
        }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {

```

```

    </button>
    <button
      onClick={() => { setIsPlaying(false); if (c
      disabled={currentStepIndex >= steps.length
      className="flex items-center gap-1 bg-indigo
      px-3 py-2 rounded-lg text-sm font-medium tr
    >
      ⏮
    </button>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 m
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-indigo-950/2
    stify-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-indi
      ont-bold shadow-sm">
      ⏮ {currentStepIndex + 1} из {steps.length
    </div>

    <svg className="w-full h-auto max-h-[500px]"
      <defs>
        <marker id="arrow-dh-normal" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#47556
        </marker>
        <marker id="arrow-dh-active" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#f59e0
        </marker>
        <marker id="arrow-dh-opt" markerWidth="6"
          <path d="M0,0 L6,3 L0,6 z" fill="#10b98
        </marker>
      </defs>
      {/* Пёпса */}
      {edges.map((edge, idx) => {
        const uNode = nodes.find((n) => n.id ===
        const vNode = nodes.find((n) => n.id ===
        const isChecking =

```

```

const isCurrent = currentStep.currentNode
const isVisited = currentStep.visited[node.id]
const dist = currentStep.distances[node.id]

return (
  <g key={node.id} className="transition-all duration-300">
    <circle
      cx={node.x} cy={node.y}
      r={isCurrent ? 22 : 18}
      fill={isCurrent ? '#4f46e5' : isVisited ? '#a5b4fc' : '#f1f3f4'}
      stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f1f3f4' : '#f1f3f4'}
      strokeWidth={isCurrent ? 4 : 2}
      className="transition-all duration-300"
    />
    <text x={node.x} y={node.y + 5} fill="black" font-size="12px">
      {node.id}
    </text>
    <text x={node.x} y={node.y - 28} fill="white" font-size="12px">
      {node.name}
    </text>
    <text x={node.x} y={node.y + 35} fill="white" font-size="12px">
      {node.name.split(' ')[0]}
    </text>
  </g>
);
}}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded">
  <p className="font-semibold text-amber-400">
    <p className="min-h-[40px] flex items-center">
  </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50">
  <h4 className="text-lg font-bold text-emerald-400">
  <div className="space-y-2.5 flex-1 overflow-y-auto">

```

```

<div className="w-full lg:w-1/2 bg-rose-950/10
  <div>
    <h4 className="text-lg font-bold text-rose-
  <div className="overflow-x-auto">
    <table className="w-full text-left border
      <thead>
        <tr className="border-b border-rose-9
          <th className="py-2 px-3">Вершина</
          <th className="py-2 px-3">Расстояни
          <th className="py-2 px-3">Из (пред.
        </tr>
      </thead>
      <tbody className="text-sm">
        {nodes.map((n) => {
          const dist = currentStep.distances[n
          const prev = currentStep.previous[n
          const vis = currentStep.visited[n.id
          const isCurr = currentStep.currentN

          return (
            <tr key={n.id} className={`border
              isCurr ? 'bg-indigo-950/60 for
            }`}>
              <td className="py-2.5 px-3 flex
                <span className={`w-2 h-2 roun
                  {n.name}
                </td>
                <td className="py-2.5 px-3 font
                  {dist === 999 ? '∞' : dist}
                </td>
                <td className="py-2.5 px-3 text
                  {prev || '-'}
                </td>
              </tr>
            );
          }}}
        </tbody>
      </table>

```

```

interface DPStep {
  id: number;
  n: number;
  action: 'call' | 'memo_hit' | 'base_case' | 'calc' |
  desc: string;
  codeLine: number;
  memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "    if (n in memo) return memo[n];",
  /* 3 */ "    if (n <= 2) return 1;",
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo",
  /* 5 */ "    return memo[n];",
  /* 6 */ "}"
];

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |

  useEffect(() => {
    const generatedSteps: DPStep[] = [];
    let stepId = 0;
    const memoMap: { [key: number]: number } = {};

    function simulateFib(n: number): number {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вызов fibMemo(${n}). Проверяем наличие в

```

```

    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'calc',
      desc: `Сохраняем в кэш: мемо[${n}] = ${res1} + ${res2}`,
      codeLine: 5,
      memoState: { ...memoMap }
    });

    return memoMap[n];
  }

  const finalRes = simulateFib(targetN);

  generatedSteps.push({
    id: stepId++,
    n: targetN,
    action: 'done',
    desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
    codeLine: 6,
    memoState: { ...memoMap }
  });

  setSteps(generatedSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [targetN]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex(prev => prev + 1);
    }, speed);
  } else if (currentStepIndex >= steps.length - 1) {

```

```
</div>
```

```
<div className="flex items-center bg-slate-950" >
  <button
    onClick={() => { setIsPlaying(false); set
    className="p-2 text-slate-400 hover:text-white"
    title="Сбросить"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
  <button
    onClick={() => { setIsPlaying(false); set
    disabled={currentStepIndex === 0}
    className="p-2 text-slate-400 hover:text-white"
    title="Шаг назад"
  >
    <ChevronLeft className="w-5 h-5" />
  </button>
  <button
    onClick={() => setIsPlaying(!isPlaying)}
    className={`flex items-center gap-1.5 px-2
      isPlaying
        ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
        : 'bg-emerald-600 text-white hover:bg-emerald-700'
    }`
  >
    {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
    {isPlaying ? 'Пауза' : 'Пуск'}
  </button>
  <button
    onClick={() => { setIsPlaying(false); set
    disabled={currentStepIndex === steps.length - 1}
    className="p-2 text-slate-400 hover:text-white"
    title="Шаг вперед"
  >
    <ChevronRight className="w-5 h-5" />
  </button>
</div>
```

```

        </div>
        <div className="text-xs font-mono bg-slate-500 text-emerald-400 p-2 rounded">
          Текущий N: <strong className="text-emerald-400">{currentN}</strong>
        </div>
      </div>
    )}
  </div>

```

```

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-500 p-2">
  <button
    onClick={() => setActiveTab('table')}
    className={`flex items-center gap-2 px-6 py-3 rounded ${
      activeTab === 'table'
        ? 'border-emerald-500 text-emerald-400 bg-slate-500'
        : 'border-transparent text-slate-400 hover:text-emerald-400'
    }`}
  >
    <Eye className="w-4 h-4" />
    Степ-бай-степ состояние
  </button>
  <button
    onClick={() => setActiveTab('code')}
    className={`flex items-center gap-2 px-6 py-3 rounded ${
      activeTab === 'code'
        ? 'border-emerald-500 text-emerald-400 bg-slate-500'
        : 'border-transparent text-slate-400 hover:text-emerald-400'
    }`}
  >
    <Code className="w-4 h-4" />
    Интерактивный код
  </button>
</div>

```

```

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify-start p-2">
  {activeTab === 'table' && (
    <div className="bg-slate-950 p-6 rounded-2xl">

```



```

        }}}
      </pre>
      <div className="p-6 bg-slate-900/50 border-1 border-slate-800">
        <span className="p-1.5 bg-emerald-500/20 border border-slate-800 rounded">
          <span>{currentStep?.desc}</span>
        </div>
      </div>
    ))
  </div>

  {/* Progress Bar */}
  <div className="mt-8 pt-6 border-t border-slate-800">
    <div className="flex items-center justify-between">
      <span>Прогресс выполнения</span>
      <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
    </div>
    <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
      <div
        className="h-full bg-gradient-to-r from-emerald-500 to-emerald-400"
        style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
      ></div>
    </div>
  </div>
</div>
);
}

```

ФАЙЛ 36/82: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState } from "react";
import { Undo } from "lucide-react";

```

```

const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },

```

```

        setC1Msg("Это ребро уже пройдено! Эйлер не прощает");
        setC1Done(true); return;
    }
    const nextEdges = [...c1VisitedEdges, key];
    const nextPath = [...c1Path, vId];
    setC1VisitedEdges(nextEdges);
    setC1Path(nextPath);

    if (nextEdges.length === C1_EDGES.length) {
        const startDegree = C1_EDGES.filter(e => e.u === vId).length;
        const endDegree = C1_EDGES.filter(e => e.v === vId).length;
        const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
        if (isCycle && nextPath[0] === vId) {
            setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, факт!");
        } else {
            setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! Сделайте шаг назад!`);
        }
        setC1Done(true);
    } else {
        setC1Msg(`Пройдено рёбер: ${nextEdges.length} / ${C1_EDGES.length}`);
    }
};

return (
    <div className="space-y-4">
        <div className="flex items-center justify-between">
            <h4 className="text-base font-bold text-white">Решение задачи
            <button onClick={resetC1} className="flex items-center justify-center gap-2
                old border border-slate-700 transition-all">
                <Undo className="h-3.5 w-3.5" /> Сброс
            </button>
        </div>

        <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
            <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)]
                -hidden">
                <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)]
                <svg className="w-full h-[260px] z-10 relative">
                    {C1_EDGES.map((e, i) => {

```

```
    </div>
  );
}
```

ФАЙЛ 37/82: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';
```

```
interface Question {
  id: number;
  question: string;
  options: string[];
  correctAnswer: number;
  explanation: string;
}
```

```
const quizQuestions: Question[] = [
  {
    id: 1,
    question: 'Что мы делаем в алгоритме Краскала, если ребро, красным) цвет?',
    options: [
      'Мы радуемся и добавляем его в минимальное остовное дерево',
      'Мы перекрашиваем их в синий цвет и делим граф по компонентам связности',
      'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
      'Мы вызываем алгоритм Дейкстры для поиска нового минимального ребра'
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане, добавление этого ребра создаст цикл. Это нарушит структуру дерева.'
  },
  {
    id: 2,
```

```

    id: 5,
    question: 'Какая структура данных идеально помогает
options: [
    'СНМ (Система непересекающихся множеств / Disjoin
    'Стек (LIFO).',
    'Красно-черное дерево.',
    'Хэш-таблица со списками коллизий.'
],
    correctAnswer: 0,
    explanation: 'Блестяще! СНМ (DSU) позволяет за прак
    '
}
];

```

```

export const ExpressQuiz: React.FC = () => {
    const [currentQIndex, setCurrentQIndex] = useState<num
    const [selectedOption, setSelectedOption] = useState<
    const [isAnswered, setIsAnswered] = useState<boolean>
    const [score, setScore] = useState<number>{0};
    const [showResults, setShowResults] = useState<boolean>

    const currentQ = quizQuestions[currentQIndex];

    const handleSelectOption = (index: number) => {
        if (isAnswered) return;
        setSelectedOption(index);
        setIsAnswered(true);
        if (index === currentQ.correctAnswer) {
            setScore(prev => prev + 1);
        }
    };

    const handleNext = () => {
        if (currentQIndex + 1 < quizQuestions.length) {
            setCurrentQIndex(prev => prev + 1);
            setSelectedOption(null);
            setIsAnswered(false);
        } else {

```

```

        d rounded-xl shadow-lg shadow-indigo-600/30
      >
      <RotateCcw className="w-5 h-5" />
      <span>Пройти тест заново</span>
    </button>
  </div>
);
}

return (
  <div className="bg-slate-900/90 border border-slate
    >
    <div className="flex items-center justify-between
      <div className="flex items-center gap-3">
        <span className="bg-purple-600 text-white px-1
          Экспресс-тест
        </span>
        <h3 className="text-xl font-bold text-white">
      </div>
      <span className="text-sm font-mono text-slate-400
        Bonpoc {currentQIndex + 1} / {quizQuestions.l
      </span>
    </div>

    <div className="mb-8">
      <h4 className="text-xl md:text-2xl font-bold te
        {currentQ.question}
      </h4>

      <div className="space-y-4">
        {currentQ.options.map((option, idx) => {
          const isSelected = selectedOption === idx;
          const isCorrect = idx === currentQ.correctA

          let optionStyle = "bg-slate-800/80 hover:bg
          if (isAnswered) {
            if (isCorrect) {
              optionStyle = "bg-emerald-950/80 border

```

```

        <span>Объяснение:</span>
      </div>
      <p className="text-slate-300 text-sm leading-
        {currentQ.explanation}
      </p>
    </div>
  })

  <div className="flex justify-end pt-4 border-t bo
    <button
      onClick={handleNext}
      disabled={!isAnswered}
      className={
        "px-8 py-3.5 font-bold rounded-xl transition
        (!isAnswered
          ? "bg-slate-800 text-slate-500 border bor
          : "bg-indigo-600 hover:bg-indigo-500 activ
        }
      >
        {currentQIndex + 1 < quizQuestions.length ? '
      </button>
    </div>
  </div>
);
};

```

ФАЙЛ 38/82: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import { useState, useEffect } from 'react';
```

```

interface Step {
  k: number; i: number; j: number;
  matrix: number[][]; updated: boolean;
  log: string; activeCodeLine: number;

```

```

const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V' },
  { line: 2, text: '    dist = copy(matrix)' },
  { line: 3, text: '    for k from 0 to V - 1: // Пос' },
  { line: 4, text: '        for i from 0 to V - 1:' },
  { line: 5, text: '            for j from 0 to V - 1' },
  { line: 6, text: '                if dist[i][k] + d' },
  { line: 7, text: '                    dist[i][j] = ' },
  { line: 8, text: '    return dist' },
];

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist = initialMatrix.map(row => [...row]);

  simulationSteps.push({
    k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: '    Инициализация матрицы прямых путей, ∞ (999)',
    activeCodeLine: 2,
  });

  const N = 7;
  for (let k = 0; k < N; k++) {
    simulationSteps.push({
      k, i: -1, j: -1, matrix: dist.map(r => [...r]),
      log: `+ Внешний цикл. Разрешаем пути через пром`,
      activeCodeLine: 3,
    });

    for (let i = 0; i < N; i++) {
      for (let j = 0; j < N; j++) {
        if (i === j || i === k || j === k) continue;

        const oldVal = dist[i][j];
        const viaVal = dist[i][k] + dist[k][j];
        if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
          dist[i][j] = viaVal;
          simulationSteps.push({

```



```

    }}}
  </svg>

  <div className="w-full bg-slate-950/80 p-4 rounded-2xl" >
    <p className="font-semibold text-amber-400" >
      <p className="min-h-[40px] flex items-center" >
    </div>
  </div>

  { /* Список смежности */ }
  <div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl" >
    <h4 className="text-lg font-bold text-emerald-400" >
      <div className="space-y-2 text-sm font-mono text-white" >
        {Object.keys(adjList).map((uStr) => {
          const u = parseInt(uStr);
          const isI = currentStep.i === u;
          const isK = currentStep.k === u;
          return (
            <div key={u} className={`flex flex-wrap` >
              <div key={u} className={`flex flex-wrap` >
                isI ? 'bg-amber-900/40 border-amber-400' :
                isK ? 'bg-indigo-900/40 border-indigo-400' :
              }` >
              <span className="font-bold text-white text-400" >→</span>
              {adjList[u].map((edge, idx) => {
                const isUpd = currentStep.i === u;
                const isVia = currentStep.k !== u;
                const isDown = currentStep.j === edge.v);
                return (
                  <span key={idx} className={`px-2 py-1` >
                    'down' : isVia ? 'bg-indigo-500 text-white border-indigo-400' :
                    {edge.v} ({edge.w})
                  </span>
                )
              })}
            </div>
          )
        })}
      </div>
    );
  }}}

```

```

        </table>
      </div>
    </div>

    { /* Код алгоритма */
    <div className="w-full lg:w-1/2 bg-slate-900/60" >
      <div>
        <h4 className="text-lg font-bold text-slate-100">
          <div className="bg-slate-950/80 rounded-lg p-4">
            {codeLines.map(item => (
              <div key={item.line} className={`py-1.5 px-3
                e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'}`>
                <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
                <span className="whitespace-pre flex-grow-1">{item.code}</span>
              </div>
            ))}
          </div>
        </div>
      </div>
    </div>
  </div>
);
}

```

ФАЙЛ 39/82: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';

```

```

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```

```

const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 400, y: 40 },
  { id: 'C', label: 'C', x: 600, y: 40 },
  { id: 'D', label: 'D', x: 800, y: 40 },
  { id: 'E', label: 'E', x: 1000, y: 40 },
  { id: 'F', label: 'F', x: 200, y: 140 },
  { id: 'G', label: 'G', x: 400, y: 140 },
  { id: 'H', label: 'H', x: 600, y: 140 },
  { id: 'I', label: 'I', x: 800, y: 140 },
  { id: 'J', label: 'J', x: 1000, y: 140 },
  { id: 'K', label: 'K', x: 200, y: 240 },
  { id: 'L', label: 'L', x: 400, y: 240 },
  { id: 'M', label: 'M', x: 600, y: 240 },
  { id: 'N', label: 'N', x: 800, y: 240 },
  { id: 'O', label: 'O', x: 1000, y: 240 },
  { id: 'P', label: 'P', x: 200, y: 340 },
  { id: 'Q', label: 'Q', x: 400, y: 340 },
  { id: 'R', label: 'R', x: 600, y: 340 },
  { id: 'S', label: 'S', x: 800, y: 340 },
  { id: 'T', label: 'T', x: 1000, y: 340 },
  { id: 'U', label: 'U', x: 200, y: 440 },
  { id: 'V', label: 'V', x: 400, y: 440 },
  { id: 'W', label: 'W', x: 600, y: 440 },
  { id: 'X', label: 'X', x: 800, y: 440 },
  { id: 'Y', label: 'Y', x: 1000, y: 440 },
  { id: 'Z', label: 'Z', x: 200, y: 540 },
  { id: 'AA', label: 'AA', x: 400, y: 540 },
  { id: 'AB', label: 'AB', x: 600, y: 540 },
  { id: 'AC', label: 'AC', x: 800, y: 540 },
  { id: 'AD', label: 'AD', x: 1000, y: 540 },
  { id: 'AE', label: 'AE', x: 200, y: 640 },
  { id: 'AF', label: 'AF', x: 400, y: 640 },
  { id: 'AG', label: 'AG', x: 600, y: 640 },
  { id: 'AH', label: 'AH', x: 800, y: 640 },
  { id: 'AI', label: 'AI', x: 1000, y: 640 },
  { id: 'AJ', label: 'AJ', x: 200, y: 740 },
  { id: 'AK', label: 'AK', x: 400, y: 740 },
  { id: 'AL', label: 'AL', x: 600, y: 740 },
  { id: 'AM', label: 'AM', x: 800, y: 740 },
  { id: 'AN', label: 'AN', x: 1000, y: 740 },
  { id: 'AO', label: 'AO', x: 200, y: 840 },
  { id: 'AP', label: 'AP', x: 400, y: 840 },
  { id: 'AQ', label: 'AQ', x: 600, y: 840 },
  { id: 'AR', label: 'AR', x: 800, y: 840 },
  { id: 'AS', label: 'AS', x: 1000, y: 840 },
  { id: 'AT', label: 'AT', x: 200, y: 940 },
  { id: 'AU', label: 'AU', x: 400, y: 940 },
  { id: 'AV', label: 'AV', x: 600, y: 940 },
  { id: 'AW', label: 'AW', x: 800, y: 940 },
  { id: 'AX', label: 'AX', x: 1000, y: 940 },
  { id: 'AY', label: 'AY', x: 200, y: 1040 },
  { id: 'AZ', label: 'AZ', x: 400, y: 1040 },
  { id: 'BA', label: 'BA', x: 600, y: 1040 },
  { id: 'BB', label: 'BB', x: 800, y: 1040 },
  { id: 'BC', label: 'BC', x: 1000, y: 1040 },
  { id: 'BD', label: 'BD', x: 200, y: 1140 },
  { id: 'BE', label: 'BE', x: 400, y: 1140 },
  { id: 'BF', label: 'BF', x: 600, y: 1140 },
  { id: 'BG', label: 'BG', x: 800, y: 1140 },
  { id: 'BH', label: 'BH', x: 1000, y: 1140 },
  { id: 'BI', label: 'BI', x: 200, y: 1240 },
  { id: 'BJ', label: 'BJ', x: 400, y: 1240 },
  { id: 'BK', label: 'BK', x: 600, y: 1240 },
  { id: 'BL', label: 'BL', x: 800, y: 1240 },
  { id: 'BM', label: 'BM', x: 1000, y: 1240 },
  { id: 'BN', label: 'BN', x: 200, y: 1340 },
  { id: 'BO', label: 'BO', x: 400, y: 1340 },
  { id: 'BP', label: 'BP', x: 600, y: 1340 },
  { id: 'BQ', label: 'BQ', x: 800, y: 1340 },
  { id: 'BR', label: 'BR', x: 1000, y: 1340 },
  { id: 'BS', label: 'BS', x: 200, y: 1440 },
  { id: 'BT', label: 'BT', x: 400, y: 1440 },
  { id: 'BU', label: 'BU', x: 600, y: 1440 },
  { id: 'BV', label: 'BV', x: 800, y: 1440 },
  { id: 'BW', label: 'BW', x: 1000, y: 1440 },
  { id: 'BX', label: 'BX', x: 200, y: 1540 },
  { id: 'BY', label: 'BY', x: 400, y: 1540 },
  { id: 'BZ', label: 'BZ', x: 600, y: 1540 },
  { id: 'CA', label: 'CA', x: 800, y: 1540 },
  { id: 'CB', label: 'CB', x: 1000, y: 1540 },
  { id: 'CC', label: 'CC', x: 200, y: 1640 },
  { id: 'CD', label: 'CD', x: 400, y: 1640 },
  { id: 'CE', label: 'CE', x: 600, y: 1640 },
  { id: 'CF', label: 'CF', x: 800, y: 1640 },
  { id: 'CG', label: 'CG', x: 1000, y: 1640 },
  { id: 'CH', label: 'CH', x: 200, y: 1740 },
  { id: 'CI', label: 'CI', x: 400, y: 1740 },
  { id: 'CJ', label: 'CJ', x: 600, y: 1740 },
  { id: 'CK', label: 'CK', x: 800, y: 1740 },
  { id: 'CL', label: 'CL', x: 1000, y: 1740 },
  { id: 'CM', label: 'CM', x: 200, y: 1840 },
  { id: 'CN', label: 'CN', x: 400, y: 1840 },
  { id: 'CO', label: 'CO', x: 600, y: 1840 },
  { id: 'CP', label: 'CP', x: 800, y: 1840 },
  { id: 'CQ', label: 'CQ', x: 1000, y: 1840 },
  { id: 'CR', label: 'CR', x: 200, y: 1940 },
  { id: 'CS', label: 'CS', x: 400, y: 1940 },
  { id: 'CT', label: 'CT', x: 600, y: 1940 },
  { id: 'CU', label: 'CU', x: 800, y: 1940 },
  { id: 'CV', label: 'CV', x: 1000, y: 1940 },
  { id: 'CW', label: 'CW', x: 200, y: 2040 },
  { id: 'CX', label: 'CX', x: 400, y: 2040 },
  { id: 'CY', label: 'CY', x: 600, y: 2040 },
  { id: 'CZ', label: 'CZ', x: 800, y: 2040 },
  { id: 'DA', label: 'DA', x: 1000, y: 2040 },
  { id: 'DB', label: 'DB', x: 200, y: 2140 },
  { id: 'DC', label: 'DC', x: 400, y: 2140 },
  { id: 'DD', label: 'DD', x: 600, y: 2140 },
  { id: 'DE', label: 'DE', x: 800, y: 2140 },
  { id: 'DF', label: 'DF', x: 1000, y: 2140 },
  { id: 'DG', label: 'DG', x: 200, y: 2240 },
  { id: 'DH', label: 'DH', x: 400, y: 2240 },
  { id: 'DI', label: 'DI', x: 600, y: 2240 },
  { id: 'DJ', label: 'DJ', x: 800, y: 2240 },
  { id: 'DK', label: 'DK', x: 1000, y: 2240 },
  { id: 'DL', label: 'DL', x: 200, y: 2340 },
  { id: 'DM', label: 'DM', x: 400, y: 2340 },
  { id: 'DN', label: 'DN', x: 600, y: 2340 },
  { id: 'DO', label: 'DO', x: 800, y: 2340 },
  { id: 'DP', label: 'DP', x: 1000, y: 2340 },
  { id: 'DQ', label: 'DQ', x: 200, y: 2440 },
  { id: 'DR', label: 'DR', x: 400, y: 2440 },
  { id: 'DS', label: 'DS', x: 600, y: 2440 },
  { id: 'DT', label: 'DT', x: 800, y: 2440 },
  { id: 'DU', label: 'DU', x: 1000, y: 2440 },
  { id: 'DV', label: 'DV', x: 200, y: 2540 },
  { id: 'DW', label: 'DW', x: 400, y: 2540 },
  { id: 'DX', label: 'DX', x: 600, y: 2540 },
  { id: 'DY', label: 'DY', x: 800, y: 2540 },
  { id: 'DZ', label: 'DZ', x: 1000, y: 2540 },
  { id: 'EA', label: 'EA', x: 200, y: 2640 },
  { id: 'EB', label: 'EB', x: 400, y: 2640 },
  { id: 'EC', label: 'EC', x: 600, y: 2640 },
  { id: 'ED', label: 'ED', x: 800, y: 2640 },
  { id: 'EE', label: 'EE', x: 1000, y: 2640 },
  { id: 'EF', label: 'EF', x: 200, y: 2740 },
  { id: 'EG', label: 'EG', x: 400, y: 2740 },
  { id: 'EH', label: 'EH', x: 600, y: 2740 },
  { id: 'EI', label: 'EI', x: 800, y: 2740 },
  { id: 'EJ', label: 'EJ', x: 1000, y: 2740 },
  { id: 'EK', label: 'EK', x: 200, y: 2840 },
  { id: 'EL', label: 'EL', x: 400, y: 2840 },
  { id: 'EM', label: 'EM', x: 600, y: 2840 },
  { id: 'EN', label: 'EN', x: 800, y: 2840 },
  { id: 'EO', label: 'EO', x: 1000, y: 2840 },
  { id: 'EP', label: 'EP', x: 200, y: 2940 },
  { id: 'EQ', label: 'EQ', x: 400, y: 2940 },
  { id: 'ER', label: 'ER', x: 600, y: 2940 },
  { id: 'ES', label: 'ES', x: 800, y: 2940 },
  { id: 'ET', label: 'ET', x: 1000, y: 2940 },
  { id: 'EU', label: 'EU', x: 200, y: 3040 },
  { id: 'EV', label: 'EV', x: 400, y: 3040 },
  { id: 'EW', label: 'EW', x: 600, y: 3040 },
  { id: 'EX', label: 'EX', x: 800, y: 3040 },
  { id: 'EY', label: 'EY', x: 1000, y: 3040 },
  { id: 'EZ', label: 'EZ', x: 200, y: 3140 },
  { id: 'FA', label: 'FA', x: 400, y: 3140 },
  { id: 'FB', label: 'FB', x: 600, y: 3140 },
  { id: 'FC', label: 'FC', x: 800, y: 3140 },
  { id: 'FD', label: 'FD', x: 1000, y: 3140 },
  { id: 'FE', label: 'FE', x: 200, y: 3240 },
  { id: 'FF', label: 'FF', x: 400, y: 3240 },
  { id: 'FG', label: 'FG', x: 600, y: 3240 },
  { id: 'FH', label: 'FH', x: 800, y: 3240 },
  { id: 'FI', label: 'FI', x: 1000, y: 3240 },
  { id: 'FJ', label: 'FJ', x: 200, y: 3340 },
  { id: 'FK', label: 'FK', x: 400, y: 3340 },
  { id: 'FL', label: 'FL', x: 600, y: 3340 },
  { id: 'FM', label: 'FM', x: 800, y: 3340 },
  { id: 'FN', label: 'FN', x: 1000, y: 3340 },
  { id: 'FO', label: 'FO', x: 200, y: 3440 },
  { id: 'FP', label: 'FP', x: 400, y: 3440 },
  { id: 'FQ', label: 'FQ', x: 600, y: 3440 },
  { id: 'FR', label: 'FR', x: 800, y: 3440 },
  { id: 'FS', label: 'FS', x: 1000, y: 3440 },
  { id: 'FT', label: 'FT', x: 200, y: 3540 },
  { id: 'FU', label: 'FU', x: 400, y: 3540 },
  { id: 'FV', label: 'FV', x: 600, y: 3540 },
  { id: 'FW', label: 'FW', x: 800, y: 3540 },
  { id: 'FX', label: 'FX', x: 1000, y: 3540 },
  { id: 'FY', label: 'FY', x: 200, y: 3640 },
  { id: 'FZ', label: 'FZ', x: 400, y: 3640 },
  { id: 'GA', label: 'GA', x: 600, y: 3640 },
  { id: 'GB', label: 'GB', x: 800, y: 3640 },
  { id: 'GC', label: 'GC', x: 1000, y: 3640 },
  { id: 'GD', label: 'GD', x: 200, y: 3740 },
  { id: 'GE', label: 'GE', x: 400, y: 3740 },
  { id: 'GF', label: 'GF', x: 600, y: 3740 },
  { id: 'GH', label: 'GH', x: 800, y: 3740 },
  { id: 'GI', label: 'GI', x: 1000, y: 3740 },
  { id: 'GJ', label: 'GJ', x: 200, y: 3840 },
  { id: 'GK', label: 'GK', x: 400, y: 3840 },
  { id: 'GL', label: 'GL', x: 600, y: 3840 },
  { id: 'GM', label: 'GM', x: 800, y: 3840 },
  { id: 'GN', label: 'GN', x: 1000, y: 3840 },
  { id: 'GO', label: 'GO', x: 200, y: 3940 },
  { id: 'GP', label: 'GP', x: 400, y: 3940 },
  { id: 'GQ', label: 'GQ', x: 600, y: 3940 },
  { id: 'GR', label: 'GR', x: 800, y: 3940 },
  { id: 'GS', label: 'GS', x: 1000, y: 3940 },
  { id: 'GT', label: 'GT', x: 200, y: 4040 },
  { id: 'GU', label: 'GU', x: 400, y: 4040 },
  { id: 'GV', label: 'GV', x: 600, y: 4040 },
  { id: 'GW', label: 'GW', x: 800, y: 4040 },
  { id: 'GX', label: 'GX', x: 1000, y: 4040 },
  { id: 'GY', label: 'GY', x: 200, y: 4140 },
  { id: 'GZ', label: 'GZ', x: 400, y: 4140 },
  { id: 'HA', label: 'HA', x: 600, y: 4140 },
  { id: 'HB', label: 'HB', x: 800, y: 4140 },
  { id: 'HC', label: 'HC', x: 1000, y: 4140 },
  { id: 'HD', label: 'HD', x: 200, y: 4240 },
  { id: 'HE', label: 'HE', x: 400, y: 4240 },
  { id: 'HF', label: 'HF', x: 600, y: 4240 },
  { id: 'HG', label: 'HG', x: 800, y: 4240 },
  { id: 'HI', label: 'HI', x: 1000, y: 4240 },
  { id: 'HJ', label: 'HJ', x: 200, y: 4340 },
  { id: 'HK', label: 'HK', x: 400, y: 4340 },
  { id: 'HL', label: 'HL', x: 600, y: 4340 },
  { id: 'HM', label: 'HM', x: 800, y: 4340 },
  { id: 'HN', label: 'HN', x: 1000, y: 4340 },
  { id: 'HO', label: 'HO', x: 200, y: 4440 },
  { id: 'HP', label: 'HP', x: 400, y: 4440 },
  { id: 'HQ', label: 'HQ', x: 600, y: 4440 },
  { id: 'HR', label: 'HR', x: 800, y: 4440 },
  { id: 'HS', label: 'HS', x: 1000, y: 4440 },
  { id: 'HT', label: 'HT', x: 200, y: 4540 },
  { id: 'HU', label: 'HU', x: 400, y: 4540 },
  { id: 'HV', label: 'HV', x: 600, y: 4540 },
  { id: 'HW', label: 'HW', x: 800, y: 4540 },
  { id: 'HX', label: 'HX', x: 1000, y: 4540 },
  { id: 'HY', label: 'HY', x: 200, y: 4640 },
  { id: 'HZ', label: 'HZ', x: 400, y: 4640 },
  { id: 'IA', label: 'IA', x: 600, y: 4640 },
  { id: 'IB', label: 'IB', x: 800, y: 4640 },
  { id: 'IC', label: 'IC', x: 1000, y: 4640 },
  { id: 'ID', label: 'ID', x: 200, y: 4740 },
  { id: 'IE', label: 'IE', x: 400, y: 4740 },
  { id: 'IF', label: 'IF', x: 600, y: 4740 },
  { id: 'IG', label: 'IG', x: 800, y: 4740 },
  { id: 'IH', label: 'IH', x: 1000, y: 4740 },
  { id: 'IJ', label: 'IJ', x: 200, y: 4840 },
  { id: 'IK', label: 'IK', x: 400, y: 4840 },
  { id: 'IL', label: 'IL', x: 600, y: 4840 },
  { id: 'IM', label: 'IM', x: 800, y: 4840 },
  { id: 'IN', label: 'IN', x: 1000, y: 4840 },
  { id: 'IO', label: 'IO', x: 200, y: 4940 },
  { id: 'IP', label: 'IP', x: 400, y: 4940 },
  { id: 'IQ', label: 'IQ', x: 600, y: 4940 },
  { id: 'IR', label: 'IR', x: 800, y: 4940 },
  { id: 'IS', label: 'IS', x: 1000, y: 4940 },
  { id: 'IT', label: 'IT', x: 200, y: 5040 },
  { id: 'IU', label: 'IU', x: 400, y: 5040 },
  { id: 'IV', label: 'IV', x: 600, y: 5040 },
  { id: 'IW', label: 'IW', x: 800, y: 5040 },
  { id: 'IX', label: 'IX', x: 1000, y: 5040 },
  { id: 'IY', label: 'IY', x: 200, y: 5140 },
  { id: 'IZ', label: 'IZ', x: 400, y: 5140 },
  { id: 'JA', label: 'JA', x: 600, y: 5140 },
  { id: 'JB', label: 'JB', x: 800, y: 5140 },
  { id: 'JC', label: 'JC', x: 1000, y: 5140 },
  { id: 'JD', label: 'JD', x: 200, y: 5240 },
  { id: 'JE', label: 'JE', x: 400, y: 5240 },
  { id: 'JF', label: 'JF', x: 600, y: 5240 },
  { id: 'JG', label: 'JG', x: 800, y: 5240 },
  { id: 'JH', label: 'JH', x: 1000, y: 5240 },
  { id: 'JI', label: 'JI', x: 200, y: 5340 },
  { id: 'JJ', label: 'JJ', x: 400, y: 5340 },
  { id: 'JK', label: 'JK', x: 600, y: 5340 },
  { id: 'JL', label: 'JL', x: 800, y: 5340 },
  { id: 'JM', label: 'JM', x: 1000, y: 5340 },
  { id: 'JO', label: 'JO', x: 200, y: 5440 },
  { id: 'JP', label: 'JP', x: 400, y: 5440 },
  { id: 'JQ', label: 'JQ', x: 600, y: 5440 },
  { id: 'JR', label: 'JR', x: 800, y: 5440 },
  { id: 'JS', label: 'JS', x: 1000, y: 5440 },
  { id: 'JT', label: 'JT', x: 200, y: 5540 },
  { id: 'JU', label: 'JU', x: 400, y: 5540 },
  { id: 'JV', label: 'JV', x: 600, y: 5540 },
  { id: 'JW', label: 'JW', x: 800, y: 5540 },
  { id: 'JX', label: 'JX', x: 1000, y: 5540 },
  { id: 'JY', label: 'JY', x: 200, y: 5640 },
  { id: 'JZ', label: 'JZ', x: 400, y: 5640 },
  { id: 'KA', label: 'KA', x: 600, y: 5640 },
  { id: 'KB', label: 'KB', x: 800, y: 5640 },
  { id: 'KC', label: 'KC', x: 1000, y: 5640 },
  { id: 'KD', label: 'KD', x: 200, y: 5740 },
  { id: 'KE', label: 'KE', x: 400, y: 5740 },
  { id: 'KF', label: 'KF', x: 600, y: 5740 },
  { id: 'KG', label: 'KG', x: 800, y: 5740 },
  { id: 'KH', label: 'KH', x: 1000, y: 5740 },
  { id: 'KI', label: 'KI', x: 200, y: 5840 },
  { id: 'KJ', label: 'KJ', x: 400, y: 5840 },
  { id: 'KK', label: 'KK', x: 600, y: 5840 },
  { id: 'KL', label: 'KL', x: 800, y: 5840 },
  { id: 'KM', label: 'KM', x: 1000, y: 5840 },
  { id: 'KO', label: 'KO', x: 200, y: 5940 },
  { id: 'KP', label: 'KP', x: 400, y: 5940 },
  { id: 'KQ', label: 'KQ', x: 600, y: 5940 },
  { id: 'KR', label: 'KR', x: 800, y: 5940 },
  { id: 'KS', label: 'KS', x: 1000, y: 5940 },
  { id: 'KT', label: 'KT', x: 200, y: 6040 },
  { id: 'KU', label: 'KU', x: 400, y: 6040 },
  { id: 'KV', label: 'KV', x: 600, y: 6040 },
  { id: 'KW', label: 'KW', x: 800, y: 6040 },
  { id: 'KX', label: 'KX', x: 1000, y: 6040 },
  { id: 'KY', label: 'KY', x: 200, y: 6140 },
  { id: 'KZ', label: 'KZ', x: 400, y: 6140 },
  { id: 'LA', label: 'LA', x: 600, y: 6140 },
  { id: 'LB', label: 'LB', x: 800, y: 6140 },
  { id: 'LC', label: 'LC', x: 1000, y: 6140 },
  { id: 'LD', label: 'LD', x: 200, y: 6240 },
  { id: 'LE', label: 'LE', x: 400, y: 6240 },
  { id: 'LF', label: 'LF', x: 600, y: 6240 },
  { id: 'LG', label: 'LG', x: 800, y: 6240 },
  { id: 'LH', label: 'LH', x: 1000, y: 6240 },
  { id: 'LI', label: 'LI', x: 200, y: 6340 },
  { id: 'LJ', label: 'LJ', x: 400, y: 6340 },
  { id: 'LK', label: 'LK', x: 600, y: 6340 },
  { id: 'LL', label: 'LL', x: 800, y: 6340 },
  { id: 'LM', label: 'LM', x: 1000, y: 6340 },
  { id: 'LO', label: 'LO', x: 200, y: 6440 },
  { id: 'LP', label: 'LP', x: 400, y: 6440 },
  { id: 'LQ', label: 'LQ', x: 600, y: 6440 },
  { id: 'LR', label: 'LR', x: 800, y: 6440 },
  { id: 'LS', label: 'LS', x: 1000, y: 6440 },
  { id: 'LT', label: 'LT', x: 200, y: 6540 },
  { id: 'LU', label: 'LU', x: 400, y: 6540 },
  { id: 'LV', label: 'LV', x: 600, y: 6540 },
  { id: 'LW', label: 'LW', x: 800, y: 6540 },
  { id: 'LX', label: 'LX', x: 1000, y: 6540 },
  { id: 'LY', label: 'LY', x: 200, y: 6640 },
  { id: 'LZ', label: 'LZ', x: 400, y: 6640 },
  { id: 'MA', label: 'MA', x: 600, y: 6640 },
  { id: 'MB', label: 'MB', x: 800, y: 6640 },
  { id: 'MC', label: 'MC', x: 1000, y: 6640 },
  { id: 'MD', label: 'MD', x: 200, y: 6740 },
  { id: 'ME', label: 'ME', x: 400, y: 6740 },
  { id: 'MF', label: 'MF', x: 600, y: 6740 },
  { id: 'MG', label: 'MG', x: 800, y: 6740 },
  { id: 'MH', label: 'MH', x: 1000, y: 6740 },
  { id: 'MI', label: 'MI', x: 200, y: 6840 },
  { id: 'MJ', label: 'MJ', x: 400, y: 6840 },
  { id: 'MK', label: 'MK', x: 600, y: 6840 },
  { id: 'ML', label: 'ML', x: 800, y: 6840 },
  { id: 'MM', label: 'MM', x: 1000, y: 6840 },
  { id: 'MO', label: 'MO', x: 200, y: 6940 },
  { id: 'MP', label: 'MP', x: 400, y: 6940 },
  { id: 'MQ', label: 'MQ', x: 600, y: 6940 },
  { id: 'MR', label: 'MR', x: 800, y: 6940 },
  { id: 'MS', label: 'MS', x: 1000, y: 6940 },
  { id: 'MT', label: 'MT', x: 200, y: 7040 },
  { id: 'MU', label: 'MU', x: 400, y: 7040 },
  { id: 'MV', label: 'MV', x: 600, y: 7040 },
  { id: 'MW', label: 'MW', x: 800, y: 7040 },
  { id: 'MX', label: 'MX', x: 1000, y: 7040 },
  { id: 'MY', label: 'MY', x: 200, y: 7140 },
  { id: 'MZ', label: 'MZ', x: 400, y: 7140 },
  { id: 'NA', label: 'NA', x: 600, y: 7140 },
  { id: 'NB', label: 'NB', x: 800, y: 7140 },
  { id: 'NC', label: 'NC', x: 1000, y: 7140 },
  { id: 'ND', label: 'ND', x: 200, y: 7240 },
  { id: 'NE', label: 'NE', x: 400, y: 7240 },
  { id: 'NF', label: 'NF', x: 600, y: 7240 },
  { id: 'NG', label: 'NG', x: 800, y: 7240 },
  { id: 'NH', label: 'NH', x: 1000, y: 7240 },
  { id: 'NI', label: 'NI', x: 200, y: 7340 },
  { id: 'NJ', label: 'NJ', x: 400, y: 7340 },
  { id: 'NK', label: 'NK', x: 600, y: 7340 },
  { id: 'NL', label: 'NL', x: 800, y: 7340 },
  { id: 'NM', label: 'NM', x: 1000, y: 7340 },
  { id: 'NO', label: 'NO', x: 200, y: 7440 },
  { id: 'NP', label: 'NP', x: 400, y: 7440 },
  { id: 'NQ', label: 'NQ', x: 600, y: 7440 },
  { id: 'NR', label: 'NR', x: 800, y: 7440 },
  { id: 'NS', label: 'NS', x: 1000, y: 7440 },
  { id: 'NT', label: 'NT', x: 200, y: 7540 },
  { id: 'NU', label: 'NU', x: 400, y: 7540 },
  { id: 'NV', label: 'NV', x: 600, y: 7540 },
  { id: 'NW', label: 'NW', x: 800, y: 7540 },
  { id: 'NX', label: 'NX', x: 1000, y: 7540 },
  { id: 'NY', label: 'NY', x: 200, y: 7640 },
  { id: 'NZ', label: 'NZ', x: 400, y: 7640 },
  { id: 'OA', label: 'OA', x: 600, y: 7640 },
  { id: 'OB', label: 'OB', x: 800, y: 7640 },
  { id: 'OC', label: 'OC', x: 1000, y: 7640 },
  { id: 'OD', label: 'OD', x: 200, y: 7740 },
  { id: 'OE', label: 'OE', x: 400, y: 7740 },
  { id: 'OF', label: 'OF', x: 600, y: 7740 },
  { id: 'OG', label: 'OG', x: 800, y: 7740 },
  { id: 'OH', label: 'OH', x: 1000, y: 7740 },
  { id: 'OI', label: 'OI', x: 200, y: 7840 },
  { id: 'OJ', label: 'OJ', x: 400, y: 7840 },
  { id: 'OK', label: 'OK', x: 600, y: 7840 },
  { id: 'OL', label: 'OL', x: 800, y: 7840 },
  { id: 'OM', label: 'OM', x: 1000, y: 7840 },
  { id: 'ON', label: 'ON', x: 200, y: 7940 },
  { id: 'OP', label: 'OP', x: 400, y: 7940 },
  { id: 'OQ', label: 'OQ', x: 600, y: 7940 },
  { id: 'OR', label: 'OR', x: 800, y: 7940 },
  { id: 'OS', label: 'OS', x: 1000, y: 7940 },
  { id: 'OT', label: 'OT', x: 200, y: 8040 },
  { id: 'OU', label: 'OU', x: 400, y: 8040 },
  { id: 'OV', label: 'OV', x: 600, y: 8040 },
  { id: 'OW', label: 'OW', x: 800, y: 8040 },
  { id: 'OX', label: 'OX', x: 1000, y: 8040 },
  { id: 'OY', label: 'OY', x: 200, y: 8140 },
  { id: 'OZ', label: 'OZ', x: 400, y: 8140 },
  { id: 'PA', label: 'PA', x: 600, y: 8140 },
  { id: 'PB', label: 'PB', x: 800, y: 8140 },
  { id: 'PC', label: 'PC', x: 1000, y: 8140 },
  { id: 'PD', label: 'PD', x: 200, y: 8240 },
  { id: 'PE', label: 'PE', x: 400, y: 8240 },
  { id: 'PF', label: 'PF', x: 600, y: 8240 },
  { id: 'PG', label: 'PG', x: 800, y:
```

```

        vis.add(v);
        stack.push(v);
        steps.push({ type: 'visit', node: v, desc: `Зашли в ${v}`,
            visitedNodes: Array.from(vis) });

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                steps.push({ type: 'process', node: v, desc: `Процесс в ${v}`,
                    .stack], visitedNodes: Array.from(vis) });
                dfs(u);
            }
        }

        stack.pop();
        steps.push({ type: 'backtrack', node: v, desc: `Вышли из ${v}`,
            ueOrStack: [...stack], visitedNodes: Array.from(vis) });
    }

    dfs(start);
    return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начали с ${start}`,
        ray.from(vis)] });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Действие в ${v}`,
            q], visitedNodes: [...Array.from(vis)] });
        q.shift();
    }
}

```

```

        });
      }, 1200);
      return () => clearInterval(t);
    }
  }, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
  <div className="space-y-4">
    <div className="flex bg-slate-900 border border-slate-800 p-4">
      <button onClick={() => setMode("dfs")}
        className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
        DFS (В глубину)
      </button>
      <button onClick={() => setMode("bfs")}
        className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
        BFS (В ширину / Волна)
      </button>
    </div>

    <div className="flex flex-col md:flex-row gap-4">
      <div /* SVG Graph View */
        <div className="bg-slate-900 border border-slate-800 p-4"
          style={{minHeight: '300px'}}>
          <svg className="w-full h-full max-w-[600px] max-h-[300px]">
            <Edges /* Edges */ />
            {edges.map((e, i) => {
              const u = nodes.find(n => n.id === e.u);
              const v = nodes.find(n => n.id === e.v);
              // Check if edge is part of current step
              const edgeTraversed = isVisited(e.u) || isVisited(e.v);
            })}
          </svg>
        </div>
      </div>
    </div>
  </div>
);

```

```

                                { /* Ripple for BFS */
                                {current && mode === 'bfs' ? 'bg-slate-500' : 'bg-slate-400'}
                                <circle cx={n.x} cy={n.y} r={10}
                                className="stroke-emerald-500 stroke-width-2"/>
                                )}
                                </g>
                            )
                        }}}}
                    </svg>
                </div>

                { /* Structure View (Stack / Queue) */ }
                <div className="bg-slate-900 border border-emerald-500 p-10" >
                    <div className={`absolute -top-3 left-3 right-3 bottom-3
                        ${mode === 'dfs' ? 'bg-slate-800' : 'bg-slate-900'}
                        border border-emerald-500`} `}>
                        {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
                    </div>

                    <div className="flex flex-col gap-2 mt-5" >
                        {step.queueOrStack && step.queueOrStack.length > 0 ?
                            <div key={idx} className={`w-full
                                ${mode === 'dfs' ? 'bg-slate-800' : 'bg-slate-900'}
                                text-emerald-200`} >
                                {idx === (mode === 'dfs' ? step.queue.length - 1 : step.queueOrStack.length - 1)
                                    ? `Узел: {item}`
                                    : ''}
                                <span className="ml-2" >{item}
                                </span>
                            </div>
                            : ''
                        }
                    </div>
                </div>
            </div>
        </div>
    </div>
    <div className="text-slate-500" >
        {(!step.queueOrStack || step.queueOrStack.length === 0) ? 'Очередь пуста' : ''}
    </div>

```

}

ФАЙЛ 40/82: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';

```
interface Patient {  
  id: string;  
  name: string;  
  emoji: string;  
  symptom: string;  
  priority: number; // 1 to 99 (Severity)  
  animState: 'in' | 'idle' | 'winner' | 'out';  
}
```

```
// Max-heap helpers for Patients  
function siftUp(arr: Patient[]): Patient[] {  
  const a = arr.map(x => ({ ...x }));  
  let idx = a.length - 1;  
  while (idx > 0) {  
    const parent = Math.floor((idx - 1) / 2);  
    if (a[parent].priority < a[idx].priority) {  
      [a[parent], a[idx]] = [a[idx], a[parent]];  
      idx = parent;  
    } else break;  
  }  
  return a;  
}
```

```
function siftDown(arr: Patient[]): Patient[] {  
  const a = arr.map(x => ({ ...x }));  
  const n = a.length;  
  let idx = 0;  
  while (idx < n) {  
    let left = 2 * idx + 1;  
    let right = 2 * idx + 2;  
    let largest = idx;  
    if (left < n && a[left].priority > a[largest].priority) {  
      largest = left;  
    }  
    if (right < n && a[right].priority > a[largest].priority) {  
      largest = right;  
    }  
    if (largest !== idx) {  
      [a[idx], a[largest]] = [a[largest], a[idx)];  
      idx = largest;  
    }  
  }  
  return a;  
}
```

```
-----
const INITIAL_PATIENTS: Patient[] = (() => {
  let arr: Patient[] = [];
  const start = [
    { name: 'Иван (Инфаркт)', symptom: ' Болят сердце',
    { name: 'Маша (Перелом)', symptom: ' Открытый перелом',
    { name: 'Семён (Температура)', symptom: ' Жар 39.5',
    { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля',
    { name: 'Оля (Порез)', symptom: ' Глубокий порез',
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emotion: ' Грусть'
  ];
  start.forEach((p, i) => {
    arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0.5,
  });
  return arr;
})();
```

```
let uid = 300;
```

```
export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>([]);
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(0);
  const [log, setLog] = useState<string[]>([]);
  const [shakeEmpty, setShake] = useState(false);
  const [busy, setBusy] = useState(false);
  const [healingPatient, setHealing] = useState<Patient>({});
```

```
const addLog = (msg: string) => setLog(prev => [...prev, msg],
```

```
// --- INSERT: Новый пациент поступает в приемный покой
```

```
const insertPatient = useCallback((name: string, priority: number) => {
  if (busy || heap.length >= 15) {
    addLog('⚠ Все палаты переполнены! Дождитесь выписки');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);
```

```

    if (!customName.trim()) return;
    insertPatient(customName, customPriority);
    setCustomName('');
  });

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
  if (busy || heap.length === 0) {
    setShake(true);
    addLog('⚠ Нет пациентов для экстренной госпитализации');
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);
  const topPatient = heap[0];
  setHealing(topPatient);

  addLog(`  СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

  // Step 1: Root lights up as winner
  setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 1 } : x));

  setTimeout(() => {
    // Step 2: Root exits the tree, tree reorganizes
    setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 0 } : x));

    setTimeout(() => {
      // Run binary heap extraction
      setHeap(prev => {
        if (prev.length === 0) return [];
        const a = prev.map(x => ({ ...x }));
        a[0] = { ...a[a.length - 1] };
        a.pop();
        return siftDown(a).map(x => ({ ...x, animStat: 0 }));
      });

      // Patient gets healed in the bed
      setTimeout(() => {

```



```

        <line key={`er${idx}`}
          x1={`${p.x}%`} y1={`${p.y + 4}%`}
          x2={`${rp.x}%`} y2={`${rp.y - 4}%`}
          stroke="#475569" strokeWidth="2"
        />
      );
    }
    return res;
  });

return (
  <div className="flex flex-col gap-6">
    { /* МНЕМОНИКА / ПРАВИЛО */ }
    <div className="bg-emerald-950/40 border border-e
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-emerald-400 te
          ty Queue)</h3>
        <p className="text-xs text-slate-300 mt-1 lea
          В реанимации не важно, кто пришел раньше, а
          остояние</b> (максимальный приоритет).
          Куча (Heap) автоматически перестраивает пац
          оказывался на самом верху дерева (в корне).
        </p>
      </div>
    </div>

    { /* УПРАВЛЕНИЕ */ }
    <div className="flex flex-col lg:flex-row items-s

      { /* Добавить случайного */ }
      <button
        onClick={handleRandomInsert}
        disabled={busy || heap.length >= 15}
        className="flex-1 min-w-[150px] bg-gradient-t
          opacity-40 disabled:cursor-not-allowed text
          ve:scale-95 transition-all cursor-pointer f
      >

```

```

disabled={busy || heap.length === 0}
className="flex-1 min-w-[170px] bg-gradient-to
40 disabled:cursor-not-allowed text-white fr
transition-all cursor-pointer flex items-c
>
  <span> В реанимацию (EXTRACT MAX)</span>
</button>

<button
  onClick={reset}
  className="px-5 py-3.5 rounded-2xl bg-slate-800
    r-pointer border border-slate-700"
>
  Сброс
</button>
</div>

{/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
<div className="grid grid-cols-1 lg:grid-cols-12" >

  {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
  <div className={`lg:col-span-8 bg-slate-900 rounded
    -[380px] overflow-hidden ${shakeEmpty ? 'anim
  <div className="absolute top-4 left-4 text-[14px]
    Сортировка по тяжести (Двоичная куча / Max-Heap)
  </div>

  <div className="relative w-full h-full min-h-[380px]
    <svg className="absolute inset-0 w-full h-full
      {edges}
    </svg>

    {heap.map((patient, idx) => {
      const pos = nodePos(idx);
      const isRoot = idx === 0;
      return (
        <div
          key={patient.id}

```

```

<div className="absolute top-4 left-4 text-[10px] text-white">
  Операционная койка
</div>

```

```

<div className="flex-1 flex flex-col justify-between">
  {healingPatient ? (
    <div className="flex flex-col items-center">
      <div className="relative">
        <span className="text-6xl block animate-pulse text-white">
          <span className="absolute -top-4 -right-4 text-white">
            <span className="absolute -bottom-2 -right-4 text-white">



```

```

<div className="w-full h-3 bg-gradient-to-r from-emerald-400 to-rose-500">
</div>

```

```

</div>

```

Универсальное пособие • полный исходный код

```
-----
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу – мо...
export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
  const [step, setStep] = useState(0);
  const paused = useContext(DemoPauseContext);
  const period = Math.round(ms * SPEED);

  useEffect(() => {
    if (steps <= 1 || paused) return;
    const t = setInterval(() => setStep((s) => (s + 1) % steps), period);
    return () => clearInterval(t);
  }, [steps, period, paused]);

  return step;
}

export const C = {
  idle: "#334155",
  idleStroke: "#475569",
  text: "#e2e8f0",
  dim: "#94a3b8",
  active: "#6366f1",
  done: "#10b981",
  warn: "#f59e0b",
  bad: "#f43f5e",
  edge: "#475569",
};

export const Svg: React.FC<{ children: React.ReactNode;
  const paused = useContext(DemoPauseContext);
  return (
    <div className="w-full">
      <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
        {children}
      </svg>
    </div>
  );
}
```

```
<line
  x1={a[0]}
  y1={a[1]}
  x2={b[0]}
  y2={b[1]}
  stroke={color}
  strokeWidth={width}
  strokeDasharray={dashed ? "4 3" : undefined}
  style={{ transition: MOVE }}
/>
);
```

ФАЙЛ 42/82: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7
  const pos: Record<string, [number, number]> = { i: [4, 9], j: [9, 4] };
  const viaOpen = step >= 1;
  const relaxed = step >= 2;
  return (
    <Svg
      caption={
        step === 0
          ? "k ещё запрещена: путь i → j = 9"
          : step === 1
            ? "Разрешаем пересадку через k: 3 + 4 = 7"
            : "7 < 9 → D[i][j] = 7"
          }
    />
  );
```

```

<Svg caption={step === 0 ? "Граф распался на острова" : "Граф распался на острова"}
  {comps.flatMap((edges, ci) =>
    edges.map(([u, v], i) => {
      <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]} />
    })
  )}
  {Object.entries(pos).map(([k, [x, y]]) => {
    const ci = compOf(k);
    return <Node key={k} x={x} y={y} r={12} label={ci} />
  })}
</Svg>
);
};

```

/** Что такое граф: вершины, рёбра, направление и вес.

```

export const GraphBasicsDemo: React.FC = () => {
  const step = useLoop(3, 1300);
  const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50], C: [250, 50], D: [350, 50], E: [450, 50], F: [550, 50], G: [650, 50], H: [750, 50], I: [850, 50], J: [950, 50] };
  const edges: [string, string, number][] = [
    ["A", "B", 1],
    ["B", "C", 1],
    ["C", "D", 1],
    ["D", "E", 1],
    ["E", "F", 1],
    ["F", "G", 1],
    ["G", "H", 1],
    ["H", "I", 1],
    ["I", "J", 1],
    ["A", "C", 2],
    ["C", "E", 2],
    ["E", "G", 2],
    ["G", "I", 2],
    ["I", "J", 2],
    ["A", "E", 3],
    ["E", "I", 3],
    ["I", "J", 3],
    ["A", "I", 4],
    ["I", "J", 4],
    ["A", "J", 5],
    ["J", "J", 6]
  ];
  return (
    <Svg
      caption={
        step === 0
          ? "Вершины – объекты (города, слова, состояния)"
          : step === 1
            ? "Рёбра – связи между ними"
            : "Вес ребра – цена перехода"
      }
    >
      {edges.map(([u, v, w], i) => {
        const a = pos[u];
        const b = pos[v];
        return (
          <g key={i}>
            <Edge a={a} b={b} color={step >= 1 ? C.active : C.inactive} />
            {step >= 2 && (
              <text
                x={{(a[0] + b[0]) / 2}

```

```

    <g key={i}>
      <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={boxH} />
      <text x={startX + i * (boxW + 4) + boxW / 2} y={40} textAnchor="middle">
        {v}
      </text>
    </g>
  )))

{step >= 1 &&
  sorted.map(({v, i}) => (
    <g key={v}>
      <rect x={30 + i * 52} y={62} width={46} height={30} />
      <text x={53 + i * 52} y={77} textAnchor="middle">
        {v}
      </text>
    </g>
  )))

{step >= 2 &&
  raw.map(({v, i}) => (
    <g key={`c${i}`}>
      <rect x={startX + i * (boxW + 4)} y={98} width={boxW} height={boxH} />
      <text x={startX + i * (boxW + 4) + boxW / 2} y={120} textAnchor="middle">
        {sorted.indexOf(v)}
      </text>
    </g>
  )))
</Svg>
);
};

/* ----- Splay: три случая поворота -----

type Pt = [number, number];

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, string][];
  captions: string[];

```

```

        { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p
        { pos: { x: [130, 32], p: [182, 92] }, edges: [{"
    ]}
  />
};

/** Zig-Zig – x и p с одной стороны, крутим сначала вер
export const ZigZigDemo: React.FC = () => {
  <SplayFrames
    captions=[["Линия g → p → x («бамбук»)", "Поворот В
    frames=[
      { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
      { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
      { pos: { x: [102, 24], p: [152, 70], g: [202, 112
    ]}
  />
};

/** Zig-Zag – x и p с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
  <SplayFrames
    captions=[["Зигзаг: g влево, x вправо", "Поворот НИ
    frames=[
      { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
      { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
      { pos: { x: [130, 30], p: [80, 100], g: [180, 100
    ]}
  />
};

```

ФАЙЛ 43/82: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

```



```

        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
      )}
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
      <Node key={k} x={x} y={y} label={k} fill={k === "trousers" ? "#f08080" : "#add8e6"} />
    ))}
  </Svg>
);
};

```

```

export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "трусы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["трусы", "штаны"],
    ["штаны", "боты"],
    ["носки", "боты"]
  ];
  const order = ["трусы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, [n.x, n.y]]));
  return (
    <Svg caption={`Порядок: ${placed.join(" → ") || "..."} />
    {edges.map(([u, v], i) => (
      <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
    ))}
    {nodes.map((n) => {
      const idx = placed.indexOf(n.id);
      return (
        <g key={n.id}>
          <rect x={n.x - 34} y={n.y - 12} width={68} height={24} fill={n.fill} />
          <text x={n.x} y={n.y} textAnchor="middle" dx={0} dy={0}>{n.id}</text>
          {idx >= 0 && <circle cx={n.x + 30} cy={n.y - 12} r={12} fill={placed[idx]} />
          {idx >= 0 && (
            <text x={n.x + 30} y={n.y - 12} textAnchor="middle" dx={0} dy={0}>{placed[idx]}</text>
          )}
        </g>
      );
    })}
  );
};

```

```

        return <Edge key={i} a={pos[u]} b={pos[v]} color={color} />
      )}
    {Object.entries(pos).map(([k, [x, y]]) => (
      <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "blue"} />
    ))}
  </Svg>
);
};

```

```

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
  };
  const edges: [string, string][] = [["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"]];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
      {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : (
        <Edge key={i} a={pos[u]} b={pos[v]} color={cut ? "red" : "blue"} />
      )))}
      {Object.entries(pos).map(([k, [x, y]]) => (
        cut && k === "C" ? null : (
          <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "blue"} />
        )
      ))}
    </Svg>
  );
};

```

```

export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100], E: [190, 100]
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];

```



```

const step = useLoop(parents.length, 950);
const p = parents[step];
const pos: [number, number][] = [[50, 95], [105, 95],
const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
const root = (i: number): number => (p[i] === i ? i : p[i]);
return (
  <Svg caption="find = «кто твой староста», union = 0"
    {p.map((par, i) =>
      par !== i ? {
        <path key={i} d={`M ${pos[i][0]} ${pos[i][1]}
          fill="none" stroke={rootColor[root(i)]} stroke-width=2
        } : null
      )}
    {pos.map(([x, y], i) => (<Node key={i} x={x} y={y}
  </Svg>
);
};

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

export const TrieDemo: React.FC = () => {
  const step = useLoop(TRIE_NODES.length + 1, 700);
  return (
    <Svg caption="Путь от корня по буквам = слово; общий путь = префикс"
      {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]}
      })}
      {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x} y={n.y}
    </Svg>
  );
};

```

```

    })))
    {TRIE_NODES.map((n) => {
      <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
    })}
  </Svg>
);
};

```

```

export const SegmentTreeDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const nodes = [
    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
    { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
    { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых"
      {nodes.map((n, i) => {
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w}
            fill={n.lvl === step ? C.active : C.idle} s
          <text x={n.x} y={n.y} textAnchor="middle" dom
        </g>
      })}
    </Svg>
  );
};

```

```

export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;

```

```

    {Array.from({ length: count }).map((_, i) => {
      <rect key={i} x={12 + i * 30} y={52 + (i % 4) *
        fill={C.active} opacity={i === 0 ? 1 : 0.4} s
    }}}
  </Svg>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b);
  const avg = step ? (total / step).toFixed(2) : "-";
  return (
    <Svg caption={`Сделано ${step} операций, суммарно $
      {costs.map((c, i) => {
        <rect key={i} x={14 + i * 30} y={104 - c * 10} w
        fill={i < step ? (c > 2 ? C.bad : C.done) : C
      }}}
    <line x1={8} y1={106} x2={252} y2={106} stroke={C
    <text x={130} y={20} textAnchor="middle" fontSize=
  </Svg>
);
};

```

```

export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (
    <Svg caption={step === 0 ? "Найти решение – долго (
      <ellipse cx={130} cy={64} rx={112} ry={48} fill="
      <ellipse cx={92} cy={64} rx={52} ry={32} fill="#3
      <text x={92} y={64} textAnchor="middle" dominantB
      <text x={206} y={34} textAnchor="middle" fontSize
      <circle cx={196} cy={84} r={13} fill={step === 1
      <text x={196} y={84} textAnchor="middle" dominant
        ?"></text>
      <text x={130} y={124} textAnchor="middle" fontSize
    </Svg>
  );
};

```

```

        const idx = r * cols + c;
        const filled = idx < step;
        return (
          <g key={idx}>
            <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
              fill={idx === step - 1 ? C.active : filled} fillOpacity={
                filled && {
                  <text x={57 + c * 38} y={36 + r * 32} text={C[idx]}
                    fontSize={10} fontWeight="700" fill="black"/>
                }
              }
            </g>
          );
        })
      )
    <text x={130} y={122} textAnchor="middle" fontSize={14}
      каждая клетка считается один раз и переиспользуется
    </text>
  </Svg>
);
};

```

ФАЙЛ 45/82: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
  import {
    AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
    SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
  } from "../demosStruct";
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

```

```
};
```

```
export const MiniDemo: React.FC<{ kind: DemoKind }> = ({
  const [paused, setPaused] = useState(false);
  const Cmp = REGISTRY[kind];
  if (!Cmp) return null;
  return (
    <DemoPauseContext.Provider value={paused}>
      <div
        className="bg-slate-950/70 rounded-lg border bo
        onMouseEnter={() => setPaused(true)}
        onMouseLeave={() => setPaused(false)}
      >
        <Cmp />
      </div>
    </DemoPauseContext.Provider>
  );
};
```

ФАЙЛ 46/82: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
```

```
interface Props {
  vizId: string | null;
  onClose: () => void;
}
```

```
/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
  useEffect(() => {
```



```
    </div>,
    document.body
  );
};
```

ФАЙЛ 47/82: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}
```

```
interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
  onCardLeave: () => void;
}
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor,
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: n
```

```

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 0),
      visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
  >
    <div className="bg-slate-900 border border-indigo-500" >
      <div className="flex items-start gap-2 px-3 pt-3" >
        <h4 className="text-sm font-bold text-indigo-400" >Подсказка
        <button
          onClick={onClose}
          className="text-slate-500 hover:text-white"
          aria-label="Заккрыть подсказку"
        >
          <X className="w-4 h-4" />
        </button>
      </div>

      <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
        <p className="text-[13px] leading-relaxed text-slate-300">
          {entry.demo && <MiniDemo kind={entry.demo} />}

          {entry.formal && (
            <p className="text-[11.5px] leading-relaxed text-slate-300">
              {entry.formal}
            </p>
          )}

          {entry.complexity && (

```

```

    { id: 'C', x: 80, y: 220, label: 'C' },
  ];

const edges = [
  { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
  { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
  { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

const isNodeVisible = (id: string) => {
  if (id === 'S' && (step === 0 || step === 7)) return true;
  return false;
};

const getPotential = (id: string) => {
  if (step >= 2 && step < 7) {
    if (id === 'S') return 0;
    if (id === 'A') return 0;
    if (id === 'B') return -2;
    if (id === 'C') return -1;
  }
  return null;
};

const getEdgeColorClass = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "opacity-0";
    if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
    if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
    return "stroke-slate-700 stroke-dasharray-4";
  }

  if (e.id === 'AB') {
    if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 4) return "stroke-emerald-500";
  }
};
```

```

        if (step === 4) return `${e.weight} + (${0})
        return e.newWeight;
    }
    if (e.id === 'BC') {
        if (step < 5) return e.weight;
        if (step === 5) return `${e.weight} + (${-2})
        return e.newWeight;
    }
    if (e.id === 'CA') {
        if (step < 6) return e.weight;
        if (step === 6) return `${e.weight} + (${-1})
        return e.newWeight;
    }
    return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
    if (e.id === 'AB' && step === 4) return true;
    if (e.id === 'BC' && step === 5) return true;
    if (e.id === 'CA' && step === 6) return true;
    return false;
};

const isEdgeTextEmerald = (e: any) => {
    if (e.id === 'AB' && step > 4) return true;
    if (e.id === 'BC' && step > 5) return true;
    if (e.id === 'CA' && step > 6) return true;
    return false;
};

const getDesc = () => {
    switch (step) {
        case 0: return (
            <div>
                <b>Исходный граф.</b> Имеется ребро
                Если мы запустим быстрый алгоритм Д
            </div>
        );
    }
};

```

```

        case 6: return (
            <div>
                <b>Шаг 3.3: Пересчет ребра C → A.</b>
                Старый вес: 2. <br/>
                Вычисляем:  $2 + (-1) - 0 =$  <span cla
            </div>
        );
        case 7: return (
            <div className="text-emerald-300">
                <b>Готово!</b> Фиктивную вершину S
                Все новые веса ребер в графе стали
                При этом старые кратчайшие пути оста
            </div>
        );
        default: return "";
    }
};

return (
    <div className="bg-slate-800 p-4 rounded-xl border">
        <h4 className="text-sm md:text-base font-bo">
            <span className="w-2.5 h-2.5 bg-amber-400">
                Визуализация: Перевзвешивание алгоритмом
            </h4>

        <div className="flex flex-col md:flex-row gap-4">
            <div className="bg-[#0f172a] flex-1 min-h-100 border p-4">
                <svg width="400" height="300" class="dark">
                    <defs>
                        <marker id="arrow-slate" viewBox="0 0 10 5" fill="white"
                            start-reverse">
                            <path d="M 0 0 L 10 5 L 10 0" />
                        </marker>
                        <marker id="arrow-slate-dim" viewBox="0 0 10 5" fill="white"
                            uto-start-reverse">
                            <path d="M 0 0 L 10 5 L 10 0" />

```

```

        <text
          x={midX} y={midY}
          textAnchor="middle"
          dominantBaseline="bottom"
          className={`text ${
            textHeight > 0
              ? 'text-emergent'
              : 'text-faded'
          }`}
          >{getEdgeContent(edge)}
        </text>
      </g>
    )
  })
}

{/* Nodes */}
{nodes.map(n => {
  if (!isNodeVisible(n.id)) return null
  const pot = getPotential(n.id)

  return (
    <g key={n.id} className="node">
      <circle
        cx={n.x} cy={n.y}
        className={`stroke-${
          n.id === 'S'
            ? 'fill-1'
            : 'fill-slate'
        }`}
      />
      <text x={n.x} y={n.y}
        >{n.label}
      </text>

      {pot !== null && (
        <g transform={`translate(0, ${
          pot > 0 ? 10 : -10
        })`}
          <rect x="-10" y="0" width="20" height="20"
            style={
              pot > 0
                ? {fill: 'white', stroke: 'black', stroke-width: 1px}
                : {fill: 'white', stroke: 'black', stroke-width: 1px}
            }
          />
        <rect x="10" y="0" width="20" height="20"
            style={
              pot < 0
                ? {fill: 'white', stroke: 'black', stroke-width: 1px}
                : {fill: 'white', stroke: 'black', stroke-width: 1px}
            }
          />
      )}
    </g>
  )
})}

```

```
                {step !== MAX_STEP && <Skip  
                </button>  
            </div>  
  
            {/* Progress Dots */}  
            <div className="flex justify-center  
                {Array.from({length: MAX_STEP +  
                    <div key={i} className={`w-  
125' : i < step ? 'bg-indigo-900' : 'bg-sla  
                }}}  
            </div>  
        </div>  
    </div>  
</div>  
);  
}
```

ФАЙЛ 49/82: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from "react";  
import {  
    Play,  
    Pause,  
    RotateCcw,  
    Layers,  
} from "lucide-react";  
  
interface GNode {  
    id: number;  
    x: number;  
    y: number;  
    label: string;  
}
```

```

useEffect(() => {
  // Generate step-by-step Kosaraju steps
  const generatedSteps: AlgoStep[] = [];
  const visited = new Set<number>();
  const order: number[] = [];
  const sccMap: Record<number, number> = {};

  const recordStep = (
    phase: AlgoStep["phase"],
    desc: string,
    currentNode: number | null,
    transposed: boolean,
    currentSccId: number,
  ) => {
    generatedSteps.push({
      phase,
      desc,
      visited: new Set(visited),
      currentNode,
      order: [...order],
      transposed,
      sccMap: { ...sccMap },
      currentSccId,
      activeEdges: transposed
        ? initialEdges.map((e) => ({ u: e.v, v: e.u })
          : [...initialEdges],
    ));
  };

  recordStep(
    "init",
    "Инициализация алгоритма Косарайю. Начинаем первый",
    null,
    false,
    -1,
  );

```



```

const topoOrder = [...order].reverse();
recordStep(
  "transpose",
  `Первый проход завершен! Стек выхода в топологическом порядке.`,
  null,
  true,
  -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
  visited.add(u);
  sccMap[u] = currentScc;
  recordStep(
    "dfs2",
    `DFS 2: заходим в ${u} на транспонированном графе`,
    u,
    true,
    currentScc,
  );

  for (const to of adjT[u]) {
    if (!visited.has(to)) {
      dfs2(to, currentScc);
    }
  }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {

```

```

    currentNode: null,
    order: [],
    transposed: false,
    sccMap: {},
    currentSccId: -1,
    activeEdges: initialEdges,
  });

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1) {
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1500);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getSccColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Layers className="w-5 h-5 text-emerald-400"
        Визуализатор Алгоритма Косарайю (Поиск SCC)

```

```

        markerHeight="6"
        orient="auto-start-reverse"
    >
        <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
<marker
    id="arrow-active"
    viewBox="0 0 10 10"
    refX="22"
    refY="5"
    markerWidth="6"
    markerHeight="6"
    orient="auto-start-reverse"
>
    <path d="M 0 1 L 10 5 L 0 9 z" fill="#1
</marker>
<marker
    id="arrow-transposed"
    viewBox="0 0 10 10"
    refX="22"
    refY="5"
    markerWidth="6"
    markerHeight="6"
    orient="auto-start-reverse"
>
    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
</defs>

```

```

{ /* Edges */ }
{initialEdges.map((edge, idx) => {
    const u = initialNodes.find((n) => n.id ==
    const v = initialNodes.find((n) => n.id ==

    // If transposed, draw reversed coordinates
    const x1 = step.transposed ? v.x : u.x;
    const y1 = step.transposed ? v.y : u.y;
    const x2 = step.transposed ? u.x : v.x;

```

```

        className="transition-all duration-300"
      />
    );
  }}}

  { /* Nodes */
  {initialNodes.map((node) => {
    const isCurrent = step.currentNode === node.id;
    const isVisited =
      step.visited.has(node.id) ||
      (step.phase === "dfs1" && step.currentNode === node.id);
    const sccId = step.sccMap[node.id];

    let classes = "fill-slate-800 stroke-slate-800";
    if (sccId) {
      classes = getSccColor(sccId);
    } else if (isCurrent) {
      classes = "fill-amber-600 stroke-amber-600";
    } else if (isVisited && step.phase === "dfs2") {
      classes = "fill-emerald-700 stroke-emerald-700";
    }

    return (
      <g key={node.id}>
        <circle
          cx={node.x}
          cy={node.y}
          r="20"
          className={`transition-all duration-300`}
          strokeWidth="3"
        />
        <text
          x={node.x}
          y={node.y}
          dy=".3em"
          textAnchor="middle"
          fill="white"
          fontSize="13px"

```

```

<div className="bg-slate-900 border border-slate-800" style={step.desc}
  <p className="text-xs text-white leading-relaxed" style={step.desc}
    {step.desc}
  </p>
</div>

<div className="flex-1 space-y-4 overflow-y-auto" style={step.desc}
  {/* Top-Sort output / Order stack representation */}
  <div>
    <span className="text-[10px] text-slate-500" style={step.desc}
      СТЕК ВЫХОДА (DFS 1 ORDER):
    </span>
    <div className="flex flex-wrap gap-1 bg-slate-900 border border-slate-800" style={step.desc}
      {step.order.length === 0 ? (
        <span className="text-slate-600 text-sm" style={step.desc}
          Пустой стек
        </span>
      ) : (
        [...step.order].reverse().map(({val, idx}) => (
          <div
            key={idx}
            className="bg-indigo-950 border border-indigo-900 text-white text-sm px-2 py-1"
            style={step.desc}
          >
            {val}
          </div>
        ))
      )}
    </div>
  </div>

  {/* List of actions log */}
  <div>
    <span className="text-[10px] text-slate-500" style={step.desc}
      ЛОГ ОПЕРАЦИЙ:
    </span>
    <div className="space-y-1.5 max-h-[150px] bg-slate-900 border border-slate-800" style={step.desc}
      {steps
        .slice(0, currentStepIdx + 1)
        .reverse()
      }
    </div>
  </div>

```

```
};
```

ФАЙЛ 50/82: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Checkmark } from 'lucide-react';
```

```
interface Vertex {
  id: number;
  label: string;
  x: number;
  y: number;
  color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}
```

```
interface Edge {
  id: string;
  u: number;
  v: number;
  weight: number;
  status: 'pending' | 'inspecting' | 'included' | 'rejected';
  color: string;
}
```

```
interface StepLog {
  title: string;
  description: string;
  type: 'info' | 'success' | 'warning' | 'error';
}
```

```
// 9 Vertices layout
```

```
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 40, y: 20, color: 'none' },
  { id: 2, label: 'C', x: 60, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 80, y: 20, color: 'none' },
  { id: 4, label: 'E', x: 20, y: 40, color: 'none' },
  { id: 5, label: 'F', x: 40, y: 40, color: 'none' },
  { id: 6, label: 'G', x: 60, y: 40, color: 'none' },
  { id: 7, label: 'H', x: 80, y: 40, color: 'none' },
  { id: 8, label: 'I', x: 20, y: 60, color: 'none' },
];
```

```

        овы начинать!',
        type: 'info',
    },
]);

// --- KRUSKAL STEPS ---
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Шаг 1: Берем самое дешевое ребро A-B (в
      description: 'Оно соединяет две бесцветные верш
      type: 'info'
    }, ...prev]));
  },
  // Step 2: Include A-B
  () => {
    setVertices(prev => prev.map(v => v.id === 0 || v
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро A-B в осто́ве',
      description: 'Вершины A and B окрашены в красны
      type: 'success'
    }, ...prev]));
  },
  // Step 3: Inspect D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
      description: 'Оно соединяет две другие бесцветн
      type: 'info'
    }, ...prev]));
  },
  // Step 4: Include D-E
  () => {
    setVertices(prev => prev.map(v => v.id === 3 || v

```

```

setVertices(prev => prev.map(v => v.color === 'blue' ? v : {id: 5, color: 'blue'}));
setEdges(prev => prev.map(e => e.status === 'included' ? e : {id: '1-4', status: 'included'}));
setLogs(prev => [{
  title: 'Слияние кланов: Ребро A-D добавлено',
  description: 'Мы объединили красный и синий кланы',
  type: 'success'
}], ...prev));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {id: '1-4', status: 'rejected'} : e));
  setLogs(prev => [{
    title: 'Шаг 5: Берем ребро B-E (вес 6)',
    description: 'НО: это ребро соединяет вершины B и E, которые уже соединены!',
    type: 'warning'
  }], ...prev));
},
// Step 10: Reject B-E
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {id: '1-4', status: 'rejected'} : e));
  setLogs(prev => [{
    title: 'Отказ: Цикл обнаружен!',
    description: 'Мы выбрасываем ребро B-E. Иначе получится цикл A-B-E-D-A',
    type: 'error'
  }], ...prev));
},
// Step 11: Inspect E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {id: '4-5', status: 'included'} : e));
  setLogs(prev => [{
    title: 'Шаг 6: Берем ребро E-F (вес 7)',
    description: 'Оно соединяет фиолетовую вершину E и синюю вершину F',
    type: 'info'
  }], ...prev));
},
// Step 12: Include E-F
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {id: 5, color: 'blue'} : v));

```



```

        setLogs(prev => [{
            title: 'Отказ: Цикл обнаружен!',
            description: 'Выбрасываем ребро C-F.',
            type: 'error'
        }, ...prev]));
    },
    // Step 17: Inspect G-H (10)
    () => {
        setEdges(prev => prev.map(e => e.id === '6-7' ? {
        setLogs(prev => [{
            title: 'Шаг 9: Берем ребро G-H (вес 10)',
            description: 'Соединяет фиолетовую G и бесцветную H.',
            type: 'info'
        }, ...prev]));
    },
    // Step 18: Include G-H
    () => {
        setVertices(prev => prev.map(v => v.id === 7 ? {
        setEdges(prev => prev.map(e => e.id === '6-7' ? {
        setLogs(prev => [{
            title: 'Успех: Ребро G-H в осто́ве',
            description: 'Вершина H перекрашена в фиолетовую.',
            type: 'success'
        }, ...prev]));
    },
    // Step 19: Inspect E-H (11) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
            title: 'Шаг 10: Берем ребро E-H (вес 11)',
            description: 'Оба конца E и H уже фиолетовые. Соединение образует цикл!',
            type: 'warning'
        }, ...prev]));
    },
    // Step 20: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{

```

```

        type: 'info'
    }, ...prev]);
},
// Step 2: Pop D-E (3)
() => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
        description: 'Плесень тянется по ребру D-E к вершине E',
        type: 'info'
    }, ...prev]);
},
// Step 3: Include D-E, add D's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 3 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
        ..e, status: 'in_heap' } : e));
    setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-F (8)']);
    setLogs(prev => [{
        title: 'Успех: Плесень захватила вершину D',
        description: 'В кучу добавлены новые пути из D: A-D (5), B-E (6), E-F (7), D-F (8)',
        type: 'success'
    }, ...prev]);
},
// Step 4: Pop A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
        description: 'Плесень расширяется в сторону вершины A',
        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));

```

```

() => {
  setVertices(prev => prev.map(v => v.id === 2 ? {
  setEdges(prev => prev.map(e => e.id === '1-2' ? {
    eap' } : e));
  setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
  setLogs(prev => [{
    title: 'Успех: Вершина C захвачена',
    description: 'Ребро C-F(9) добавлено в кучу.',
    type: 'success'
  }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setLogs(prev => [{
    title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
    description: 'ВНИМАНИЕ! Плесень проверяет вершину B',
    type: 'warning'
  }, ...prev]));
},
// Step 11: Reject B-E
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
  setLogs(prev => [{
    title: 'Отказ: Игнорируем внутреннее ребро',
    description: 'Ребро B-E отброшено. Плесень не р
    type: 'error'
  }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (E) тянет
    type: 'info'
  }, ...prev]));
},

```

```

        title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
        description: 'Вершины C и F уже заражены плесенью',
        type: 'warning'
    }, ...prev]);
},
// Step 17: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
    setLogs(prev => [{
        title: 'Отказ: Игнорируем ребро C-F',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
        description: 'Плесень ползет к вершине H.',
        type: 'info'
    }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
        eap' } : e));
    setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
    setLogs(prev => [{
        title: 'Успех: Вершина H захвачена',
        description: 'В кучу добавлено H-I(12).',
        type: 'success'
    }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {

```

```

    ];

    // --- BORUVKA STEPS ---
    const boruvkaSteps = [
      // Step 1: Five-Year Plan 1 (Inspecting outgoing)
      () => {
        // Highlight the cheapest outgoing edges for EACH
        // Node 0(A) -> A-B(2)
        // Node 1(B) -> A-B(2)
        // Node 2(C) -> B-C(4)
        // Node 3(D) -> D-E(3)
        // Node 4(E) -> D-E(3)
        // Node 5(F) -> E-F(7)
        // Node 6(G) -> D-G(8)
        // Node 7(H) -> G-H(10)
        // Node 8(I) -> H-I(12)
        setEdges(prev => prev.map(e =>
          ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
        ]));
        setLogs(prev => [{
          title: 'Первая пятилетка: Каждая деревня выбирает',
          description: 'Каждая из 9 деревень одновременно выбирает',
          text: 'А выбирает A-B(2), D выбирает D-E(3), а G выбирает D-G(8)!',
          type: 'info'
        }, ...prev]);
      },
      // Step 2: Concurrently merge into Kolkhozes (Components)
      () => {
        // Concurrently build chosen roads. Merge into:
        // Component 1: {A, B, C} (colored red)
        // Component 2: {D, E, F, G, H, I} (colored gold/olive)
        setVertices(prev => prev.map(v =>
          [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
        ));
        setEdges(prev => prev.map(e => {
          if (['0-1', '1-2'].includes(e.id)) {
            return { ...e, status: 'included', color: 'red' };
          }
        }));
      }
    ];
  }
}

```

```

    },
    // Step 5: Check remaining edges & mark as rejected
    () => {
        setEdges(prev => prev.map(e => e.status === 'pend
        setLogs(prev => [{
            title: '    Успех: MST построен Борувкой за 2 шага
            description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
                вес: 51. Благодаря параллельности, мы потрати
            type: 'success'
        }, ...prev]));
    }
];

const currentSteps =
    activeAlgo === 'kruskal' ? kruskalSteps :
    activeAlgo === 'prim' ? primSteps :
    boruvkaSteps;

const handleNextStep = () => {
    if (stepIndex < currentSteps.length) {
        currentSteps[stepIndex]();
        setStepIndex(stepIndex + 1);
    }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
    setActiveAlgo(algo);
    setVertices(initialVertices);
    setEdges(initialEdges);
    setStepIndex(0);
    setHeapQueue([]);
    if (algo === 'kruskal') {
        setLogs([
            {
                title: 'Введение в раскраску (Краскал)',
                description: 'Представь, что все 9 вершин гра
                type: 'info',
            },
        ]),
    },

```

```

    if (status === 'rejected') return '#ef4444';
    if (status === 'in_heap') return '#0284c7';
    if (status === 'included') {
      if (color === 'red') return '#ef4444';
      if (color === 'blue') return '#3b82f6';
      if (color === 'purple') return '#a855f7';
      if (color === 'mold') return '#10b981';
    }
    return '#475569';
  };

return (
  <div className="space-y-12">

    {/* 3-in-1 MST Simulator Box */}
    <div className="bg-slate-900/90 border border-slate-800" >
      <div className="flex flex-col lg:flex-row justify-between" >
        <div>
          <div className="flex items-center gap-2 mb-2" >
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-weight-bold uppercase tracking-wider">
              Сунер-Интерактив 3 в 1
            </span>
            <h3 className="text-2xl font-bold text-white">
              Алгоритмы
            </h3>
          </div>
          <p className="text-slate-400 text-sm">
            Наблюдай за работой Краскала (Раскраска),
          </p>
        </div>

        {/* Algorithm Tabs */}
        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center bg-slate-800/50 p-2" >
            <button
              onClick={() => handleReset('kruskal')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
              }
            >
              Краскала
            </button>
          </div>
          <div className="flex items-center bg-slate-800/50 p-2" >
            <button
              onClick={() => handleReset('prim')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
              }
            >
              Прима
            </button>
          </div>
          <div className="flex items-center bg-slate-800/50 p-2" >
            <button
              onClick={() => handleReset('mst')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
              }
            >
              MST
            </button>
          </div>
        </div>
      </div>
    </div>
  )
)

```

```

        className="flex items-center justify-center gap-2
        -200 font-semibold text-xs rounded-xl border-1
      >
        <RotateCcw className="w-3.5 h-3.5" />
        <span>Сбросить</span>
      </button>

    <button
      onClick={handleNextStep}
      disabled={stepIndex >= currentSteps.length}
      className={
        "flex items-center justify-center gap-2
        initial cursor-pointer " +
        (stepIndex >= currentSteps.length
          ? "bg-slate-800 text-slate-500 border-1
            : activeAlgo === 'kruskal'
          ? "bg-indigo-600 hover:bg-indigo-500
            : activeAlgo === 'prim'
          ? "bg-emerald-600 hover:bg-emerald-500
            : "bg-rose-600 hover:bg-rose-500 text
        )
      }
    >
      <SkipForward className="w-4 h-4" />
      <span>{stepIndex === 0 ? 'Начать' : stepIndex + 1}</span>
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
  /* Graph Canvas */
  <div className="lg:col-span-2 bg-slate-950 rounded-2xl
    active min-h-[480px] overflow-hidden">

    /* Legend */
    <div className="absolute top-4 left-4 bg-slate-900
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5">
        <div className="w-3 h-3 rounded-full bg-
```



```

ext-slate-300">
  ⌚ {stepIndex} / {currentSteps.length}
</div>

{/* SVG Edges */}
<svg className="absolute inset-0 w-full h-full"
  spectRatio="none">
  {edges.map(edge => {
    const u = vertices[edge.u];
    const v = vertices[edge.v];
    const strokeColor = getEdgeStroke(edge);
    const isInspecting = edge.status === 'inspect';
    const isRejected = edge.status === 'rejected';
    const isInHeap = edge.status === 'in_heap';

    return (
      <g key={edge.id}>
        <line
          x1={u.x + "%"}
          y1={u.y + "%"}
          x2={v.x + "%"}
          y2={v.y + "%"}
          stroke={strokeColor}
          strokeWidth={isInspecting ? 4.5 : 1}
          strokeDasharray={isRejected ? '4,4' : ''}
          className={"transition-all duration-300"}
        />
        <text
          x={({u.x + v.x} / 2 + "%")}
          y={({u.y + v.y} / 2 + "%")}
          fill={isInspecting ? '#f59e0b' : isRejected ? '#94a3b8' : '#4a7c59'}
          fontSize="4.5"
          fontFamily="monospace"
          fontWeight="bold"
          textAnchor="middle"
          dominantBaseline="middle"
          className="select-none filter drop-shadow"
        />
      </g>
    );
  })}
</svg>

```

```

    })
  </div>

  { /* Logs / Heap */
  <div className="bg-slate-950 rounded-2xl border-2 border-b
    <div className="p-4 bg-slate-900 border-b border-slate-800
      <h4 className="font-bold text-slate-200 text-sm">
        Ход выполнения
      </h4>
    </div>

    {activeAlgo === 'prim' && {
      <div className="p-3 bg-slate-900/60 border-2 border-slate-800
        <p className="text-[11px] font-bold text-slate-200">
          ⚡ Приоритетная очередь (Min-Heap):
        </p>
        <div className="flex flex-wrap gap-1.5">
          {heapQueue.length === 0 ? {
            <span className="text-xs text-slate-200">
              Очередь пуста
            </span>
          } : {
            heapQueue.map(({item, idx}) => {
              <span key={idx} className={"px-2 py-1 text-xs font-mono"
                0 shadow-md animate-pulse" : "bg-slate-800 text-slate-200"}>
                {item}
              </span>
            })
          })
        </div>
      </div>
    })
  }

  <div className="flex-1 p-4 overflow-y-auto">
    {logs.map((log, idx) => {
      <div
        key={idx}
        className={
          "p-4 rounded-xl border transition-all duration-200"
          (log.type === 'success'

```

```

    </button>
    <button
      onClick={() => setActiveCheatTab('prim')}
      className={"px-3 py-1.5 rounded text-xs font-medium"}
      == 'prim' ? "bg-emerald-600 text-white" : "bg-white text-gray-700"
    >
      Прима (Билет 19)
    </button>
    <button
      onClick={() => setActiveCheatTab('boruvka')}
      className={"px-3 py-1.5 rounded text-xs font-medium"}
      == 'boruvka' ? "bg-rose-600 text-white" : "bg-white text-gray-700"
    >
      Боровка (Билет 20)
    </button>
  </div>
</div>

{activeCheatTab === 'kruskal' && (
  <div className="grid grid-cols-1 lg:grid-cols-2" style={{width: '100%'}}>
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-inherit" />
        </p>
        <p className="text-2xl font-black text-white">Крускал</p>
        <p className="text-slate-400 text-xs font-medium">Метод нахождения минимального остовного дерева</p>
      </div>
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Award className="w-3.5 h-3.5 text-inherit" />
        </p>
        <p className="text-2xl font-black text-white">Авард</p>
        <p className="text-slate-400 text-xs font-medium">Метод нахождения минимального остовного дерева</p>
      </div>
    </div>
    <div className="bg-slate-950 p-4 rounded-2xl">
      <p className="text-slate-400 text-xs font-medium">
        ожении: </p>

```



```

        </div>
        <div className="lg:col-span-2">
          <div className="bg-slate-950 p-4 rounded-1">
            <p className="text-slate-400 text-xs font-medium">
              <Code className="w-3.5 h-3.5 text-rose-500">
            </p>
            <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`def boruvka(n, edges):
    parent = list(range(n))
    mst = []
    while len(mst) < n - 1:
        # Для каждого колхоза ищем самый дешёвый мост на
        cheapest = [-1] * n
        for u, v, w in edges:
            ru, rv = find(u), find(v)
            if ru != rv:
                if cheapest[ru] == -1 or cheapest[ru][2] > w:
                    cheapest[ru] = (u, v, w)
                if cheapest[rv] == -1 or cheapest[rv][2] > w:
                    cheapest[rv] = (u, v, w)
        # Объединяем все колхозы по выбранным мостам
        for bridge in cheapest:
            if bridge != -1 and union(bridge[0], bridge[1]):
                mst.append(bridge)`}
            </pre>
          </div>
        </div>
      </div>
    })
  </div>
);
};

```

```

    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: '}'. Ищет пути от всех ко всем.',
    complexity: 'O(V³)',
    border: 'border-emerald-500/30 hover:border-emerald-500/30',
    titleColor: 'text-emerald-400',
    badge: 'text-emerald-400/70 bg-emerald-950/40',
  },
];

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4">
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt} className="w-full h-full object-cover grid" />
          </div>
          <div className="p-4">
            <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
            <p className="text-slate-400 text-sm">{c.desc} <span className={`font-bold ${c.complexity}`}>{c.complexity}</span>
          </p>
          <div className={`mt-3 text-xs font-mono px-2`} >{c.rest}</div>
        </div>
      ))}
    </div>
  );
}
```

```

        className="w-full flex items-center gap-2.5 p-2"
        aria-label="Открыть содержание"
      >
        <span className="p-1.5 rounded-lg bg-indigo-600 text-white">
          <List className="w-4 h-4" />
        </span>
        <span className="min-w-0 flex-1">
          <span className="block text-[10px] uppercase">
            Содержание • {index + 1} из {total}
          </span>
          <span className="block text-[13px] font-bold">
            {children}
          </span>
        </span>
      </button>
    </div>

    {open && (
      <div className="lg:hidden fixed inset-0 z-[90]">
        <div className="absolute inset-0 bg-slate-950">
          <div className="absolute inset-y-0 left-0 w-[180px] bg-white text-slate-[tocIn_.18s_ease-out]">
            <div className="flex items-center justify-between p-2">
              <span className="font-bold text-white text-[12px]">
                Содержание
              <button
                  onClick={() => setOpen(false)}
                  className="p-2 -mr-2 text-slate-400 hover:bg-slate-200"
                  aria-label="Заккрыть содержание"
                >
                  <X className="w-5 h-5" />
                </button>
            </div>
          </div>
          <div className="flex-1 min-h-0">
            <Sidebar
              selectedChapterId={selectedChapterId}
              setSelectedChapterId={setSelectedChapterId}
              onNavigate={() => setOpen(false)}
            />
          </div>
        </div>
      </div>
    )}
  </div>

```

```

<div className="flex items-center gap-3 w-full" >
  <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
    <Sparkles className="w-6 h-6" />
  </div>
  <div className="min-w-0 flex-1" >
    <h1 className="text-xl font-extrabold text-slate-900" >
      Универсальное пособие
    </h1>
    <p className="text-xs text-indigo-400 font-medium" >
      Подготовка к экзаменам: Алгоритмы, Структуры данных
    </p>
  </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-slate-900 font-medium rounded-md shadow-sm transition-colors
      duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-slate-900 font-medium rounded-md shadow-sm transition-colors
      duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <Code className="w-4 h-4" /> Симулятор
  </button>
</div>

```



```
    </div>
  </header>
);
};
```

ФАЙЛ 54/82: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотрим
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        { /* K5 */ }
        <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
          <div className="absolute top-2 left-2 bg-slate-900 text-white p-2">
            00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
          <svg className="w-full h-full absolute inset-0">
            {(() => {
              const pts = [
                {id:0,x:160,y:40},
                {id:1,x:270,y:100},
                {id:2,x:230,y:230},
                {id:3,x:90,y:230},
                {id:4,x:50,y:100}
              ];
              const edges = [
                [0,1],[0,2],[0,3],[0,4],
                [1,2],[1,3],[1,4],
                [2,3],[2,4],
                [3,4]
              ];
              function findPt(id:number) { return pts.f
```

```

    </div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border"
  <div className="absolute top-2 left-2 bg-slate-
    00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
  <svg className="w-full h-full absolute inset-0"
    {(() => {
      const pts = [
        {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
        {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
      ];
      const edges = [
        [0,3],[0,4],[0,5],
        [1,3],[1,4],[1,5],
        [2,3],[2,4],[2,5]
      ];
      function findPt(id:number) { return pts.find(p => p.id === id); }
      function intersect(a:number,b:number,c:number,d:number) {
        if (a===c||a===d||b===c||b===d) return true;
        const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
        if (!p1||!p2||!p3||!p4) return false;
        function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
          return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
          && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
      }
      const crosses:number[]=[];
      for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++)
        if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
          if (!crosses.includes(i)) crosses.push(i);
          if (!crosses.includes(j)) crosses.push(j);
      }
    )}
  </svg>
</div>

const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {

```

```

interface Customer {
  id: string;
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
  { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue', bubbleText: 'Матана!' },
  { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet', bubbleText: 'Порцию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber', bubbleText: 'Сметаны, плиз.' },
  { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose', bubbleText: 'Помогите!' },
  { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald', bubbleText: 'Помогите! Пиво для нашего НЛО!' },
  { name: 'Бизнесмен', emoji: '👔', color: 'from-slate', bubbleText: 'Тартаны.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue', bubbleText: 'Матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet', bubbleText: 'Порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber', bubbleText: 'Сметаны, плиз.', animState: 'idle' },
  ]);

  const [servedHistory, setServedHistory] = useState<st

```

```

    addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsServing(true);
  const headGuest = queue[0];

  addLog(`  DEQUEUE: Повар наливает суп первому – ${h

  // Step 1: Head guest gets eating animation
  setQueue(prev => prev.map((c, idx) => idx === 0 ? {

  setTimeout(() => {
    // Step 2: Guest leaves with happy face, queue sh
    setQueue(prev => prev.map((c, idx) => idx === 0 ?

    setServedHistory(prev => [`${headGuest.emoji} ${h

    setTimeout(() => {
      setQueue(prev => prev.slice(1));
      setIsServing(false);
      addLog(`  ${headGuest.name} поел и ушел сытым.
    }, 400);
  }, 900);
}, [queue, isServing]));

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: ' ', c
      т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: ' ', c
      апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: ' ', col
      без сметаны, плиз.', animState: 'in' },
  ]);
  setServedHistory([]);

```

```

        opacity-40 disabled:cursor-not-allowed text-
        :scale-95 transition-all cursor-pointer fle
    >
    <span> Налить суп первому (DEQUEUE)</span>
</button>

<button
  onClick={reset}
  className="px-5 py-3.5 rounded-2xl bg-slate-800
    r-pointer border border-slate-700"
>
  Сброс
</button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

  {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
  <div className={`md:col-span-9 bg-slate-900 rounded-2xl
    -[300px] overflow-hidden ${shakeEmpty ? 'anim
  <div className="absolute top-4 left-4 text-[1.2em]
    Очередь голодных гостей (Буфер FIFO / BFS)
  </div>

  <div className="flex-1 flex flex-col justify-between
    <div className="flex items-end justify-between

    {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
    <div className="flex flex-col items-center
      <div className="relative" >
        <div className="absolute -top-10 left-10
          <div className="w-1.5 h-6 bg-slate-400
            <div className="w-2 h-4 bg-slate-400
          </div>
          <span className="text-5xl block animate-pulse
        </div>
        <div className="text-center">

```



```
        { /* Пол */ }
        <div className="w-full h-3 bg-gradient-to-r f
    </div>
</div>

{ /* ЛОГ ДЕЙСТВИЙ */ }
<div className="bg-slate-900 border border-slate-:
    <div className="text-[10px] font-mono text-slate
        {log.map((entry, idx) => {
            <div
                key={idx}
                className={`text-xs font-mono flex items-ce
            >
                {idx === 0 ? <span className="text-indigo-5
                <span>{entry}</span>
            </div>
        }}}
    </div>
</div>
);
};
```

ФАЙЛ 56/82: src/components/SalmonAutomatonWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term link: number;
```

```

const buildInitialTrie = (wordList: string[]) => {
  const newNodes: { [key: number]: TrieNode } = {
    0: { id: 0, char: 'ROOT', parent: 0, children: {} }
  };
  let count = 0;

  wordList.forEach((word) => {
    let curr = 0;
    for (let i = 0; i < word.length; i++) {
      const c = word[i].toUpperCase();
      if (newNodes[curr].children[c] === undefined) {
        count++;
        newNodes[curr].children[c] = count;
        newNodes[count] = {
          id: count,
          char: c,
          parent: curr,
          children: {},
          fail: 0,
          term_link: 0,
          is_terminal: false,
          words: [],
          depth: newNodes[curr].depth + 1
        };
      }
      curr = newNodes[curr].children[c];
    }
    if (!newNodes[curr].words.includes(word)) {
      newNodes[curr].words.push(word);
      newNodes[curr].is_terminal = true;
    }
  });

  let currentX = 0;
  const paddingX = 70;
  const paddingY = 80;

  const layoutDFS = (u: number, depth: number) => {

```



```

    if (!cleanWord) return;
    if (words.includes(cleanWord)) {
        setSpeechText(`Слово «${cleanWord}» уже есть в сл
        return;
    }
    const updated = [...words, cleanWord];
    setWords(updated);
    setNewWord('');
    buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
    if (words.length <= 1) {
        setSpeechText('Оставь хотя бы одно слово, иначе м
        return;
    }
    const updated = words.filter((w) => w !== wordToRemo
    setWords(updated);
    buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
    const rootChildren = Object.values(nodes[0].children
    const queue = [...rootChildren];
    const updatedNodes = { ...nodes };
    rootChildren.forEach((childId) => {
        updatedNodes[childId].fail = 0;
    });

    setNodes(updatedNodes);
    setBfsQueue(queue);
    setSimulationStep(1);
    setCurrentNode(null);
    setSpeechText(
        'Начинаем обход в ширину (BFS)! Все вершины на гл
        'оче суффикса просто нет. Жми «Следующий шаг»!'
    );
};

```

```

    logs += `Для #${u} ('${char}') fail = #${failTarget}
  }

  if (childrenIds.length === 0) {
    logs += 'У этой вершины нет детей, просто переход
  }

  setNodes(updatedNodes);
  setBfsQueue(newQueue);
  setCurrentNode(r);
  setSpeechText(logs);
};

return (
  <div className="bg-slate-800/90 rounded-2xl p-6 border
    { /* Шапка виджета */ }
    <div className="flex flex-wrap items-center justify
      <div className="flex items-center space-x-3">
        <span className="text-3xl"> </span>
        <div>
          <h4 className="text-2xl font-bold text-white">
            Интерактивный конструктор: Говорящий Эхо-
          </h4>
          <p className="text-sm text-slate-400">
            Построение дерева и расчет суффиксных ссы
          </p>
        </div>
      </div>
    <div className="flex items-center gap-2">
      <button
        onClick={() => buildInitialTrie(words)}
        className="flex items-center gap-1 bg-slate
        ransition-colors"
        title="Сбросить автомат"
      >
        <RotateCcw className="w-4 h-4" /> Сбросить
      </button>
    </div>

```

```

        <line x1="133" y1="85" x2="157" y2="85"
    ) : (
        <
        <circle cx="148" cy="85" r="6" fill="
        <circle cx="151" cy="82" r="2" fill="
    </>
    )}

    {/* Рот (анимация разговора) */}
    {isTalking ? (
        <path d="М 165 105 Q 155 120 165 125 Q
    ) : (
        <path d="М 165 110 Q 155 115 168 118" s
    )}

    {/* Улыбка / жабры */}
    <path d="М 130 80 Q 125 100 130 120" stro
    <path d="М 120 85 Q 115 100 120 115" stro
</svg>

    {/* Декоративные пузыри */}
    <div className="absolute -top-1 right-4 tex
    <div className="absolute top-8 right-1 text
</div>
    <span className="mt-3 bg-rose-600/30 border b
        Эхо-Лосось Сеня
    </span>
</div>

    {/* Пузырь с текстом (Баббл) */}
    <div className="md:col-span-2 bg-slate-800 bord
        tween min-h-[140px]">
        {/* Треугольник баббла */}
        <div className="absolute -left-3 top-1/2 -tran
            slate-800 border-b-[12px] border-b-transpar

    <div className="flex items-center space-x-2 t
        <Volume2 className={`w-4 h-4 ${isTalking ?

```

```

        transition-transform active:scale-95"
      >
        <RotateCcw className="w-4 h-4" /> Пер
      </button>
    })
  </div>
</div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-s
  <h5 className="text-white font-bold mb-3 flex i
    <span> </span> Дерево и Суффиксные (fail) ссы
  </h5>

  {{{() => {
    let maxX = 0;
    let maxY = 0;
    Object.values(nodes).forEach(n => {
      if (n.x && n.x > maxX) maxX = n.x;
      if (n.y && n.y > maxY) maxY = n.y;
    });
    const svgWidth = Math.max(maxX + 80, 400);
    const svgHeight = Math.max(maxY + 60, 200);

    return (
      <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
        <defs>
          <marker id="arrowhead" markerWidth="6"
            <polygon points="0 0, 6 3, 0 6" fill=
          </marker>
          <marker id="fail-arrow" markerWidth="6"
            <polygon points="0 0, 6 3, 0 6" fill=
          </marker>
        </defs>

        {/* Draw Edges */}

```

```

        const target = nodes[node.fail];
        if (!target || !node.x || !node.y || !target.x || !target.y) return null;

        // Simple bezier curve for fail link
        const isCurrent = currentNode === node;

        return (
            <path
                key={`fail-${node.id}`}
                d={`M ${node.x} ${node.y - 10} Q ${node.x + 16} ${node.y - 16} ${target.x} ${target.y - 10}`}
                fill="none"
                stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
                strokeWidth={isCurrent ? 2 : 1.5}
                strokeDasharray="4 4"
                markerEnd="url(#fail-arrow)"
            />
        );
    });
}

```

```

{/* Draw Nodes */}
Object.values(nodes).map(node => {
    if (!node.x || !node.y) return null;
    const isRoot = node.id === 0;
    const isCurrent = currentNode === node;
    const isInQueue = bfsQueue.includes(node.id);

```

```

    let fill = '#1e293b';
    let stroke = '#475569';

```

```

    if (isCurrent) {
        fill = '#2563eb';
        stroke = '#60a5fa';
    } else if (isInQueue) {
        fill = '#b45309';
        stroke = '#fbbf24';
    } else if (node.is_terminal) {
        fill = '#059669';
    }

```

```

    }}())}
  </div>

  { /* Управление словарем */ }
  <div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
    <div className="bg-slate-900/50 p-5 rounded-xl">
      <div>
        <h5 className="text-white font-bold mb-3">
          <span> </span> Словарь (Паттерны)
        </h5>
        <div className="flex flex-wrap gap-2 mb-4">
          {words.map((w) => {
            <span
              key={w}
              className="bg-slate-800 border border-slate-700
                px-2 py-1 rounded group"
            >
              {w}
              <button
                onClick={() => removeWord(w)}
                className="text-slate-500 hover:text-white
                  transition-colors"
                title="Удалить"
              >
                x
              </button>
            </span>
          )}}
        </div>
      </div>

      <form onSubmit={handleAddWord} className="flex">
        <input
          type="text"
          value={newWord}
          onChange={(e) => setNewWord(e.target.value)}
          placeholder="НОВОЕ СЛОВО..."
          maxLength={8}
          className="bg-slate-800 border border-slate-700
            px-2 py-1 rounded"
        />
      </form>
    </div>
  </div>

```

```

const isCurrent = currentNode === node
const isInQueue = bfsQueue.includes(node)

return (
  <tr
    key={node.id}
    className={`transition-colors ${
      isCurrent
        ? 'bg-blue-900/40 font-bold text-white'
        : isInQueue
        ? 'bg-slate-800/80'
        : 'hover:bg-slate-800/40'
      }`}
  >
    <td className="py-2.5 px-3 flex items-center"
      >
      <span
        className={`w-2 h-2 rounded-full ${
          node.id === 0
            ? 'bg-purple-500'
            : isCurrent
            ? 'bg-blue-400 animate-pulse'
            : isInQueue
            ? 'bg-amber-400'
            : 'bg-slate-600'
          }`}
        ></span>
      <span>#{node.id}</span>
      {node.id === 0 && <span className="font-size-0.8"
        >
        </td>
      <td className="py-2.5 px-3"
        >
        <span className="bg-slate-800 border border-slate-700 p-1"
          >
          {node.char}
        </span>
      </td>
      <td className="py-2.5 px-3 text-slate-400"
        >
        {node.id === 0 ? '-' : `#${node.id}`}
      </td>
      <td className="py-2.5 px-3"

```

```
        </table>
      </div>
    </div>
  </div>
</div>
);
};
```

ФАЙЛ 57/82: src/components/SegmentTreeVisualizer.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
// ===== TYPES =====
interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>; // node index to value
  highlightNodes: number[]; // current highlighted nodes
  mergeChildren?: [number, number]; // pair being merged
  arrayHighlight?: [number, number]; // [L, R] range in array
  result?: number; // partial result
}
```

```
// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, number> {
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
  }
  go(1, 0, n - 1);
  return tree;
}
```



```

    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}
      { ...tree }, highlightNodes: [v], arrayHighlight:
go(2 * v, l, m);
steps.push({ id: id++, desc: `Вернулись в build(${v}
  lightNodes: [v], arrayHighlight: [l, r] }));
go(2 * v + 1, m + 1, r);
tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
steps.push({ id: id++, desc: `Объединяем: tree[${v}]
  tate: { ...tree }, highlightNodes: [v], mergeChild
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `  Дерево построено! Корень
  Highlight: [0, n - 1] }));
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
  const n = arr.length;
  const tree = buildTreeFull(arr);
  const steps: Step[] = [];
  let id = 0;

  function go(v: number, l: number, r: number, ql: number, qr: number) {
    if (ql > r || qr < l) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
        codeLine: 2, treeState: tree, highlightNodes: [v] });
      return 0;
    }
    if (ql <= l && r <= qr) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
        codeLine: 4, treeState: tree, highlightNodes: [v] });
      return tree[v];
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
      codeLine: 6, treeState: tree, highlightNodes: [v] });
    const left = go(2 * v, l, m, ql, qr);
    const right = go(2 * v + 1, m + 1, r, ql, qr);
    return left + right;
  }
}

```

```

    if (r & 1) {
        r--;
        res += tree[r] ?? 0;
        steps.push({ id: id++, desc: `r=${r + 1} нечётный`, treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR], result: res });
        highlights.push(r);
    }
    l >>= 1;
    r >>= 1;
    if (l < r) {
        steps.push({ id: id++, desc: `Поднимаемся: l = ${l}`, treeState: tree, highlightNodes: [qL, qR], arrayHighlight: [qL, qR], result: res });
    }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; left: VNode; right: VNode; }
```

```
function buildVisualTree(treeState: Record<number, number>) {
    if (l === r) return { v, l, r, val: treeState[v] ?? null, left: null, right: null };
    const m = (l + r) >> 1;
    return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m), right: buildVisualTree(treeState, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
    "build(v, l, r) {",
    "  if (l === r) { tree[v] = A[l]; return; }",
    "  // -----",
    "  // -----",
    "  mid = (l + r) >> 1;",
    "  build(2*v, l, mid);",
    "  build(2*v+1, mid+1, r);",
    "}"
]

```

```

    return () => clearTimeout(t);
  }, [playing, idx, steps, speed]);

  const step = steps[idx] || steps[0];
  if (!step) return null;

  const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' }, [color];

  return { step, idx, setIdx, playing, setPlaying, speed };
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr, arrType }) {
  const { step, idx, setIdx, playing, setPlaying, speed } = usePlayer({ steps, color });
  if (!step) return null;
  const { step, idx, setIdx, playing, setPlaying, speed } = usePlayer({ steps, color });
  const n = arr.length;

  const vTree = useMemo(() => buildVisualTree(step.tree, n));

  const renderNode = useCallback((nd: VNode): React.ReactNode => {
    const isHigh = step.highlightNodes.includes(nd.v);
    const isChild = step.mergeChildren?.includes(nd.v);
    const has = nd.val !== null;

    let cls = 'bg-slate-800 border-slate-700 text-slate-200';
    if (isHigh) cls = `${clr.ring} text-white ring-2 shadow-lg`;
    else if (isChild) cls = `${clr.childRing} text-amber-200`;
    else if (has) cls = `${clr.filled} text-slate-200`;

    return (

```

```

    <span className="text-[7px] text-slate-500">
      <span className="text-[11px] font-bold font-sans">
        </div>
      </div>
    );
  }
}
</div>

```

```
{/* Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-slate-800">
  <div className="scale-[0.85] origin-top min-w-max">
    </div>
```

```

{ /* Code */ }
<pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
  {code.map((line, i) => {
    const ln = i + 1;
    const active = step.codeLine === ln;
    return (
      <div key={ln} className={`px-2 py-0.5 rounded
        2 border-indigo-500 text-indigo-200' : color
        ' : 'bg-amber-600/20 border-l-2 border-amber-600' : active}`}>
        <span className="inline-block w-4 text-slate-400">{ln}</span>
        {line}
      </div>
    );
  })}
</pre>

```

```

{ /* Description + result */ }
<div className="px-3 py-2.5 bg-slate-900/60 border
  px]">
  <span className={`mt-0.5 inline-block w-2 h-2 r
  <span className="leading-relaxed">{step.desc}</
  {step.result !== undefined && {
    <span className={`ml-auto shrink-0 font-mono
      ' : color === 'emerald' ? 'text-emerald-300
      = {step.result}
    </span>

```

```

const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
const [customInput, setCustomInput] = useState('5, 8,');
const [qL, setQL] = useState(1);
const [qR, setQR] = useState(4);
const [view, setView] = useState<'build' | 'queries'>('build');

// Clamp query range when array changes
useEffect(() => {
  setQL(prev => Math.min(prev, arr.length - 1));
  setQR(prev => Math.min(prev, arr.length - 1));
}, [arr]);

const handleApply = (e: React.FormEvent) => {
  e.preventDefault();
  const parsed = customInput.split(',').map(s => parseInt(s));
  if (parsed.length >= 2 && parsed.length <= 16) {
    setArr(parsed);
  }
};

const randomize = () => {
  const size = arr.length;
  const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
  setArr(a);
  setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr), [arr]);
const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr), [arr]);
const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr), [arr]);

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border border-slate-800 p-4">
    {/* ---- TOP BAR: array editor + query range ---- */}
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4">

```

```

        max={arr.length - 1}
        value={qL}
        onChange={e => setQL(Math.min(Number(
        className="w-14 bg-slate-900 border b
ocus:border-indigo-500"
      />
    </div>
    <div className="flex items-center gap-1">
      <span className="text-xs text-slate-400"
      <input
        type="number"
        min={0}
        max={arr.length - 1}
        value={qR}
        onChange={e => setQR(Math.min(Number(
        className="w-14 bg-slate-900 border b
ocus:border-indigo-500"
      />
    </div>
  </div>
  <div className="text-[10px] text-slate-500 p
    Запрос: sum(A[{qL}..{qR}]) = {arr.slice(q
  </div>
</div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1"
  <button onClick={() => setView('build')} classN
    -colors ${view === 'build' ? 'border-indigo-5
    -slate-200'}`}>
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} clas
    on-colors ${view === 'queries' ? 'border-emerg
    er:text-slate-200'}`}>
    <Search className="w-3.5 h-3.5" /> Запрос: св

```

```

        color="amber"
        arr={arr}
      />
    </div>
  })

{view === 'theory' && {
  <div className="space-y-5">
    {/* Why it splits this way */}
    <div className="bg-slate-950 p-5 rounded-xl b
      <h4 className="text-base font-bold text-whi
        во разбилось именно так для N={arr.length}?
        <p className="text-sm text-slate-300 leadin
          Каждый узел берёт <code className="bg-sla
          к получает <code className="bg-slate-900 px
          -slate-900 px-1 rounded font-mono text-emerg
          евая часть забирает на 1 больше (округление
        </p>
        <div className="bg-slate-900 p-4 rounded-lg
          <div className="text-slate-400">Корень: L
          <div className="text-slate-300">mid = l(0
          <div className="text-indigo-300">→ Левый:
          <div className="text-emerald-300">→ Правы
          h - 1) >> 1)) эл.)</div>
          <div className="text-slate-500 mt-2">Всег
        } внутренних.</div>
      </div>
    </div>

    {/* Comparison cards */}
    <div className="grid grid-cols-1 xl:grid-cols
      <div className="bg-slate-950 p-5 rounded-xl
        <div className="absolute -top-3 left-4 bg
        e border border-emerald-500 shadow-md">
          1 Запрос Сверху-вниз (Рекурсия)
        </div>
        <p className="text-xs text-slate-300 mb-3
          Начинаем с корня. Если отрезок узла пол

```

```
        </p>
      </div>
    </div>
  )}
</div>
);
}
```

ФАЙЛ 58/82: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Мобильный режим: список живёт в выдвижной панели */
  onNavigate?: () => void;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate }) => {
  const [query, setQuery] = useState("");

  const groups = useMemo(() => {
    const q = query.trim().toLowerCase();
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
    const map = new Map<string, typeof chapters>();
    for (const c of filtered) {
      const key = c.category?.trim() || "Прочие темы";
      if (!map.has(key)) map.set(key, []);
      (map.get(key) as typeof chapters).push(c);
    }
  }, [query]);
```



```

const isSelected = chapter.id === selectedChapterId
const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
return (
  <button
    key={chapter.id}
    onClick={() => {
      setSelectedChapterId(chapter.id);
      onNavigate?.();
      window.scrollTo({ top: 0, behavior: 'smooth' })
    }}
    className={`w-full text-left px-3 py-2 border-1
      ${isSelected
        ? "bg-indigo-600/15 border-indigo-600/20"
        : "bg-slate-800/40 border-transparent"}
    `}
  >
    {hasViz && (
      <span
        className="w-1.5 h-1.5 rounded-full border-1"
        title="Есть интерактивная визуализация"
      />
    )}
    <span className="font-semibold text-white">
      <ChevronRight
        className={`w-4 h-4 shrink-0 mt-0.5
          ${isSelected ? "text-indigo-400" : "text-white"}
        `}
      />
    </button>
  )
);
}}}
</div>
</div>
)))
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-500 to-indigo-600
  ] text-slate-300 space-y-1.5 hidden lg:block">

```

```

}

export const Simulator: React.FC = () => {
  const [activeTab, setActiveTab] = useState<'stack' |

  // Stack state
  const [stack, setStack] = useState<Plate[]>([
    { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
    { id: '3', name: 'Блюдец от торта', icon: ' ' },
  ]);
  const [popMessage, setPopMessage] = useState<string |

  // Queue state
  const [queue, setQueue] = useState<Person[]>([
    { id: '1', name: 'Студент Вася', icon: ' ' },
    { id: '2', name: 'Голодный кодер', icon: ' ' },
    { id: '3', name: 'Кот Борис', icon: ' ' },
  ]);
  const [dequeueMessage, setDequeueMessage] = useState<

  // Heap state
  const [heap, setHeap] = useState<Hypothesis[]>([
    { id: '1', text: 'Кандидат: "Привет, я искусственный"', icon: ' ' },
    { id: '2', text: 'Кандидат: "Привет, я испек вкусный"', icon: ' ' },
    { id: '3', text: 'Кандидат: "Привет, я испанский ин"', icon: ' ' },
  ]);
  const [newCustomText, setNewCustomText] = useState('');
  const [newCustomProb, setNewCustomProb] = useState(0);
  const [heapMessage, setHeapMessage] = useState<string |

  // Stack handlers
  const addPlate = () => {
    const presets = [
      { name: 'Сковорода от омлета', icon: ' ' },
      { name: 'Кастрюля от супа', icon: ' ' },
      { name: 'Чашка от кофе', icon: ' ' },
      { name: 'Миска с остатками салата', icon: ' ' },
    ],
  }

```

```

    ];
    const randomPreset = presets[Math.floor(Math.random() * presets.length)];
    const newPerson = {
      id: Date.now().toString(),
      name: randomPreset.name,
      icon: randomPreset.icon
    };
    setQueue(prev => [...prev, newPerson]);
    setDequeueMessage(` Встал в конец очереди: ${newPerson.name}`);
  };

  const servePerson = () => {
    if (queue.length === 0) {
      setDequeueMessage("⚠ Очередь пуста! Суп ждет нового кандидата");
      return;
    }
    const firstPerson = queue[0];
    setQueue(prev => prev.slice(1));
    setDequeueMessage(` Выдан суп первому в очереди (Фамилия: ${firstPerson.name})`);
  };

  const resetQueue = () => {
    setQueue([
      { id: '1', name: 'Студент Вася', icon: '👤' },
      { id: '2', name: 'Голодный кодер', icon: '👤' },
      { id: '3', name: 'Кот Борис', icon: '🐈' },
    ]);
    setDequeueMessage(" Очередь восстановлена.");
  };

  // Heap handlers
  const addHypothesis = (e: React.FormEvent) => {
    e.preventDefault();
    if (!newCustomText.trim()) return;
    const item: Hypothesis = {
      id: Date.now().toString(),
      text: `Кандидат: "${newCustomText}"`,
      probability: Number(newCustomProb)
    };
    setQueue(prev => [...prev, item]);
  };

```

```

<h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-indigo-600" style="text-decoration: none;">
  <span> Интерактивный симулятор структур данных
</h2>
<p className="text-slate-400 text-sm mt-2">
  Проверь работу принципов LIFO, FIFO и Приоритет
</p>
</div>

```

```

{/* Tabs */}
<div className="grid grid-cols-1 md:grid-cols-3 gap-4">
  <button
    onClick={() => setActiveTab('stack')}
    className={`flex items-center justify-between gap-2 px-4 py-2 transition-colors duration-200
      ${activeTab === 'stack'
        ? 'bg-blue-600/20 border-blue-500 text-blue-600'
        : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
    `}
  >
    <div className="flex items-center gap-3">
      <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-600' : 'text-slate-400'} fill-current`} />
      <div className="text-left">
        <div className="font-bold text-sm text-white">Стек
        <div className="text-xs opacity-80">LIFO
      </div>
    </div>
    <span className="text-xs bg-blue-500/20 text-blue-600 border border-blue-500/20 padding-2 5px">
      DFS
    </span>
  </button>

  <button
    onClick={() => setActiveTab('queue')}
    className={`flex items-center justify-between gap-2 px-4 py-2 transition-colors duration-200
      ${activeTab === 'queue'
        ? 'bg-indigo-600/20 border-indigo-500 text-indigo-600'
        : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
    `}
  >
    <div className="flex items-center gap-3">
      <Layers className={`w-5 h-5 ${activeTab === 'queue' ? 'text-indigo-600' : 'text-slate-400'} fill-current`} />
      <div className="text-left">
        <div className="font-bold text-sm text-white">Очередь
        <div className="text-xs opacity-80">FIFO
      </div>
    </div>
    <span className="text-xs bg-indigo-500/20 text-indigo-600 border border-indigo-500/20 padding-2 5px">
      BFS
    </span>
  </button>

```

```

tify-between gap-4">
<div>
  <h3 className="text-lg font-bold text-white">
    <span> Симуляция стопки тарелок</span>
    <span className="text-xs font-mono bg-blue-950/60 border border-white/10 padding-2">
      0
    </span>
  </h3>
  <p className="text-slate-400 text-xs mt-1">
    Новые тарелки ставятся наверх. Мыть можно.
  </p>
</div>
<div className="flex flex-wrap items-center gap-4">
  <button
    onClick={addPlate}
    className="flex-1 md:flex-none flex items-center justify-center gap-2 rounded-xl text-sm font-bold shadow-lg transition-colors"
  >
    <Plus className="w-4 h-4" /> Поставить
  </button>
  <button
    onClick={popPlate}
    className="flex-1 md:flex-none flex items-center justify-center gap-2 border-blue-500/40 px-4 py-2.5 rounded-xl transition-colors"
  >
    Помыть верхнюю
  </button>
  <button
    onClick={resetStack}
    title="Сбросить"
    className="p-2.5 bg-slate-800 hover:bg-slate-700 text-white transition-colors cursor-pointer"
  >
    <RotateCcw className="w-5 h-5" />
  </button>
</div>
</div>

{popMessage && (
  <div className="bg-blue-950/60 border border-white/10 padding-4">
    {popMessage}
  </div>
)}

```

```

        <span className={`font-bold text-
            {plate.name}
        </span>
    </div>
    <span className={`text-xs font-mon
        isTop ? 'bg-blue-500 text-white
    }`}>
        {isTop ? 'Готово к помывке' : `
    </span>
</div>
    );
    }}}}
</div>
}}
<div className="mt-6 text-xs text-slate-500
    ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
}}
```

```

{/* QUEUE TAB */}
{activeTab === 'queue' && {
    <div className="space-y-6">
        <div className="bg-slate-800/80 p-5 rounded-x
            tify-between gap-4">
            <div>
                <h3 className="text-lg font-bold text-whi
                    <span> Симуляция очереди за супом</span>
                    <span className="text-xs font-mono bg-i
                /span>
                </h3>
                <p className="text-slate-400 text-xs mt-1
                    Кто первым пришел, тот первым получил с
                </p>
            </div>
            <div className="flex flex-wrap items-center
                <button
```

```

    <p className="text-base font-bold">Очер
    <p className="text-xs mt-1">Добавьте го
  </div>
) : (
  <div className="w-full flex items-center"
    <div className="flex flex-col items-cen
300">
    <span className="text-3xl"> </span>
    <span className="text-xs font-bold font
  </div>

  <div className="text-indigo-500 flex it
    <ArrowRight className="w-6 h-6 animat
  </div>

  {queue.map((person, index) => {
    const isFirst = index === 0;
    return (
      <div
        key={person.id}
        className={`flex flex-col items-c
          isFirst
            ? 'bg-gradient-to-b from-indi
relative'
            : 'bg-slate-900/90 border-sla
        }`}
      >
        {isFirst && (
          <span className="absolute -top-1
se tracking-wider shadow">
            Первый (Head)
          </span>
        )}
        <span className="text-4xl mb-2">{
        <span className={`font-bold text-
          {person.name}
        </span>
        <span className={`text-[10px] font

```

```

        <button
            onClick={popHeap}
            className="flex-1 md:flex-none flex item
py-2.5 rounded-xl text-sm font-bold shadow-
        >
            Взять кандидата с макс. вероятностью
        </button>
        <button
            onClick={resetHeap}
            title="Сбросить"
            className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
        >
            <RotateCcw className="w-5 h-5" />
        </button>
    </div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
    <div className="md:col-span-6">
        <label className="block text-xs font-bold">
            <input
                type="text"
                value={newCustomText}
                onChange={e => setNewCustomText(e.target
                placeholder='например: "Привет, я лучший"'
                className="w-full bg-slate-900 border b
rder-emerald-500 transition-colors"
            />
        </div>
        <div className="md:col-span-3">
            <label className="block text-xs font-bold">
                Вероятность: <span className="text-emerg
            </label>
            <input
                type="range"

```



```

    ) : (
      <div className="w-full max-w-2xl space-y-2">
        <div className="flex items-center justify-between">
          <span>КАНДИДАТ / ГИПОТЕЗА</span>
          <span>ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)</span>
        </div>

        {sortedHeap.map(({item, index}) => {
          const isTop = index === 0;
          const percent = (item.probability * 100).toFixed(
00/10 scale-[1.01])'
            : 'bg-slate-900/80 border-slate-800'
          }`)}
        >
          <div className="flex items-center justify-between">
            <span className={`flex items-center justify-between`}
              isTop ? 'bg-emerald-500 text-white' : ''>
              <div>
                <div className={`font-bold text-sm`}
                  {item.text}>
                </div>
                {isTop && (
                  <span className="text-[10px] font-medium"
-0.5 rounded inline-block mt-1">
                    Вершина Кучи (Root) • Будущее
                  </span>
                )}
              </div>
            </div>
          </div>

```

```
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";

interface Props {
  /** Открыть тренажёр в модальном окне. */
  onOpen: (vizId: string) => void;
  /** Перейти к теме, к которой привязан тренажёр. */
  onGoChapter: (chapterId: string) => void;
}

/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно поты
 */
export const SimulatorHub: React.FC<Props> = ({ onOpen,
  const [query, setQuery] = useState("");

  const items = useMemo(() => {
    const titleById = new Map(chapters.map((c) => [c.id, c.title]));
    const all = Object.entries(VIZ_REGISTRY).map(([id, entry],
      id,
      entry,
      chapterTitle: titleById.get(id),
    ]));
    const q = query.trim().toLowerCase();
    if (!q) return all;
    return all.filter(
      (i) =>
        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q) ||
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
  }, [query]);

  return (
    <div>
```

```

{!entry.hint && <div className="flex-1" />}

<div className="flex items-center gap-2 mt-4">
  <button
    onClick={() => onOpen(id)}
    className="flex items-center gap-1.5 text-white transition-colors"
  >
    <Play className="w-3.5 h-3.5" /> Запустить
  </button>
  {chapterTitle && (
    <button
      onClick={() => onGoChapter(id)}
      title={chapterTitle}
      className="flex items-center gap-1.5 text-white transition-colors min-w-[100px]"
    >
      <BookOpen className="w-3.5 h-3.5 shrink-0" />
      <span className="truncate max-w-[90px]" />
    </button>
  )}
</div>
</div>
)}}
</div>
</div>
);
};

```

ФАЙЛ 61/82: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";

```

```

        const n3 = leaf(3);
        const n2: TNode = { key: 2, left: leaf(1), right:
        const n4: TNode = { key: 4, left: n2, right: leaf
        return { key: 6, left: n4, right: leaf(7) };
    },
},
{
    name: "Один уровень",
    hintCase: "Zig",
    target: 2,
    build: () => ({ key: 4, left: { key: 2, left: leaf(
},
{
    name: "Смешанный",
    hintCase: "Zig-Zag, затем Zig-Zig",
    target: 5,
    build: () => {
        const n5 = leaf(5);
        const n6: TNode = { key: 6, left: n5, right: leaf
        const n4: TNode = { key: 4, left: leaf(3), right:
        return { key: 8, left: n4, right: leaf(9) };
    },
},
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number
const path: TNode[] = [];
let cur = root;
while (cur) {
    path.push(cur);
    if (key === cur.key) return path;
    cur = key < cur.key ? cur.left : cur.right;
}
return [];
};

```

```

/* ----- раскладка -----

interface Placed {
  key: number;
  x: number;
  y: number;
  depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
  const nodes: Placed[] = [];
  const edges: [number, number][] = [];
  let order = 0;

  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
    if (n.left) edges.push([n.key, n.left.key]);
    if (n.right) edges.push([n.key, n.right.key]);
    walk(n.right, depth + 1);
  };
  walk(root, 0);

  const cols = Math.max(1, order);
  const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
  const colW = 46;
  const rowH = 62;
  return {
    nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
    edges,
    width: cols * colW,
    height: rows * rowH + 20,
  };
}

/* ----- КОМПОНЕНТ -----

```

```

const undo = () => {
  if (history.length === 0) return;
  setTree(history[history.length - 1]);
  setHistory((h) => h.slice(0, -1));
  setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build()))

return (
  <div className="w-full bg-slate-950 rounded-2xl border-2 border-indigo-500">
    <div className="flex items-start justify-between p-4">
      <div>
        <h4 className="text-sm sm:text-base font-bold">
          <p className="text-[12px] text-slate-400 lead-4">
            Клик по узлу = один поворот с его родителем
            <span className="font-mono font-bold text-indigo-400">
              поворотов выбираешь сам, разбор покажем в к
            </p>
          </div>
          <button
            onClick={() => reset((presetIdx + 1) % PRESETS.length)}
            className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-bold transition-colors shrink-0"
          >
            <Dices className="w-3.5 h-3.5" /> Другое дерево
          </button>
        </div>

        <div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
          {PRESETS.map((p, i) => (
            <button
              key={p.name}
              onClick={() => reset(i)}
              className={`px-3 py-1.5 rounded-lg text-[11px] font-medium
                ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-400"}`}
            >{p.name}
          ))}
        </div>
      </div>
    </div>
  </div>
)

```

```

        key={n.key}
        onClick={() => clickable && clickNode(n)}
        style={{
          transform: `translate(${n.x}px, ${n.y}px)`,
          transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
          cursor: clickable ? "pointer" : "default",
        }}
      >
        {isTarget && (
          <circle r={22} fill="none" stroke="#6666ff" stroke-width={2} />
        )}
        <circle
          r={16}
          fill={isTarget ? "#6666ff" : path.hasChildren ? "#a5b4fc" : "#475568"}
          stroke={isTarget ? "#a5b4fc" : "#475568"}
          stroke-width={2}
        />
        <text textAnchor="middle" dominantBaseline="middle">
          {n.key}
        </text>
      </g>
    );
  }
}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
  <button
    onClick={undo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold"
  >
    <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
  </button>
  <button
    onClick={() => reset()}
    className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold"
  >
    <Reset2 className="w-3.5 h-3.5" /> СБРОС
  </button>
</div>

```

```

        {wrong === 0
          ? `Узел ${target} в корне за ${moves.length} ходов
          : `Узел ${target} в корне за ${moves.length} ходов
        }
      </div>
    {wrong > 0 && (
      <ul className="list-disc pl-5 space-y-0.5">
        {moves.map(
          (m, i) =>
            !m.correct && (
              <li key={i}>
                ход {i + 1}: крутили <span className="text-red-500">{m.expected}</span>
              </li>
            )
          )
        }
      </ul>
    )}
    {wrong === 0 && moves.length > minimal && (
      <div className="opacity-80">Порядок верный,
    )}
  </div>
)}
</div>
);
};

```

ФАЙЛ 62/82: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";

```

/**

* Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag

*/


```

    ranges: string;
    frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
    pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
    edges: [{"p", "x"}, {"p", "C"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
    key: "zig",
    label: "Zig",
    when: "родитель x — уже корень, деда нет",
    rotations: "1 поворот",
    keys: { x: 20, p: 40 },
    inorder: ["A", "20", "B", "40", "C"],
    ranges: "A < 20 · B между 20 и 40 · C > 40",
    frames: [
        {
            ...ZIG_BEFORE,
            kind: "start",
            title: "Было: 20 висит под корнем 40",
            text: "Нам нужен ключ 20, а он на глубине 1. Деду
            ms: RESULT_MS,
        },
        {
            ...ZIG_BEFORE,
            kind: "aim",
            title: "Прицел: крутим ребро 40 — 20",
            text: "Ничего пока не двигается. Смотри на подсве
            focus: ["p", "x"],
            ms: AIM_MS,
        },
    ],
};

```

```

        text: "Три узла выстроились в «бамбук»: каждый сл
            зглаживает.",
        ms: RESULT_MS,
    },
    {
        ...ZZ_S0,
        kind: "aim",
        title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
        text: "Главная хитрость этого случая: первым крути
        focus: ["g", "p"],
        ms: AIM_MS,
    },
    {
        ...ZZ_S1,
        kind: "result",
        title: "Поворот 1: 40 поднялось над 60",
        text: "40 встало в корень, 60 опустилось к нему п
            убине 1, а не 2.",
        moved: ["p", "g", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZZ_S1,
        kind: "aim",
        title: "Прицел 2: ребро 40 – 20",
        text: "А теперь обычный одиночный поворот – тот ж
        focus: ["p", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [100, 45], A: [45, 118], p: [188, 118],
        edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
        kind: "result",
        title: "Стало: 20 в корне, бамбук стал кустом",
        text: "Сравни с первым кадром: А и В были на глуб
            дерево, а не только достаёт ключ.",
        moved: ["x", "p", "A", "B"],
        ms: RESULT_MS,
    }

```

```

        ms: AIM_MS,
    },
    {
        ...ZG_S1,
        kind: "result",
        title: "Поворот 1: змейка выпрямилась",
        text: "40 заняло место 20 и забрало его к себе ле
        moved: ["x", "p", "C"],
        ms: RESULT_MS,
    },
    {
        ...ZG_S1,
        kind: "aim",
        title: "Прицел 2: ребро 60 – 40",
        text: "Остался одиночный поворот вокруг деда – и
        focus: ["g", "x"],
        ms: AIM_MS,
    },
    {
        pos: { x: [180, 50], p: [90, 130], g: [275, 130],
        edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
        kind: "result",
        title: "Стало: 20 и 60 – два сына сорока",
        text: "Дерево стало идеально симметричным: все че
            очку». ",
        moved: ["x", "p", "g", "C"],
        ms: RESULT_MS,
    },
    ],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: str
    x: { fill: "#6366f1", stroke: "#a5b4fc" },
    p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
    g: { fill: "#f59e0b", stroke: "#fcd34d" },
    sub: { fill: "#1e293b", stroke: "#475569" },

```

```

        y1={from[1] + (dy / len) * pad}
        x2={to[0] - (dx / len) * pad}
        y2={to[1] - (dy / len) * pad}
        stroke="#a5b4fc"
        strokeWidth={1.5}
        strokeDasharray="4 3"
        markerEnd="url(#splay-arrow)"
      />
    })
  </g>
);
}}}

{edges.map(([a, b], i) => {
  const pa = pos[a];
  const pb = pos[b];
  if (!pa || !pb) return null;
  const hot = Boolean(focus && focus.includes(a));
  return (
    <line
      key={`${a}-${b}-${i}`}
      x1={pa[0]}
      y1={pa[1]}
      x2={pb[0]}
      y2={pb[1]}
      stroke={hot ? "#fbbf24" : "#475569"}
      strokeWidth={hot ? 5 : 2}
      opacity={focus && !hot ? 0.25 : 1}
      style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
    />
  );
})}

{(Object.keys(pos) as NodeId[]).map((id) => {
  const p = pos[id];
  if (!p) return null;
  const sub = isSubtree(id);
  const c = COLOR[sub ? "sub" : id];

```

```

const [slow, setSlow] = useState(false);

const active = CASES[caseIdx];
const total = active.frames.length;
const idx = Math.min(frame, total - 1);
const current = active.frames[idx];
const prevPos = active.frames[Math.max(idx - 1, 0)].p;
const duration = useMemo(() => Math.round(current.ms - prevPos), [current, prevPos]);
const last = idx === total - 1;

useEffect(() => {
  setFrame(0);
}, [caseIdx]);

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
  return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
  <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
    <div className="flex gap-2 mb-4 overflow-x-auto">
      {CASES.map((c, i) => (
        <button
          key={c.key}
          onClick={() => setCaseIdx(i)}
          className={`px-3 sm:px-4 py-2 rounded-lg text-white
            ${i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"}
          `}
        >
          {c.label}
        </button>
      ))}
    </div>

    <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
      <span className="font-bold text-indigo-300">Кор

```

```

        {t}
      </span>
    )))
    <span className="text-emerald-400/80 ml-1">
  </div>
  <p className="text-[11px] text-slate-500 mt-1">
</div>
</div>

{/* индикатор автопрокрутки */}
<div className="h-1 w-full bg-slate-800 rounded-f
  <div
    key={` ${caseIdx}- ${frame}- ${playing}- ${slow}`
    className="h-full bg-indigo-500"
    style={
      playing
        ? { animation: `demoProgress ${duration}m
          : { width: "100%", background: "#334155"
        }
    }
  />
</div>

<div className="flex flex-wrap items-center gap-2">
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f - 1 + total) % total);
    }}
    disabled={idx === 0}
    className="px-3 py-2 rounded-lg bg-slate-800 l
      d:hover:text-slate-300 text-xs font-bold tra
  >
    ← Назад
  </button>
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f + 1) % total);

```

```

        active.keys[id] === undefined ? null : {
          <span key={id} className="flex items-center">
            <span className="w-3 h-3 rounded-full"
            </span>
          </span>
        )
      </div>
    </div>

    <p className="mt-2 text-[11px] text-slate-500">
      Листай кнопкой «Дальше» в своём темпе – кадры н
    </p>
  </div>
);
};

```

ФАЙЛ 63/82: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useState } from "react";
import {
  RotateCcw,
  HelpCircle,
  BookOpen,
} from "lucide-react";

interface SplayNode {
  key: number;
  left: SplayNode | null;
  right: SplayNode | null;
  x?: number;
  y?: number;
}

// Simple BST insertion to build initial tree

```

```
};
```

```
export const SplayTreeViz: React.FC = () => {
  // Let's create an elegant initial unbalanced or semi-balanced tree
  // Values: 10, 20, 30, 40, 50, 60
  const createInitialTree = (): SplayNode => {
    let r: SplayNode | null = null;
    const initialKeys = [40, 20, 50, 10, 30, 60];
    initialKeys.forEach((key) => {
      r = insertBST(r, key);
    });
    return r!;
  };
};
```

```
const [tree, setTree] = useState<SplayNode>(createInitialTree());
const [inputValue, setInputValue] = useState<string>('');
const [log, setLog] = useState<string[]>([
  "Дерево инициализировано. Попробуйте кликнуть на узел.",
]);
const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);
```

```
// Splay operation that tracks steps!
const splayStepByStep = (
  root: SplayNode | null,
  key: number,
): { newRoot: SplayNode | null; steps: string[] } => {
  const steps: string[] = [];
  if (!root) return { newRoot: null, steps: ["Дерево пустое."] };
  // We implement the classic top-down or bottom-up splay operation
  // For visualization logs, we do a bottom-up explanation
  let path: { node: SplayNode; dir: "left" | "right" } | null = null;
  let current: SplayNode | null = root;
  // Find the element or the element where search failed
  let lastNode: SplayNode = root;
  while (current) {
    lastNode = current;
```



```

        node.left.left = splayAction(node.left.left,
        node = rightRotate(node);
        steps.push(
            `Вращение Zig-Zig (Правый поворот родителя
        );
    } else if (k > node.left.key) {
        // Zig-Zag (Left-Right)
        node.left.right = splayAction(node.left.right
        if (node.left.right) {
            node.left = leftRotate(node.left);
            steps.push(`Компонент Zig-Zag: левый поворо
        }
    }

    if (!node.left) return node;
    steps.push(
        `Финальный поворот Zig (Правое вращение корня
    );
    return rightRotate(node);
} else {
    if (!node.right) return node;

    if (k < node.right.key) {
        // Zag-Zig (Right-Left)
        node.right.left = splayAction(node.right.left
        if (node.right.left) {
            node.right = rightRotate(node.right);
            steps.push(`Компонент Zig-Zag: правый поворо
        }
    } else if (k > node.right.key) {
        // Zag-Zag (Right-Right)
        node.right.right = splayAction(node.right.righ
        node = leftRotate(node);
        steps.push(
            `Вращение Zag-Zag (Левый поворот родителя д
        );
    }
}

```

```

const handleFind = (e: React.FormEvent) => {
  e.preventDefault();
  const val = parseInt(inputValue);
  if (isNaN(val)) return;
  handleSplay(val);
  setInputValue("");
};

const resetTree = () => {
  setTree(createInitialTree());
  setHighlightedNode(null);
  setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive function
const computeCoordinates = (
  node: SplayNode | null,
  x: number,
  y: number,
  levelWidth: number,
): void => {
  if (!node) return;
  node.x = x;
  node.y = y;
  if (node.left) {
    computeCoordinates(node.left, x - levelWidth, y + 1);
  }
  if (node.right) {
    computeCoordinates(node.right, x + levelWidth, y + 1);
  }
};

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

// Render lines and nodes
const renderLines = (node: SplayNode | null): React.R

```

```

    circles.push(
      <g
        key={`n-${node.key}`}
        className="cursor-pointer group"
        onClick={() => handleSplay(node.key)}
        data-title={`Кликните, чтобы выполнить Splay(${node.key})`}
      >
        <circle
          cx={node.x}
          cy={node.y}
          r="18"
          fill={isHighlighted ? "#4f46e5" : "#1e293b"}
          stroke={isHighlighted ? "#818cf8" : "#64748b"}
          strokeWidth={isHighlighted ? "3" : "2"}
          className="transition-all duration-300 group-hover:stroke-4"
        />
        <text
          x={node.x}
          y={node.y}
          dy=".3em"
          textAnchor="middle"
          fill="white"
          fontSize="12px"
          fontWeight="bold"
          className="select-none pointer-events-none"
        >
          {node.key}
        </text>
      </g>,
    );
    circles = circles.concat(renderNodes(node.left));
    circles = circles.concat(renderNodes(node.right));
    return circles;
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-slate-800" >
      {/* Header */}
    </div>
  );
}

```


</h4>

```
<div className="grid grid-cols-1 md:grid-cols-3">
  <div className="bg-slate-900/60 p-4 rounded-l
    <div>
      <p className="font-bold text-slate-300 mb
        1. Отсутствующий элемент
      </p>
      <p className="text-slate-400 leading-rela
        Допустим, мы ищем{" "}
        <code className="text-indigo-400 font-s
        его нет. Алгоритм спустится к листу, на
        неудачей {например, {" "}}
        <code className="text-indigo-400">[20]</c
        <code className="text-indigo-400">[30]</c
        поднимет в корень{" "}
        <strong className="text-white">
          последний успешно просмотренный узел
        </strong>
        , на котором он споткнулся! Это оставля
        сверху.
      </p>
    </div>
    <div className="mt-3 text-[10px] text-indig
      Поиск настраивает локальный кэш под реали
    </div>
  </div>
```

```
<div className="bg-slate-900/60 p-4 rounded-l
  <div>
    <p className="font-bold text-slate-300 mb
      2. Эффект качелей (Swing)
    </p>
    <p className="text-slate-400 leading-rela
      Если агент запрашивает по очереди{" "}
      <code className="text-rose-400">[10]</c
      <code className="text-rose-400">[60]</c
      углы), дерево будет превращаться в палк
```

ФАЙЛ 64/82: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
```

```
interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}
```

```
const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pasta' },
  { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pizza' },
  { name: 'Блюдец от торта', emoji: '🍰', color: 'from-cake' },
  { name: 'Тарелка от тако', emoji: '🌮', color: 'from-taco' },
  { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-sushi' },
  { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-pan-fry' },
];
```

```
let counter = 0;
```

```
export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>([
    { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', animState: 'idle' },
    { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', animState: 'idle' },
    { id: 'p3', name: 'Блюдец от торта', emoji: '🍰', animState: 'idle' },
  ]));
```

```

const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    addLog('▲ POP: Нечего мыть! Все тарелки чистые.')
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);

  // Step 1: Start washing animation (sponge and soap)
  setDirtyStack(prev => prev.map(p => p.id !== topPlate.id));
  setShowSoap(true);

  setTimeout(() => {
    // Step 2: Wash complete. Plate becomes clean and
    const cleanPlate: Plate = {
      ...topPlate,
      isDirty: false,
      animState: 'clean',
    };

    setCleanRack(prev => [cleanPlate, ...prev].slice(0, 10));
    setDirtyStack(prev => prev.slice(0, prev.length - 1));
    setShowSoap(false);
    setIsWashing(false);
    addLog(`  Готово! Чистая тарелка ${topPlate.emoji}`);
  }, 850);
}, [dirtyStack, isWashing]);

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝' },
  ]);
};

```

```

        <span> Положить грязную (PUSH)</span>
    </button>

    <button
      onClick={popPlate}
      disabled={isWashing || dirtyStack.length === 0}
      className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-900
        opacity-40 disabled:cursor-not-allowed text-white font-mono
        font-size-[10px] border border-slate-700 border-radius-[10px]
        transition-all cursor-pointer"
    >
      <span> Помыть верхнюю (POP)</span>
    </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-mono
        font-size-[10px] border border-slate-700"
    >
      Сброс
    </button>
  </div>

  {/* ИНТЕРАКТИВНАЯ КУХНЯ */}
  <div className="grid grid-cols-1 md:grid-cols-12 gap-4" >

    {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
    <div className="md:col-span-7 bg-slate-900 rounded-3xl p-4" >
      <div className="absolute top-4 left-4 text-[10px] font-mono" >
        Грязная стопка (Стек вызовов / LIFO)
      </div>

      {/* Визуализация раковины */}
      <div className="absolute top-4 right-4 flex items-center justify-center" >
        <div className="flex items-center justify-center gap-2" >
          <div className="w-1.5 h-1.5 rounded-full bg-slate-800/80 text-[10px] font-mono" >
            <span className="w-1.5 h-1.5 rounded-full bg-slate-800/80 text-[10px] font-mono" >
              <span>РАКОВИНА </span>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```



```

        const idx = dirtyStack.length - 1 - rev;
        const isTop = idx === topIndex;
        return (
          <div
            key={plate.id}
            className={`
              w-full max-w-[280px] h-10 rounded
              shadow-md select-none transition-all duration
              ${plate.color}
              ${plate.animState === 'in' ? 'anim
              ${plate.animState === 'washing' ?
              ${isTop ? 'ring-4 ring-blue-500/5
            `}
          >
            <span className="text-lg">{plate.em
            <span className="text-slate-800 tex
            {isTop && (
              <span className="text-[9px] bg-bl
                LIFO
              </span>
            )}
          </div>
        );
      }
    }
  </div>
)}

  {/* Столешница раковины */}
  <div className="w-full h-4 bg-gradient-to-r f
</div>

  {/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
  <div className="md:col-span-5 bg-slate-900 roun
    <div className="absolute top-4 left-4 text-[1
      Сушилка для чистой посуды
    </div>

    <div className="absolute top-4 right-4 flex i

```

```

        key={idx}
        className={`text-xs font-mono flex items-center justify-center gap-2`}
      >
        {idx === 0 ? <span className="text-blue-500">0</span> : <span>{entry}</span>}
      </div>
    </div>
  </div>
);
};

```

ФАЙЛ 65/82: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

```

```

function generateKmpSteps(pattern: string, text: string) {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];

```

```

  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

```

```

  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];

```

```

    steps.push({ i, j, pi: [...pi], highlight: [i, j] });
    while (j > 0 && s[i] !== s[j]) {

```

```

      steps.push({ i, j, pi: [...pi], highlight: [i, j] });
    }
  }
}

```

```

        !` }));
        z[i] = Math.min(r - i + 1, z[i - 1]);
        steps.push({ i, l, r, z: [...z], desc: `Пре
        }. Копипаста сработала.` }));
    }

    steps.push({ i, l, r, zTarget: z[i], zSource: i
    while (i + z[i] < n && s[z[i]] === s[i + z[i]])
        steps.push({ i, l, r, zTarget: z[i], zSource
        падают! Окно растёт.` }));
        z[i]++;
    }

    if (i + z[i] < n) {
        steps.push({ i, l, r, zTarget: z[i], zSource
        [i]}}' разные. Стоп.` }));
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Упё
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксиру

    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
        steps.push({ i, l, r, z: [...z], desc: ` 0
    }

    if (z[i] === pattern.length) {
        steps.push({ i, l, r, z: [...z], found: true
    }
}
return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp
const [mode, setMode] = useState<"kmp" | "z">(default
const [pattern, setPattern] = useState("aba");

```

```

const step = steps[stepIdx] || steps[0];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>

    <div className="flex items-center gap-2
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={pattern} onChange=
xt-white text-xs font-bold font-mono px-2 p
      </div>
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
      </div>
    </div>
  </div>

  <div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
    <div className="flex gap-1.5 px-4 h-ful
      {s.split("").map((char, index) => {
        const isPattern = index < patte

```

```

        {/* Index */}
        <span className={`text-

        {/* Char Box */}
        <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
        {char}
        </div>

        {/* Value array box */}
        <div className="text-[1
slate-700/50 font-mono font-bold">
            {mode === "kmp" ? s
        </div>

        {/* KMP fingers */}
        {mode === "kmp" && step
            <div className="abs

z-20">
                <span classNam
            </div>
        })
        {mode === "kmp" && step
            <div className="abs
                <span classNam
            </div>
        })
        {mode === "kmp" && step
            <div className="abs

20">
                <span classNam
            </div>
        })

        {/* Z fingers */}
        {mode === "z" && step.i
            <div className="abs

e z-20">

```

```

        {autoplay ? <><Pause size={16}
        </button>
        <button onClick={nextStep} disabled:
r:bg-slate-800 rounded-lg disabled:opacity-
        <SkipForward strokeWidth={3} si
        </button>
        <button onClick={reset} className="
border-slate-800">
        <RefreshCw strokeWidth={3} size
        </button>
    </div>

    {/* Description Log */}
    <div className="flex-1 bg-slate-900/50
erflow-hidden">
        <div className="absolute top-0 right
        <span className="font-mono text
        </div>
        <div className="text-[10px] text-sla
{steps.length}]</div>
        <div className="text-sm text-slate-
        {step.desc}
        </div>
    </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl
    <span className="text-2xl mt-1"> </span>
    <p className="text-xs text-slate-400 le
        Обрати внимание на хитрость с решёт
        Мы пишем <b>Шаблон + # + Текст</b>.
        Когда значени {mode === 'kmp' ? "пр
о было найдено!
        Пройди шаги вручную, посмотри, как
ля пропуска работы (в Z).
    </p>
</div>

```

```

    Pause,
    RotateCcw,
    Clock,
  } from "lucide-react";

interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}

interface TEdge {
  u: number;
  v: number;
}

const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
  { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
  { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
  { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
  { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
  { u: 0, v: 1 },
  { u: 0, v: 2 },
  { u: 1, v: 3 },
  { u: 2, v: 3 },
  { u: 3, v: 4 },
];

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
}
```

```

        visited.add(u);
        dfsStack.push(u);
        recordStep(
            `DFS: зашли в вершину ${u} (${dagNodes[u].name}
            u,
        );

        for (const to of adj[u]) {
            if (!visited.has(to)) {
                dfs(to);
            } else if (!fullyProcessed.has(to)) {
                // Explaining potential cycle detected (not in
                recordStep(
                    `⚠ Внимание: Рёбра в серую вершину ${to} от
                    u,
                );
            }
        }
    }

    fullyProcessed.add(u);
    dfsStack.pop();
    topoList.push(u); // prepend behavior: we write in
    recordStep(
        `DFS: закончили обработку '${dagNodes[u].name}'
        ,
        u,
    );
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        recordStep(
            `Запускаем новый обход DFS от нетронутой верши
            null,
        );
        dfs(i);
    }
}

```



```

    return [...step.topoList]
      .reverse()
      .map((id) => dagNodes[id].name)
      .join(" → ");
  });

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучен
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        500 text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

```

```

    return (
      <line
        key={`edge-${idx}`}
        x1={u.x}
        y1={u.y}
        x2={v.x}
        y2={v.y}
        stroke={isActive ? "#818cf8" : "#334155"}
        strokeWidth={isActive ? "3" : "2"}
        markerEnd={`url(#${isActive ? "dag-arrow" : ""}`}
        className="transition-all duration-300"
      />
    );
  })}
})}

```

```

{/* Nodes */}
{dagNodes.map((node) => {
  const isCurrent = step.currentNode === node.id
  const isVisited = step.visited.has(node.id)
  const isProcessed = step.fullyProcessed.has(node.id)

```

```

    let fill = "#1e293b";
    let stroke = "#334155";
    let textColor = "text-slate-400";
    void textColor;

```

```

    if (isCurrent) {
      fill = "#312e81";
      stroke = "#818cf8";
      textColor = "text-white font-bold";
    } else if (isProcessed) {
      fill = "#022c22";
      stroke = "#059669";
      textColor = "text-emerald-300";
    } else if (isVisited) {
      fill = "#3730a3";
      stroke = "#6366f1";
      textColor = "text-indigo-200";
    }

```

```

    }}}}
  </svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
  00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800">
    <p className="text-xs text-indigo-300 min-h-12">
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto">
    {/* Topological List output */}
    <div>
      <span className="text-[10px] text-slate-500">
        РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
      </span>
      <div className="bg-slate-900/60 p-2.5 rounded
        pr-1 text-slate-300">
        {step.topoList.length === 0 ? (
          <span className="text-slate-600 text-sm">
            ожидаем обработку узлов...
          </span>
        ) : (
          getTopoListString()
        )}
      </div>
    </div>

    {/* Color key */}
    <div className="space-y-1">
      <span className="text-[10px] text-slate-500">

```

```
        </div>
      </div>
    </div>
  </div>
</div>
);
};
```

ФАЙЛ 67/82: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useMemo, useState } from "react";
```

```
/**
```

```
 * Интерактивный Треп – шесть режимов:
```

```
 *   build   – сборка: ВИДИМАЯ сортировка точек (и чем сортировать, так и сортируем)
```

```
 *           координатная плоскость НАВЕРХУ (ассоциативная сортировка)
```

```
 *           чек-лист инвариантов (автопроверка, как у бинарного поиска)
```

```
 *   split   – разрез по x с двумя корзинами (грамотная сортировка)
```

```
 *   merge   – склейка двух деревьев по y.
```

```
 *   erase   – удаление ПРОИЗВОЛЬНОЙ точки: найти по x, удалить по y.
```

```
 *   layers  – миф «дети в массиве по  $2k/2k+1$ »: работает, но неэффективно
```

```
 *   search  – «отсортирую бинарным поиском»: поиск не сортировкой
```

```
 *           сортировка платит  $\geq n \cdot \log n$  сравнений, сортировка
```

```
 *           сортировка платит  $\geq n \cdot \log n$  сравнений, сортировка
```

```
 * Канонический набор: все x и все y различны  $\Rightarrow$  дерево бинарного поиска
```

```
 * Узел идентифицируется парой  $id = "x:y"$ .
```

```
 */
```

```
export interface Pair {
```

```
  key: number;
```

```
  grade: number;
```

```
}
```

```
type NodeId = string;
```

```
const pairId = (key: number, grade: number): NodeId =>
```

```

export function insert(root: TNode | null, p: Pair): TNode | null {
  if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade };
  if (p.key <= root.key) root.left = insert(root.left, p);
  else root.right = insert(root.right, p);
  return root;
}

export function buildTree(ps: Pair[]): TNode | null {
  let root: TNode | null = null;
  for (const p of sortedByY(ps)) root = insert(root, p);
  return root;
}

export interface SplitResult {
  l: TNode | null;
  r: TNode | null;
  trace: { atId: NodeId; goLeft: boolean; note: string };
}

export function splitTree(root: TNode | null, x: number): SplitResult {
  const trace: SplitResult["trace"] = [];
  const go = (t: TNode | null): { l: TNode | null; r: TNode | null } | null {
    if (!t) return { l: null, r: null };
    if (t.key <= x) {
      const sub = go(t.right);
      trace.push({ atId: t.id, goLeft: true, note: `(${t.key} > x)
        о правое поддерево.` });
      t.right = sub.l;
      return { l: t, r: sub.r };
    }
    const sub = go(t.left);
    trace.push({ atId: t.id, goLeft: false, note: `(${t.key} < x)
      о левое поддерево.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
  };
  const res = go(root);
  return { l: res.l, r: res.r, trace };
}

```

```

        trace.push({ atId: t.id, note: `${key} > ${t.key}` });
        t.right = go(t.right);
        return t;
    }
    trace.push({ atId: t.id, note: `Нашли (${t.key}), ${t.key} > ${t.key}` });
    const m = mergeTree(t.left, t.right);
    trace.push(...m.trace.map((s) => ({ atId: s.chosenId, note: s.note })));
    return m.root;
};
return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
    t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

```

/* ----- валидация treap (для чек-листа и стрелочек) ----- */

```

export function checkTreap(root: TNode | null): { bst: boolean, heap: boolean } {
    let bst = true;
    let heap = true;
    let prev: number | null = null;
    const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null): void => {
        if (!n) return;
        if (n.key <= min || n.key > max) bst = false;
        if (parentGrade !== null && n.grade > parentGrade) heap = false;
        walk(n.left, min, n.key, n.grade);
        if (prev !== null && n.key < prev) bst = false;
        prev = n.key;
        walk(n.right, n.key, max, n.grade);
    };
    walk(root, -Infinity, Infinity, null);
    return { bst, heap };
}

```

/* ----- раскладка сцены дерева ----- */

```

const layoutTree = (root: TNode | null) => {

```

```

const TreeView: React.FC<TreeViewProps> = ({ root, width, height }) => {
  const pos = useMemo(() => layoutTree(root), [root]);
  const nodes = useMemo(() => [...pos.entries()].map(([id, n]) => n));
  const edges = useMemo(() => treeEdges(root), [root]);
  const autoW = Math.max(420, ...nodes.map((n) => n.px + n.rx));
  const autoH = Math.max(260, ...nodes.map((n) => n.py + n.rh));
  const w = width ?? autoW;
  const h = height ?? autoH;
  const byId = (id: NodeId) => nodes.find((n) => n.id === id);
  return (
    <div>
      <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
        {edges.map((e) => {
          const a = byId(e.from);
          const b = byId(e.to);
          if (!a || !b) return null;
          return (
            <line
              key={`te-${e.from}-${e.to}`}
              x1={a.px} y1={a.py} x2={b.px} y2={b.py}
              stroke={edgeColor ? edgeColor(e.from, e.to) : "#ccc"}
              strokeWidth={2.5}
              strokeDasharray={edgeDash ?? edgeDash(e.from, e.to)}
              className="transition-all duration-500"
            />
          );
        })}
        {nodes.map((n) => {
          const p = pairParts(n.id);
          const pulse = pulseId === n.id;
          return (
            <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`}
              <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
              />
              {strokeWidth={pulse ? 4 : 2} className="transition-all duration-500"}
              <text y={-2} fill="#fff" fontSize="13" fontWeight="bold">{n.name}</text>
              <text y={13} fill="#cbd5e1" fontSize="10">{n.id}</text>
            </g>
          );
        })}
      </svg>
    </div>
  );
};

```

```

    )))
    {[-4, 0, 4, 8].map((g) => {
      <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) +
    )))
    <text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8"
    <text x={PX(1) - 26} y="14" fill="#94a3b8" fontSi

    {drawnEdges.map((e, i) => {
      const a = byPair(e.a);
      const b = byPair(e.b);
      return <line key={`pe-${i}`} x1={a.x} y1={a.y}
        ration-500" />;
    )))

    {points.map((p) => {
      const done = drawnEdges.some((e) => (e.a.key ===
      return (
        <g key={`pp-${p.key}:${p.grade}`} className="
          <circle cx={PX(p.key)} cy={PY(p.grade)} r={
            b8"} strokeWidth={1.6} />
          <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
            ">{p.key}</text>
        </g>
      );
    )))

    {pending && (
      <g className="transition-all duration-500">
        <circle cx={PX(pending.key)} cy={PY(pending.g
          >
        <text x={PX(pending.key)} y={PY(pending.grade
          ly="monospace">{pending.key}</text>
        </g>
      )}
    </svg>
  );
};

```



```

const [sortIdx, setSortIdx] = useState(0);
const [connIdx, setConnIdx] = useState(0);
const [playing, setPlaying] = useState(false);

const parsed = useMemo(() => parsePairs(raw), [raw]);
const userPairs = parsed.errors.length ? [] : parsed.pairs;

const n = userPairs.length;

const userTree = useMemo(() => (userPairs.length >= 1 ?
const canonEdges = useMemo(() => {
  const es: { a: Pair; b: Pair }[] = [];
  const walk = (node: TNode | null) => {
    if (!node) return;
    if (node.left) { es.push({ a: { key: node.key, group: node.group, left: node.left, right: node.right } });
    if (node.right) { es.push({ a: { key: node.key, group: node.group, left: node.left, right: node.right } });
    walk(node.left);
    walk(node.right);
  };
  walk(userTree);
  return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges].filter(e => e.a.group < e.b.group));

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => {
    if (phase === "sort") {
      if (algo === "none") { setPlaying(false); return; }
      if (sortIdx < 3) setSortIdx(sortIdx + 1);
      else setPlaying(false);
    } else {
      const total = order === "y-desc" ? canonEdges.length : leftRightEdges.length;
      if (connIdx < total) setConnIdx(connIdx + 1);
      else setPlaying(false);
    }
  }, 900);
  return () => clearTimeout(t);
}, [playing, phase, algo, sortIdx, connIdx, order, canonEdges, leftRightEdges]);

```

```

    {parsed.errors.length === 0 && (parsed.equalX |
      <div className="mt-2 text-xs text-amber-300 b
        {parsed.equalY && <div>✂ Есть <b>равные y</b>
          кто <b>раньше в списке</b> (сортировка ста
        {parsed.equalX && <div>✂ Есть <b>равные x</b>
          de.key → left).</div>}
      </div>
    )}
  </div>

  {/* Координатная плоскость – НАВЕРХУ */}
  <div className="bg-emerald-950/20 rounded-xl p-4 l
    <div className="flex flex-wrap items-center jus
      <h4 className="text-base font-bold text-emera
        <div className="flex items-center gap-2">
          {phase === "sort" ? {
            <>
              <span className="text-xs text-slate-400
                {[ "quicksort", "counting" ] as const}.m
              <button
                key={a}
                onClick={() => { setAlgo(a); setSor
                disabled={n < 2}
                className={`px-3 py-1.5 rounded-lg t
                600 text-white" : "bg-slate-700 text-slate-1
              >
                {a === "quicksort" ? "quicksort (cp
              </button>
            )}}
          </>
        ) : (
          <>
            <span className="text-xs text-slate-400
            <button
              onClick={() => startConnect("y-desc")}
              className={`px-3 py-1.5 rounded-lg te
            bg-emerald-600 text-white" : "bg-slate-700
          >

```

```

    </div>

    {/* Чек-лист инвариантов */}
    <div className={`rounded-xl p-4 border transition
      bg-slate-900/40`} `>
      <h4 className="text-sm font-bold text-slate-300">
        <ul className="space-y-1.5 text-sm">
          <li>{connDone ? " " : " "} каждое ребро ведёт
          <li>{connDone ? " " : " "} обход слева-направ
          <li>{connDone ? " " : " "} линий ровно <b>n -
        </ul>
      </div>

      {/* Дерево */}
      {order === "y-desc" && userTree && (
        <div className="bg-slate-900/40 rounded-xl p-4">
          <h4 className="text-sm font-bold text-slate-300">
            <TreeView root={userTree} subtitle="Слева-нап
          </div>
        )}
      </div>
    );
  };
};

/* ===== РЕЖИМ SPLIT ===== */

const SplitMode: React.FC = () => {
  const [x0, setX0] = useState(7);
  const [stepIdx, setStepIdx] = useState(0);
  const fullTree = useMemo(() => buildTree(POINTS), [])
  const split = useMemo(() => splitTree(buildTree(POINT
  const steps = split.trace;
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

  const inL = new Set<NodeId>();
  const inR = new Set<NodeId>();
  steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? in

```



```

const [key, setKey] = useState(9);
const [stepIdx, setStepIdx] = useState(0);
const fullTree = useMemo(() => buildTree(POINTS), []);
const res = useMemo(() => eraseNode(buildTree(POINTS)));
const steps = res.trace;
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;

const pathIds = useMemo(() => {
  const ids: NodeId[] = [];
  for (const s of steps.slice(0, idx + 1)) if (s.atId)
    return ids;
}, [steps, idx]);
const pathSet = new Set(pathIds);

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm text-slate-300">Удалить</span>
      <select value={key} onChange={(e) => { setKey(N
        te-700 rounded-lg px-3 py-1.5 text-sm font-mo
        {POINTS.map((p) => <option key={p.key} value=
      </select>
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:o
      <span className="text-xs text-slate-400">шаг {i
    </div>

    {!done ? (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <TreeView
          root={fullTree}
          nodeColor={(id) => (pathSet.has(id) ? "#783
          nodeStroke={(id) => (pathSet.has(id) ? "#f5
          subtitle="Куча сама умеет удалять только ве
        />

```

```

        walk(n.right, 2 * i + 1);
    };
    walk(full, 1);
    return arr;
}, [full]));
const nodeCount = slots.filter(Boolean).length;

return (
  <div className="space-y-4">
    <div className="flex gap-2">
      <button onClick={() => setPick("heap")} className="bg-slate-700 text-white" />
      <button onClick={() => setPick("treap")} className="bg-slate-700 text-white" />
    </div>

    {pick === "heap" ? (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-slate-300">
          4>
        <div className="flex gap-2 overflow-x-auto mb-2">
          {heapArr.slice(1).map((v, i) => (
            <div key={i} className="px-3 py-2 rounded">
              <div className="text-slate-500 text-[10px]>
                <div className="text-white font-bold">{v}</div>
                <div className="text-emerald-400 text-[10px]>
                  </div>
              </div>
            </div>
          ))}
        </div>
        <p className="text-xs text-slate-400">Полное дерево
          →  $2i/2i+1$  попадает в узел.</p>
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-rose-300">
          >
        <TreeView root={full} subtitle="Попробуй «посмотреть»>
          </div>

```


Есть ли разница, чем сортировать? Сравнения граница, каждое сравнение даёт 1 бит. Но для маленьких целых у (вкладка «Собрать», кнопка «Задать порядок соединения».

```

    </div>
  </div>
);
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "layers";

const TABS: { id: Tab; label: string }[] = [
  { id: "build", label: "Собрать" },
  { id: "split", label: "Split ≈ " },
  { id: "merge", label: "Merge " },
  { id: "erase", label: "Erase " },
  { id: "layers", label: "Миф: 2k/2k+1" },
  { id: "search", label: "Поиск ≠ сортировка" },
];

export const TreapBuildViz = () => {
  const [tab, setTab] = useState<Tab>("build");

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border">
      <div className="flex flex-wrap items-center justify-center">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-emerald-600 text-white">
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">
              <p className="text-sm text-emerald-300">Параметры
            </p>
          </div>
        </div>
      </div>
    </div>
  );
};

```



```

-----
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton";
import { SegmentTreeVisualizer } from "./SegmentTreeViz";
import { SplayTreeViz } from "./SplayTreeViz";
import { SplayRotationsViz } from "./SplayRotationsViz";
import { SplayRotationSandbox } from "./SplayRotationSandbox";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimation";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

```

```

/** Разбор поворота и песочница всегда идут парой: посмотри
**

```

```

* Анимация и песочница идут друг под другом, а не в две
* колонке дерево сжималось до нечитаемого размера, а по
* компактности. Между ними – подпись, объясняющая перемену
*/

```

```

const SplayRotationsPair: React.FC = () => (
  <div className="space-y-4">
    <SplayRotationsViz />
    <p className="flex items-start gap-2 text-[12px] leading-
      3 py-2">
      <span aria-hidden="true"> </span>
      <span>

```

```

    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы — куча сама держит лучшую",
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .",
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разрезанная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку — подсветится",
    Component: SparseTableViz,
  },
  treap: {
    title: "Treap: плоскость → дерево → Split/Merge/Erge",
    hint: "6 вкладок: Собрать (сортировка + соединение)
      , миф про  $2k/2k+1$ , поиск  $\neq$  сортировка.",
    Component: TreapBuildViz,
  },
  "prefix-sums-2d": {
    title: "Префиксные суммы (1D и 2D)",
    hint: "Наведите на число — увидите, из чего оно сложилось",
    Component: PrefixSumViz,
  },
  "dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": {
    title: "Splay-дерево",
    hint: "Покадровый разбор трёх поворотов, песочница",
    Component: () => (
      <div className="space-y-10">
        <SplayRotationsPair />
        <SplayTreeViz />
      </div>
    )
  }
}

```

```

        <FloydViz />
      </>
    ),
  },
  "johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonAlgo },
  "mst-kruskal": { title: "Краскал и DSU", Component: KruskalSimulator },
  "mst-prima": { title: "Прим", Component: KruskalSimulator },
  "mst-boruvka": { title: "Борувка", Component: KruskalSimulator },
  "string-kmp": { title: "Префикс-функция (КМП)", Component: KMP },
  "string-z-func": { title: "Z-функция", Component: ZFunc },
  "aho-corasick": {
    title: "Ахо-Корасик",
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
      <div className="space-y-10">
        <WaterfallAnimationWidget />
        <SalmonAutomatonWidget />
      </div>
    ),
  },
  intro: { title: "Мнемокарточки", Component: MnemonicCards },
  "everyday-basics": {
    title: "Бытовой тренажёр: стек, очередь, куча",
    hint: "Тарелки, очередь в столовой и мешок гипотез",
    Component: Simulator,
  },
};

```

```
export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 69/82: src/components/WaterfallAnimationWidget.tsx
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

```

```

3: { S: 4 },
4: {},
5: { S: 6 },
6: {},
7: { H: 8 },
8: { E: 9 },
9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training'>('training')

  // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
  const [textToScan, setTextToScan] = useState<string>('')
  const [currentIndex, setCurrentIndex] = useState<number>(0)
  const [currentNode, setCurrentNode] = useState<number>(0)
  const [isSearchDone, setIsSearchDone] = useState<boolean>(false)
  const [searchLog, setSearchLog] = useState<string>('')
  'Режим поиска: Введите текст и жмите «Следующий шаг»'
  );
  const [jumpPath, setJumpPath] = useState<{ from: number, to: number }>({ from: 0, to: 0 })
  const [foundMatches, setFoundMatches] = useState<{ index: number, value: string }[]>([])
  const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0)

  // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
  // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
  const [trainStep, setTrainStep] = useState<number>(0)

  const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const clean = e.target.value.toUpperCase().replace(/ /g, '');
    setTextToScan(clean);
    resetSearchSimulation();
  };

  const resetSearchSimulation = () => {
    setCurrentIndex(0);
    setCurrentNode(0);
  };

```

```

    if (originalCurr === 0) {
      logMsg += `В корне нет перехода по '${char}'. Л
      setJumpPath(null);
    } else {
      logMsg += `Поток по '${char}' у вершины #${orig
      ружок по fail-ссылке в #${curr} (${WATERFALL_
      . `;
      setJumpPath({ from: originalCurr, to: nextNode,
    }
    curr = nextNode;
  }

const currentMatches: { index: number; word: string
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
  if (WATERFALL_NODES[temp].is_terminal) {
    WATERFALL_NODES[temp].words.forEach((word) :
      currentMatches.push({ index: currentInd
      matchedWordsForLog.push(word);
    });
  }
  temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
  logMsg += ` БИНГО! Найдено слово: ${matchedWords
  setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
  setFoundMatches((prev) => [...prev, ...currentMat
}
};

```

```

title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель :
    а-разведчик плавает в ROOT и ищет ветку по "E".',
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'E',
checkingBranchesFrom: 0,
},
{
    targetNodeId: 5,
    title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
    desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
        H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'I',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 8,
    title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ
    desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
        ). Рыба плавает в ROOT и ищет ветку "H". У корня
        суффикс "H"!',
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 3,
    title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
    desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
        е подходят. fail[HER] = ROOT.',
    codeLine: 3,

```

```

    ];

    const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
    const activeFishPosNode = activeMode === 'training' ?
    const targetTrainingNode = WATERFALL_NODES[currentTrainStep];

    return (
      <div className="bg-slate-800/95 rounded-2xl p-6 border-1px border-slate-700">
        {/* Шапка и переключатель режимов */}
        <div className="flex flex-wrap items-center justify-between">
          <div className="flex items-center space-x-3">
            <span className="text-3xl"> </span>
            <div>
              <h4 className="text-2xl font-bold text-white">
                Анимация водопада: Полное обучение автомата
              </h4>
              <p className="text-sm text-slate-400">
                Визуализация пошагового построения всех функций
              </p>
            </div>
          </div>
        </div>

        {/* Переключатель режимов */}
        <div className="bg-slate-900 p-1.5 rounded-xl border-1px border-slate-800">
          <button
            onClick={() => setActiveMode('training')}
            className={`flex items-center space-x-1.5 p-2 transition-colors duration-200
              ${activeMode === 'training'
                ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
                : 'text-slate-400 hover:text-white'}
            `}>
            <GitBranch className="w-4 h-4" />
            <span>Режим 1: Полное обучение (BFS от корня)</span>
          </button>
          <button
            onClick={() => setActiveMode('search')}
            className={`flex items-center space-x-1.5 p-2 transition-colors duration-200
              ${activeMode === 'search'
                ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
                : 'text-slate-400 hover:text-white'}
            `}>
            <GitBranch className="w-4 h-4" />
            <span>Режим 2: Поиск по узлам</span>
          </button>
        </div>
      </div>
    );
  }
}

```

```

        {idx === trainStep && <span className=
        </button>
      }}}
    </div>
  </div>

  <div className="flex gap-2">
    <button
      onClick={() => setTrainStep((prev) => M
      disabled={trainStep === 0}
      className="flex-1 bg-slate-700 hover:bg
ion-all"
    >
      Назад
    </button>
    <button
      onClick={() => setTrainStep((prev) => M
      disabled={trainStep === TRAINING_STEPS_
      className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
    >
      {trainStep === TRAINING_STEPS_INFO.leng
    </button>
  </div>
</div>

  /* Блок с кодом построения fail-ссылки и опи
  <div className="lg:col-span-2 bg-slate-900/90
  >
    <div>
      <h5 className="text-white font-bold mb-3
      <span className="flex items-center gap-
        <span></span> Код BFS построения fail
      </span>
      <span className="text-xs bg-indigo-950
        Вершина: #{targetTrainingNode.id} ({t
      </span>
    </h5>

```



```

        <Lightbulb className="w-4 h-4 text-amber-400" />
      </h5>
      <div className="bg-slate-800 p-4 rounded-lg flex flex-direction-column align-items-center justify-content-center" style={{width: innerWidth, min-height: [72px], flex: 1, itemsCenter: true}}>
        {currentTrainInfo.desc}
      </div>
    </div>
  </div>
) : (
  /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
  <div className="grid grid-cols-1 lg:grid-cols-3" style={{width: innerWidth}}>
    <div className="bg-slate-900/60 p-5 rounded-xl" style={{width: 100%}}>
      <div>
        <h5 className="text-white font-bold mb-2">
          <span>⚡ </span> Текст для сканирования
        </h5>
        <p className="text-xs text-slate-400 mb-4">Введи текст, чтобы лосось просканировал</p>
        <input
          type="text"
          value={textToScan}
          onChange={handleTextChange}
          maxLength={15}
          placeholder="ВВЕДИТЕ ТЕКСТ..."
          className="w-full bg-slate-800 border border-slate-700 focus:outline-none focus:border-blue-500"
        />
      </div>
    </div>
    <div>
      <div className="mb-4 flex flex-wrap gap-1">
        <span className="text-xs text-slate-400">{textToScan.split('').map((char, idx) => {
          <span
            key={idx}
            className={`w-7 h-8 flex items-center justify-center rounded-sm`}
          >{char}</span>
        })}</span>
      </div>
    </div>
  </div>
)

```

```

        <div className="bg-slate-950 p-4 rounded-
          <div className={`p-1.5 rounded flex item
border-blue-500 text-blue-300 font-bold' :
          <span>1. curr = state; char = text[i]
            {activeSearchCodeLine === 1 && <span <
инициализация</span>}
          </div>
          <div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
          <span>2. while (curr !== root && !trie
            {activeSearchCodeLine === 2 && <span <
рыжок по fail!</span>}
          </div>
          <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold' :
          <span>3. curr = trie[curr].get(char)
            {activeSearchCodeLine === 3 && <span <
ем по течению</span>}
          </div>
          <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
          <span>4. if (is_terminal[curr]) report
            {activeSearchCodeLine === 4 && <span <
овпадение!</span>}
          </div>
        </div>
      </div>
    </div>

<div>
  <h5 className="text-slate-300 font-bold ml
    <AlertCircle className="w-3.5 h-3.5 tex
  </h5>
  <div className="bg-slate-800 p-3.5 rounder
tems-center">
    {searchLog}
  </div>
</div>
</div>

```

```

    { depth: 3, y: 360, label: 'Depth 3 (HER,
      { depth: 4, y: 460, label: 'Depth 4 (HERS
    ].map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line
          x1="20"
          y1={level.y}
          x2="780"
          y2={level.y}
          stroke="#334155"
          strokeWidth="1"
          strokeDasharray="4,4"
          opacity="0.4"
        />
        <rect x="25" y={level.y - 12} width="14"
      />
      <text x="32" y={level.y + 4} fill="#94a5b8"
        {level.label}
      </text>
    </g>
  }}}

```

```

{/* Рендер прямых переходов (потоков воды) */
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
  const fromNode = WATERFALL_NODES[Number(fromId)];
  return Object.entries(children).map(([childId, child]) => {
    const toNode = WATERFALL_NODES[toId];

```

```

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ||
    type === 'normal';

```

```

    // Подсветка в режиме обучения (когда ищем совпадения)
    let isTrainingExamined = activeMode === 'training' ||
    let isMatchBranch = isTrainingExamined && childId === toId;

```

```

    return (
      <g key={`edge-${fromNode.id}-${toNode.id}`}>

```

```
}}}
```

```
/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
  if (node.id === 0) return null;
  const targetNode = WATERFALL_NODES[node.fail];
```

```
  // В режиме обучения показываем fail-ссылку
  // Найдем индекс шага, на котором обработали
  const stepIdxForNode = TRAINING_STEPS_INFO[activeStep];
  if (activeMode === 'training' && trainStep === stepIdxForNode)
```

```
    let isHighlighted = activeMode === 'search' || activeMode === 'fail';
    let isJustEstablished = activeMode === 'training' && trainStep === stepIdxForNode;
    // Рендер fail-ссылку
```

```
    // Вычисляем изогнутую кривую для красоты
    const dx = targetNode.x - node.x;
    const dy = targetNode.y - node.y;
    const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -10 : 10);
    const cy = node.y + dy / 2 - 30;
```

```
    return (
      <g key={`fail-${node.id}`}>
        <path
          d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`
          fill="none"
          stroke={isHighlighted ? '#f43f5e' : '#ccc'}
          strokeWidth={isHighlighted || isJustEstablished ? 2 : 1}
          strokeDasharray="6,6"
          className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
          opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
        />
        {isJustEstablished && (
          <text x={cx} y={cy - 10} fill="#10b98f">
            fail[{node.label}] = {targetNode.label}
          </text>
        )}
      </g>
    );
  }
}
```

```

        fontSize={isCurrent || isTargetChild ? 12 : 10}
        fontWeight="bold"
        textAnchor="middle"
    >
        {node.label}
    </text>

    {/* Метка искомой вершины в режиме обучения */}
    {isTargetChild && {
        <g transform={`translate(${node.x}, ${node.y})`} >
            <rect x="-35" y="-12" width="70" height="24" fill="#8b572a" />
            <text x="0" y="2" fontSize="10" fontStyle="italic" >
                ИЩЕМ FAIL
            </text>
        </g>
    }}

    {/* Индикатор словарных слов */}
    {hasWords && {
        <g transform={`translate(${node.x} + 10, ${node.y})`} >
            <circle cx="0" cy="0" r="9" fill="#8b572a" />
            <text x="0" y="3" fontSize="10" fontStyle="italic" >
                *
            </text>
        </g>
    }}
    </g>
    </g>
    </g>
    </g>
    </g>

    {/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
    <g>
        {/* Круг-подложка под рыбу */}
        <circle
            cx={activeFishPosNode.x + 15}
            cy={activeFishPosNode.y - 25}
            r="18"
            fill={activeMode === 'training' ? '#8b572a' : '#8b572a'}
        />
    </g>

```

```
-----
    -sm font-bold shadow animate-[scaleUp_0.3s_
      >
        <CheckCircle2 className="w-4 h-4 text
        <span>{match.word}</span>
        <span className="text-[10px] bg-emera
          Индекс {match.index}
        </span>
      </div>
    )})
  </div>
})
</div>
});
};
```

=====

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

=====

ФАЙЛ 70/82: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ:

```
import React, { useMemo, useState } from "react";
```

```
/**
```

```
 * Префиксные суммы: 1D и 2D.
```

```
 *
```

```
 * Главная фишка – наведение на число:
```

```
 *   * в 1D наводим на P[i] – подсвечивается кусок массива
```

```
 *   * в 1D наводим на a[i] – видно, в какие префиксы он
```

```
 *   * в 2D наводим на ячейку – подсвечивается прямоугольник
```

```
 *   а при выбранном запросе показывается формула включ
```

```
 */
```

```

    });
  };

  /* ----- 1D -----
const Prefix1D: React.FC = () => {
  const pref = useMemo(
    () => A1.reduce<number[]>((acc, v) => [...acc, acc[
    []
  ]);

  /** Что сейчас под курсором: префикс P[i] или элемент
const [hover, setHover] = useState<{ kind: "pref" | "a"
const [range, setRange] = useState<{ l: number; r: num

const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        {/* индексы */}
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => (
            <div key={i} className={` ${cellBase} !h-5
              {i}
            </div>
          ))}
        </div>

        {/* массив a */}
        <div className="flex gap-1.5 items-center mb-1

```

```

        active
        ? "bg-emerald-500 text-white ring
        : isR
        ? "bg-emerald-700 text-white"
        : isL
        ? "bg-rose-700 text-white"
        : "bg-slate-900 text-slate-300"
    }` }
    title={`P[${i}] = сумма a[0..${i} - 1]
  >
    {v}
  </div>
  );
  }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border
  {hover?.kind === "pref" ? {
    hover.i === 0 ? {
      <p className="text-slate-300">
        <b className="text-emerald-400">P[0] = 0<
        отрезка, начинающегося с нуля.
      </p>
    } : {
      <p className="text-slate-300">
        <b className="text-emerald-400">
          P[{hover.i}] = {pref[hover.i]}
        </b>{" "}
        – это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
          a[0..{hover.i} - 1] = {A1.slice(0, hove
        </span>
      </p>
    }
  } : hover?.kind === "a" ? {

```



```

const m = A2[0].length;

const P = useMemo(() => {
  const p = Array.from({ length: n + 1 }, () => Array(
    for (let i = 1; i <= n; i++) {
      for (let j = 1; j <= m; j++) {
        p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i]
      }
    }
  ), [n, m]);

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">Исходная матрица A2</div>
        <div className="inline-block">
          {A2.map((row, i) => (
            <div key={i} className="flex gap-1.5 mb-1"

```

```

        ? "bg-rose-600 text-white"
        : "bg-slate-900 border border
return (
  <div
    key={j}
    onMouseEnter={() => setHover({ i,
    onMouseLeave={() => setHover(null
    className={`${cellBase} cursor-he
      isHover ? "bg-amber-500 text-wh
    }`}
  >
    {v}
    {corner && !isHover && (
      <span className="absolute -top-
ht">
      {corner}
    </span>
    )}
  </div>
);
}}
</div>
)}}
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border
  {hover ? (
    <p className="text-slate-300">
      <b className="text-amber-400">
        P[{hover.i}][{hover.j}] = {P[hover.i][hov
      </b>{" "}
      — сумма всего прямоугольника от левого верх
    </p>
  ) : (
    <p className="text-slate-500">Наведите курсор
  )}

```

```
-----
const st1D: number[][] = Array.from({ length: N }, () =>
  for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j) - 1][j - 1]);
    }
  }
}
```

// Data for 2D Matrix (Sparse Table 2D)

```
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
];
```

```
const ST2D: number[][][][] = Array.from({length: 8}, () =>
  Array.from({length: 8}, () =>
    Array.from({length: 4}, () =>
      Array(4).fill(0)
    )
  )
);
```

```
for (let r = 0; r < 8; r++) {
  for (let c = 0; c < 8; c++) {
    ST2D[r][c][0][0] = val2D[r][c];
  }
}
```

```
for (let kx = 0; kx <= 3; kx++) {
  for (let ky = 0; ky <= 3; ky++) {
    if (kx === 0 && ky === 0) continue;
    for (let r = 0; r + (1 << kx) <= 8; r++) {
```

```

        onClick={() => setTab("2d_query")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-
            se-600 text-white" : "bg-slate-800 text-sla
    >
    3. Запрос 2D (Формула)
  </button>
</div>

  <AnimatePresence mode="wait">
    {tab === "1d" ? (
      <Viz1D key="1d" />
    ) : tab === "2d_build" ? (
      <Viz2DBuild key="2d_build" />
    ) : (
      <Viz2DQuery key="2d_query" />
    )}
  </AnimatePresence>
</div>
  );
};

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number}>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  const covered = hover ? { from: hover.i, to: hover.i + 1 } : { from: 0, to: 0 };

  return (
    <div className="flex flex-col gap-5">

```

```

        return (
          <div key={idx} className="flex flex-col i
            <div
              className={`flex items-center justify
w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
              inCover ? "bg-indigo-500 text-white
              }` }
            >
              {v}
            </div>
            <span className="text-[9px] text-slate-
          </div>
        );
      })))
    </div>
    <div className="mt-2 text-xs min-h-[36px]">
      {hover && covered ? (
        <p className="text-slate-300">
          <b className="text-sky-400">
            ST[{hover.i}][{hover.j}] = {stID[hover.i
          </b>{" "}
          – это минимум на отрезке{" "}
          <span className="font-mono text-sky-300">
            [{covered.from}, {covered.to}]
          </span>{" "}
          длины  $2^{\text{hover.j}}$  = {1 << hover.j}:{" "}
          <span className="font-mono text-slate-400">
            min({arrID.slice(covered.from, covered.
          </span>
        </p>
      ) : (
        <p className="text-slate-500">
          Наведите курсор (или тапните) на любое чи
        </p>
      )}
    </div>
  </div>

```

```

// Active computing state
let isActive = false;
let isSrc1 = false;
let isSrc2 = false;

if (step > 0 && step <= maxStep) {
  const currentStep = computeSteps[step];
  if (currentStep.i === i && currentStep.j === j) {
    isActive = true;
    if (currentStep.j === j + 1 && currentStep.i === i) {
      isSrc1 = true;
    }
    if (
      currentStep.j === j + 1 &&
      currentStep.i === i - (1 << (currentStep.i - i))
    ) {
      isSrc2 = true; // wait, no. Src2
    }
    // Let's re-eval exact source:
    // We are asking if THIS cell [i][j]
    if (currentStep.j - 1 === j) {
      if (currentStep.i === i) isSrc1 = true;
      if (currentStep.i + (1 << (currentStep.i - i)) === i) {
        isSrc2 = true;
      }
    }
  }
}

return (
  <div key={j} className="relative flex" >
    {!isValid ? (
      <div className="w-[clamp(30px,90px,100px)] h-[clamp(30px,90px,100px)]" >
    ) : (
      <motion.div
        onMouseEnter={() => isVisible = true}
        onMouseLeave={() => setHover(false)}
        onClick={() => isVisible && setHover(true)}
        initial={{ opacity: 0, scale: 0.8 }}
        animate={{
          opacity: isVisible ? 1 : 0,
          scale: isVisible ? 1 : 0.8,
        }}
      />
    )}
  </div>
)

```

```

        strokeWidth="3"
      />
    </motion.svg>
  )}
</div>
  );
  })}
</React.Fragment>
  )})
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
  {step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-b
              ST[{c.i}][{c.j - 1}]
            </span>
            ,
            <span className="text-amber-400 font-bo
              ST[{c.i + half}][{c.j - 1}]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{st1
            <span className="text-amber-400">
              {st1D[c.i + half][c.j - 1]}
            </span>
          ) ={" "}

```



```

        return (
            <>
                <div className="flex item
                    <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
                    <span className="text
tiveCell.r][activeCell.c][k-1][k-1]}</span>
                </div>
                <div className="flex item
                    <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
                    <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
                </div>
                <div className="flex item
                    <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
                    <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
                </div>
                <div className="flex item
                    <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{activ
                    <span className="text
[activeCell.r + prevL][activeCell.c + prevL
                </div>
            </>
        )
    } ) ( ) }
</div>

    <div className="pt-3 mt-3 border-t
        ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>

```

```

<div className="grid grid-cols-[repeat(
ative shadow-inner w-max max-w-full">
  {Array.from({length: 64}).map((_,
    const r = Math.floor(idx / 8);
    const c = idx % 8;
    const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2

    return (
      <div key={idx} className={`text-rose-500 text-center rounded text-[clamp(9px,2.4vw,12px)] border-rose-500/50 border scale-105 z-10`}>
        {val2D[r][c]}
      </div>
    )
  )}
  )}
  )}

```

```

<div
  className="absolute border-[3px solid #f43f5e] border-da
  style={{
    top: `${(r1 / 8) * 100}%`,
    left: `${(c1 / 8) * 100}%`,
    marginTop: '8px',
    marginLeft: '8px',
    height: 'calc(' + (H/8*100) + 'px)',
    width: 'calc(' + (W/8*100) + 'px)',
  }}
/>

```

```

{/* Показываем 4 угла в самой матрице */}
<div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
lc(${(Ly / 8) * 100}% - 16px)`, height: `calc(
  <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
  width: `calc(${(Ly / 8) * 100}% - 16px)`, left: `calc(
  <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(
  width: `calc(${(Lx / 8) * 100}% - 16px)`, left: `calc(

```

Чтобы покрыть весь прямоугольник, мы
 Name="font-mono bg-[#0a0f1e] px-1 rounded b
 роса. (Цветные прямоугольники слева).

</p>

```
<div className="bg-[#0a0f1e] rounded-xl
text-slate-300 shadow-inner overflow-x-auto
  <div className="absolute right-3 top-3
    <span className="text-slate-500">// 1.
    <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
    <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
    <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
    <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b
```

```
    <span className="text-slate-500">// 2.
    <span className="text-indigo-400">int<
    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">В запроса)</span><br/>
```

```
    &nbsp;&nbsp;&nbsp;<span className="text-rose-950">r1</span>][<span className="text-white bg-r
```

```
    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">во, чтобы правый край уперся в c2)</span><b
    &nbsp;&nbsp;&nbsp;<span className="text-blue-500">r1</span>][<span className="text-white bg-b
```

```
    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">px, чтобы нижний край уперся в r2)</span><b
    &nbsp;&nbsp;&nbsp;<span className="text-emerald-500">nded">r2 - Lx + 1</span>][<span className="
```

```
    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">и влево от c2)</span><br/>
    &nbsp;&nbsp;&nbsp;<span className="text-amber-500">
```

```
        r-slate-700 text-white px-1 font-bold rounded
            </div>
        )
    }
  }
</div>
</div>
</div>

<div className="mt-auto bg-slate-950 font
)]">
    <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg bord
    <span className="text-slate-400 font
        <span className="text-rose-400">
        <span className="text-blue-400 m
n>
        <span className="text-emerald-40
span>
        <span className="text-amber-400 r
        &nbsp;<span className="text-slate-4
        <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
    </div>
  </div>

</div>
</div>
)
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

```
    });

    const observerOptions = {
      root: null,
      rootMargin: '-10% 0px -70% 0px',
      threshold: 0
    };

    const observer = new IntersectionObserver((
      entries.forEach(entry => {
        if (entry.isIntersecting && !document
          // Highlight active TOC link lo
          const id = entry.target.id;
          document.querySelectorAll('aside
            if (link.getAttribute('href
              link.classList.add('bg-
            } else {
              link.classList.remove('l
            }
          }));
        }
      }));
    }, observerOptions);

    document.querySelectorAll('section[id^="que
      observer.observe(section);
    });
  });
</script>
</body>
</html>
```

```
/* Global formula scroll settings */
.formula-scroll {
    overflow-x: auto;
    overflow-y: hidden;
    -webkit-overflow-scrolling: touch;
    scrollbar-width: none; /* Firefox */
    -ms-overflow-style: none; /* IE and Edge */
}
.formula-scroll::-webkit-scrollbar {
    display: none;
}

mjsx-container {
    max-width: 100% !important;
}
mjsx-container[display="true"] {
    overflow-x: auto;
    overflow-y: hidden;
    max-width: 100%;
    scrollbar-width: none;
    -ms-overflow-style: none;
    -webkit-overflow-scrolling: touch;
}
mjsx-container[display="true"]::-webkit-scrollbar {
    display: none;
}

@media (max-width: 640px) {
    html { font-size: 13px; }

    .uno-card { width: 40px; height: 60px; font-size: 10px; }
    .uno-card::before, .uno-card::after { font-size: 10px; }
    .uno-card::before { top: 1px; left: 2px; }
    .uno-card::after { bottom: 1px; right: 2px; }

    .uno-card.mini { width: 28px !important; height: 40px; font-size: 4px; }
    .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }
}
```



```

body.theme-light .bg-slate-800 { background-color: #1e293b; }
body.theme-light .bg-slate-700, body.theme-light .text-slate-800 { color: #1e293b; }
body.theme-light .text-slate-200, body.theme-light .text-slate-400 { color: #47556b; }
body.theme-light .border-slate-600, body.theme-light .border-slate-500 { border-color: #47556b; }
body.theme-light #top-controls { background-color: #1e293b; }

body.theme-notebook {
  background-color: #f8e8e8 !important; /* light pink */
  color: #3b2818 !important; /* brown ink */
  background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #f8e8e8 2px, #f8e8e8 4px);
  background-attachment: local !important;
  font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
  background-color: rgba(255, 255, 255, 0.7);
  border-color: #cbd5e1 !important;
  box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #1e293b; }
body.theme-notebook .text-slate-400 { color: #47556b; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-500 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace", "Courier New", "Serif"; }
body.theme-notebook #top-controls { background-color: #f8e8e8; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
  0%, 95%, 98%, 100% { transform: scaleY(1); }
  96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
  transform-origin: center;

```

```

        </div>
    </div>
</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
    <defs>
        <marker id="arrow-yellow" markerWidth="6" m
            <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
        </marker>
        <marker id="arrow-orange" markerWidth="6" m
            <path d="M0,0 L6,3 L0,6 Z" fill="#f97316"
        </marker>
        <marker id="arrow-pink" markerWidth="6" mar
            <path d="M0,0 L6,3 L0,6 Z" fill="#f472b6"
        </marker>
        <marker id="arrow-cyan" markerWidth="6" mar
            <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
        </marker>
        <marker id="arrow-indigo" markerWidth="6" m
            <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
        </marker>
        <marker id="arrow-lime" markerWidth="6" mar
            <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63f"
        </marker>
        <marker id="arrow-green" markerWidth="6" ma
            <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80"
        </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <g id="lego-red">
        <rect x="0" y="0" width="40" height="20"

```

```
        <svg xmlns="http://www.w3.org/2000/svg" class="lg:w-10 lg:h-10">
          <path stroke-linecap="round" stroke-linejoin="round" stroke="black" stroke-width="2" d="M6 9l9 9" />
        </svg>
      </button>
```

```
<!-- Затемнение фона на мобилках при открытом меню -->
<div id="drawer-overlay" class="lg:hidden fixed inset-0 bg-slate-500 bg-opacity-50 z-40">
```

```
<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-col">
```

```
<!-- Левое меню (Оглавление) -->
```

```
<aside id="mobile-drawer" class="fixed inset-y-0 left-0 w-full lg:w-72 lg:shrink-0 lg:z-auto
  translate-x-full transition-transform duration-300 ease-in-out
  w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto">
  <div class="h-full flex flex-col p-6 overflow-hidden">
    <div class="flex justify-between items-center">
      <h3 class="font-black text-transparent bg-clip-text text-slate-900">
        Содержание
      </h3>
      <button id="close-drawer-btn" class="lg:hidden text-slate-500">
        <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6">
          <path stroke-linecap="round" stroke-linejoin="round" stroke="black" stroke-width="2" d="M6 9l9 9" />
        </svg>
      </button>
    </div>
    <nav class="space-y-3 text-sm font-medium">
```

```
      <div class="group">
        <summary class="list-none cursor-pointer text-slate-500">
          <a href="#question-1" class="text-blue-500 pl-3 font-bold text-blue-300">
            1. Алгебраические структуры
          </a>
        </summary>
        <div class="pl-6 space-y-2 text-sm">
          <a href="#section-1-1" class="text-slate-500">
            1.1. Группы
          </a>
        </div>
      </div>
    </nav>
  </div>
</div>
```

er-amber-500 pl-3 font-bold text-amber-300">

4. Степени и корни

</summary>

<div class="pl-6 space-y-2 text

<a href="#q4-def" class="block l

<a href="#q4-moivre" class="blo

<a href="#q4-root" class="block

<a href="#q4-count" class="bloc

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-5" class:

er-fuchsia-500 pl-3 font-bold text-fuchsia-

5. Группа корней из единицы

</summary>

<div class="pl-6 space-y-2 text

<a href="#q5-def" class="block l

<a href="#q5-group" class="bloc

<a href="#q5-prim" class="block

<a href="#q5-crit" class="block

<a href="#q5-cyclic" class="blo

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-6" class:

er-lime-500 pl-3 font-bold text-lime-300">

6. Делимость, НОД и Евклид

</summary>

<div class="pl-6 space-y-2 text

<a href="#q6-1" class="block ho

```

        <a href="#q8-7" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-green-500 pl-3 font-bold text-green-400">
        9. Кольцо многочленов
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-green-400">
        <a href="#q9-1" class="block">
        <a href="#q9-2" class="block">
        <a href="#q9-3" class="block">
        <a href="#q9-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
        10. Делимость и Схема Г
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-rose-400">
        <a href="#q10-1" class="block">
        <a href="#q10-2" class="block">
        <a href="#q10-3" class="block">
        <a href="#q10-4" class="block">
        <a href="#q10-5" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">

```

```
<div class="pl-6 space-y-2 text
  <a href="#q13-1" class="blo
  <a href="#q13-2" class="blo
  <a href="#q13-3" class="blo
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-14" clas
der-fuchsia-500 pl-3 font-bold text-fuchsia
    14. Кратность корня. Ко
    </a>
  </summary>
  <div class="pl-6 mt-2 space-y-2
    <a href="#q14-1" class="blo
    <a href="#q14-2" class="blo
    <a href="#q14-3" class="blo
    deg)</a>
    <a href="#q14-4" class="blo
ition-colors border border-rose-500/30"> K
  </div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-15" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
    15. Производная. Критер
    </a>
  </summary>
  <div class="pl-6 mt-2 space-y-2
    <a href="#q15-1" class="blo
    <a href="#q15-2" class="blo
    <a href="#q15-3" class="blo
    <a href="#q15-4" class="blo
  </div>
</details>
```

```
        <span class="font-bold text-fuch
    </a>

    <a href="#proof-algorithms" class="l
-red-500 pl-3 mt-4 text-red-500 font-bold ml
    ▲ Ликбез: Алгоритмы выживания
    </a>

</nav>
<script>
    document.querySelectorAll('details.
    detail.addEventListener('toggle
        if (detail.open) {
            document.querySelectorA
            if (other !== detail
                other.removeAtt
        }
    });
    }
    });
    });

    // Scroll spy logic
    document.addEventListener('DOMConten
        const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
        let isScrolling = false;

    document.querySelectorAll('.grou
        link.addEventListener('clie
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersectio
        if (isScrolling) return;
```

```
    });

    function fallbackModal(contentRaw,
        const modal = document.createElement('div');
        modal.style.position = 'fixed';
        modal.style.top = '0';
        modal.style.left = '0';
        modal.style.width = '100vw';
        modal.style.height = '100vh';
        modal.style.backgroundColor = 'black';
        modal.style.zIndex = '99999';
        modal.style.display = 'flex';
        modal.style.flexDirection = 'column';
        modal.style.alignItems = 'center';
        modal.style.justifyContent = 'center';

        const box = document.createElement('div');
        box.style.width = '80%';
        box.style.height = '80%';
        box.style.backgroundColor = '#1a1a1a';
        box.style.padding = '20px';
        box.style.borderRadius = '10px';
        box.style.display = 'flex';
        box.style.flexDirection = 'column';
        box.style.color = '#fff';

        const header = document.createElement('div');
        header.innerText = 'Экспорт в ';
        header.style.marginBottom = '10px';

        const subHeader = document.createElement('div');
        subHeader.innerText = 'Из-за не  
ние веб-студии в новой вкладке (через меню)';
        subHeader.style.marginBottom = '10px';
        subHeader.style.color = '#f87171';
        subHeader.style.fontSize = '0.9em';

        const textarea = document.createElement('div');
```



```

        closeBtn.style.borderRadius = '50%';
        closeBtn.style.cursor = 'pointer';
        closeBtn.onclick = () => modal.close();

        box.appendChild(header);
        box.appendChild(subHeader);
        if (title !== 'PDF') {
            box.appendChild(textarea);
            btnContainer.appendChild(closeBtn);
        }
        btnContainer.appendChild(closeBtn);
        box.appendChild(btnContainer);
        modal.appendChild(box);
        document.body.appendChild(modal);
    }
}

```

```

function downloadMD() {
    let clone = document.querySelector('#md-content').cloneNode(true);

    // Smart markdown parsing based on DOM
    // Smart markdown parsing based on DOM
    clone.querySelectorAll('h2').forEach(el => {
        clone.querySelectorAll('h3').forEach(el => {
            clone.querySelectorAll('.analog').forEach(el => {
                clone.querySelectorAll('li').forEach(el => {
                    clone.querySelectorAll('svg').forEach(el => {
                        let caption = '';
                        if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
                            caption = el.nextElementSibling.alt;
                        }
                        el.textContent = '\n\n[ Иллюстрация: ' + caption + ' ]';
                    });
                });
            });
        });
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        let text = clone.innerText;
        text = text.replace(/\n{3,}/g, '\n\n');
    });
}

```



```

</script>

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo
  <!-- Кнопки Конвертации / Экспорта
  <div class="flex flex-col gap-2">
    <div class="text-xs text-slate-
    <button onclick="downloadWord()
ont-bold flex items-center justify-center g
      <span> </span> Word
    </button>
    <a href="algebra.pdf" target="_
-sm font-bold flex items-center justify-cen
      <span> </span> PDF
    </a>
    <button onclick="downloadMD()"
-bold flex items-center justify-center gap-
      <span> </span> Markdown
    </button>
    <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-cen
      <span> </span> HTML
    </button>
  </div>

  <!-- Смена темы -->
  <div>
    <div class="text-xs text-slate-
    <div class="flex items-center g
      <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
      <button id="theme-light" on
r justify-center text-amber-50 hover:text-w
">* </button>
      <button id="theme-terminal"

```

```

        <li><a href="#question-15">15. </li>
        <li><a href="#question-17">17. </li>
        <li><a href="#question-18">18. </li>
        <li><a href="#question-19">19. </li>
        <li><a href="#question-20">20. </li>
        <li><a href="#question-21">21. </li>
        <li><a href="#question-22">22. </li>
    </ul>
</div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
    <h2 class="text-3xl font-black text-cen

    <div class="overflow-x-auto w-full pb-4">
        <div class="grid grid-cols-3 gap-4">
            <!-- Column headers -->
            <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
            <div class="bg-orange-900/40 bo
ольцо полиномов</div>
            <div class="bg-cyan-900/40 bord
сные числа</div>

            <!-- Row 1: База -->
            <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
                <div class="text-xs text-yellow
                <div class="font-bold text-slat
            </a>
            <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
                <div class="text-xs text-orange
                <div class="font-bold text-slat
            </a>
            <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans

```

```

                <div class="text-xs text-yellow
                <div class="font-bold text-slate
            </a>
            <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
                <div class="text-xs text-orange
                <div class="font-bold text-slate
            </a>
            <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
                <div class="text-xs text-cyan-5
                <div class="font-bold text-slate
            </a>

            <!-- Row 5: Корни / Уравнения -->
            <div class="hidden md:block"></div>
            <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
                <div class="text-xs text-orange
                <div class="font-bold text-slate
            </a>
            <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
                <div class="text-xs text-cyan-5
                <div class="font-bold text-slate
            </a>

            <!-- Row 6: Кратность -->
            <div class="hidden md:block"></div>
            <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
                <div class="text-xs text-orange
                <div class="font-bold text-slate
            </a>
            <div class="hidden md:block"></div>

            <!-- Row 7: Производная -->
            <div class="hidden md:block"></div>

```

```
border-l-4 border-fuchsia-500 hover:bg-slate-700
    <div class="text-xs text-fuchsia-500">
    <div class="font-bold text-slate-700">
</a>
```

```
<!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ -->
<div class="col-span-1 md:col-span-3" style="border: 2px solid red; padding: 10px; border-radius: 10px;">Кластер 4:
```

```
    <a href="#proof-algorithms" onclick="document.location.href='#proof-algorithms'"
:col-span-3 bg-red-900/20 p-4 border-l-4 border-r-4 border-t-4 border-b-4" style="border: 2px solid red; padding: 10px; border-radius: 10px;">
    <div class="text-xs text-red-500">
    <div class="font-bold text-slate-700">
</a>
</div>
</div>
```

```
<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="mb-6">
    <div class="flex items-center mb-6">
        <div class="bg-red-500/20 p-3 rounded" style="border: 2px solid red; padding: 10px; border-radius: 10px;">
            <svg class="w-8 h-8 text-red-500" style="display: block; margin: 0 auto;">
                <path stroke-linecap="round" stroke-width="2" d="M110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4" />
            </svg>
        </div>
        <h2 class="text-3xl pr-4 border-r-4 border-t-4 border-b-4" style="border: 2px solid red; padding: 10px; border-radius: 10px;">
</div>
```

```
    <div class="prose prose-invert max-w-4xl">
        <p class="text-lg">
            Хватит тупить. Доказательств
            ь для того, чтобы жать на кнопки. Вот жес
        </p>
```

```
        <h2 class="text-2xl font-bold border-top-2px solid red; padding-top: 10px;">
(Меметика)</h2>
```

```

        </ul>
    </div>

    <!-- Индукция -->
    <div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
        <h3 class="text-xl text-yellow-400">Индукция
        <p class="text-sm text-slate-400">
            <ul class="list-disc pl-5" style="list-style-type: none;">
                <li><span class="text-white">
                    валось.</li>
                    <li><span class="text-white">
                        Просто перепиши формулу с буквой  $k$ .</li>
                    <li><span class="text-white">
                        усок  $k$ , замени его на жареного карася из
                </ul>
            </div>

    <!-- Двойная делимость -->
    <div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
        <h3 class="text-xl text-orange-400">Двойная делимость
        <p class="text-sm text-slate-400">
            <ul class="list-disc pl-5" style="list-style-type: none;">
                <li><span class="text-white">
                    вверх по уравнениям).</li>
                    <li><span class="text-white">
                        рху вниз, показывая, что  $\Delta \vdots r_i$ 
                    <li><span class="text-white">
                        х. Твой  $d$  больше.</li>
                </ul>
            </div>

    <h2 class="text-2xl font-bold" style="color: white; margin-top: 20px;">
        ритмы экзекуции)</h2>

    <div class="grid grid-cols-1 md:grid-cols-2" style="margin-top: 20px;">
        <div class="bg-slate-800 p-6" style="background-color: #1a202c; color: white;">
            <h4 class="text-red-400">

```

```
<span class="text-2xl">
<div>
  <h4 class="text-cyan">
    <p class="text-sm text-cyan">
      <code class="text-x
    </div>
</li>

<li class="bg-slate-800/50 p-4">
  <span class="text-2xl">
    <div>
      <h4 class="text-cyan">
        <p class="text-sm text-cyan">
          x двух, потом последних. <br>
          <code class="text-x
        </div>
      </li>

      <li class="bg-slate-800/50 p-4">
        <span class="text-2xl">
          <div>
            <h4 class="text-cyan">
              <p class="text-sm text-cyan">
                есть сложение – есть ноль. Если есть умноже
                <code class="text-x
              </div>
            </li>

            <li class="bg-slate-800/50 p-4">
              <span class="text-2xl">
                <div>
                  <h4 class="text-cyan">
                    <p class="text-sm text-cyan">
                      авь их драться. Умножение всегда врывается
                      <code class="text-x
                    </div>
                  </li>
                </ul>
```



```
* Файл получается автономным: стили защиты внутрь, инт
* его можно открыть с флешки, отправить в мессенджер и
* Живые React-визуализации в статику не переносятся —
* подставляется заметка со ссылкой на онлайн-версию, а
* которые в приложении всплывают по наведению, собираю
* в конце документа.
*/
```

```
/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
  const parts: string[] = [];
  for (const sheet of Array.from(document.styleSheets))
    try {
      const rules = (sheet as CSSStyleSheet).cssRules;
      parts.push(Array.from(rules).map((r) => r.cssText));
    } catch {
      const href = (sheet as CSSStyleSheet).href;
      if (!href) continue;
      try {
        const res = await fetch(href);
        if (res.ok) parts.push(await res.text());
      } catch {
        /* чужой домен — пропускаем */
      }
    }
  }
  return parts.join("\n");
}
```

```
const EXTRA_CSS = `
/* — правки для автономного файла — */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; paddi
.export-head { border-bottom: 1px solid #1e293b; paddin
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size
```

```

    if (!id) return;
    if (!ids.includes(id)) ids.push(id);
    const link = document.createElement("a");
    link.className = "term-hint";
    link.href = `#term-${id}`;
    link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
    link.textContent = el.textContent ?? "";
    el.replaceWith(link);
  });

  // <details> в файле лучше раскрыть: иначе код и разб
  host.querySelectorAll("details").forEach((d) => d.set

  return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
    })
    .join("\n");

  return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
  ны сюда.</p>
  ${items}
</section>`;
}

```

```
-----  
function triggerDownload(html: string, filename: string) {  
  const blob = new Blob([html], { type: "text/html;charset=utf-8" });  
  const url = URL.createObjectURL(blob);  
  const a = document.createElement("a");  
  a.href = url;  
  a.download = filename;  
  document.body.appendChild(a);  
  a.click();  
  a.remove();  
  setTimeout(() => URL.revokeObjectURL(url), 2000);  
}
```

```
/** Имя файла: «04-splay-derevo.html» – без кириллицы и пробелов  
function safeName(title: string, id: string): string {  
  const num = title.match(/^s*(\d+)/)?.[1];  
  const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();  
  return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;  
}
```

```
/** Выгрузить одну главу. */  
export async function downloadChapterHtml(chapter: Chapter) {  
  const css = await collectCss();  
  const { body, termIds } = prepareBody(chapter.content);  
  const html = wrap({  
    title: chapter.title,  
    subtitle: chapter.category || "Универсальное пособие",  
    css,  
    body,  
    gloss: glossarySection(termIds),  
    origin: window.location.origin,  
  });  
  triggerDownload(html, safeName(chapter.title, chapter.id));  
}
```

```
/** Выгрузить всё пособие одним файлом со сквозным оглавлением  
export async function downloadBookHtml(chapters: Chapter[]) {  
  const css = await collectCss();  
  const allTerms: string[] = [];  
  const { body, termIds } = prepareBody(chapters.map(chapter => chapter.content));  
  const html = wrap({  
    title: "Универсальное пособие",  
    subtitle: "Универсальное пособие",  
    css,  
    body,  
    gloss: glossarySection(allTerms),  
    origin: window.location.origin,  
  });  
  triggerDownload(html, safeName("Универсальное пособие", "book"));  
}
```

ФАЙЛ 76/82: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 *   node scripts/audit-coverage.mjs           # отчёт
 *   node scripts/audit-coverage.mjs --md      # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2
```

```

-----
console.log(`Глав в пособии: ${rows.length}\n`);
const dump = (title, list) => {
  console.log(`${title} (${list.length}):`);
  list.forEach((r) => console.log(`  * ${r.title}  [$
  console.log();
};
dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
dump("НЕТ 2D (аналогия есть)", gapViz);
console.log(`Пропущенные номера билетов 1-24: ${missi
console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

ФАЙЛ 77/82: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

-----
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 *   node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossa
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m
  external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY LABELS, GLOSSARY } = await import(path

```

```
    if (ids.size === 0) bad.push(c.title);
    console.log(
      String(ids.size).padStart(3), "|", String(highlight
      c.title.slice(0, 46).padEnd(47), [...ids].slice(0,
    );
  }
  console.log("\nГлав без единой подсказки:", bad.length,
  console.log("Терминов в глоссарии:", GLOSSARY.length, "
  console.log("Не встретились:", GLOSSARY.filter((g) => !
```

ФАЙЛ 78/82: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРӨК: 64 | РАЗМ

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
  if (!fs.existsSync(PAGES_DIR)) {
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`
  }

  const pages = fs
    .readdirSync(PAGES_DIR)
```

```
main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
```

ФАЙЛ 79/82: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТР0К: 196 | РАЗМ:

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текст
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недо
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"]
];
```

```

    files = [];
    const walk = (dir) => {
      for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
        const abs = path.join(dir, e.name);
        const rel = path.relative(ROOT, abs).split(path.sep);
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
        if (e.isDirectory()) walk(abs);
        else files.push(rel);
      }
    };
    walk(ROOT);
  }

  return files
    .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
    .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
    .filter((rel) => CODE_EXT.has(path.extname(rel)))
    .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
    .sort((a, b) => {
      const ia = ORDER.indexOf(categoryOf(a));
      const ib = ORDER.indexOf(categoryOf(b));
      return ia !== ib ? ia - ib : a.localeCompare(b);
    });
}

```

/** Достаём заголовки глав из данных пособия (без запуска)

```

function extractChapters(files) {
  const chapters = [];
  const seen = new Set();
  for (const rel of files.filter((f) => f.startsWith("chapters/"))) {
    const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
    const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
    let m;
    while ((m = re.exec(src)) !== null) {
      const [, id, title] = m;
      if (seen.has(id)) continue;
      seen.add(id);
      chapters.push({ id, title, file: rel });
    }
  }
}

```



```

push(HR);
push("
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
  push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`);
});
push();
push();

current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push(HR);
    push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
    push(HR);
    push();
  }
  const content = fs.readFileSync(path.join(ROOT, rel), "utf-8");
  const lines = content.split("\n");
  push(SUBHR);
  push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
  push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
  push(SUBHR);
  push();
  push(content.replace(/\n+$/, ""));
  push();
  push();
});

```

Универсальное пособие • полный исходный код

```
-----
export const OUT_DIR = path.join(ROOT, "public", "export");
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_and_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_and_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
  density: Number(process.env.EXPORT_DENSITY ?? 150),
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",
  size: "A4",
  font: "DejaVu-Sans-Mono",
  fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
  pointsize: 8,
  /** Максимум символов в строке (далее – жёсткий перенос) */
  cols: 138,
  /** Строк содержимого на странице (+2 служебные: шапка, подвал) */
  rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  return process.env.IMAGEMAGICK_CMD || "magick";
}
```

```
-----  
/** Раскрываем табы и жёстко переносим длинные строки,  
function wrap(line) {  
  const expanded = line.replace(/\t/g, "    ").replace(  
  if (expanded.length <= PAGE.cols) return [expanded];  
  const parts = [];  
  const indent = " ".repeat(Math.min((expanded.match(/  
  let rest = expanded;  
  let first = true;  
  while (rest.length > 0) {  
    const width = first ? PAGE.cols : PAGE.cols - indent;  
    parts.push((first ? "" : indent) + rest.slice(0, wi  
    rest = rest.slice(width);  
    first = false;  
  }  
  return parts;  
}  
  
function paginate(txt) {  
  const source = txt.replace(/\r\n/g, "\n").split("\n")  
  const pages = [];  
  for (let i = 0; i < source.length; i += PAGE.rows) {  
    pages.push(source.slice(i, i + PAGE.rows));  
  }  
  return pages;  
}  
  
async function main() {  
  if (!fs.existsSync(TXT_PATH)) {  
    throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)  
  }  
  
  const im = imageMagickCmd();  
  const pages = paginate(fs.readFileSync(TXT_PATH, "utf8  
  
  fs.rmSync(PAGES_DIR, { recursive: true, force: true }  
  ensureDir(PAGES_DIR);  
  
  const total = pages.length;
```

```
        process.stdout.write(`      отрендерено ${done}`);
      }
    }
  }

  await Promise.all(Array.from({ length: concurrency },
    process.stdout.write("\n"));

  const missing = jobs.filter((j) => !fs.existsSync(j.path));
  if (missing.length) throw new Error(`Не отрендерены с ${missing.length} страниц`);

  const bytes = jobs.reduce((s, j) => s + fs.statSync(j.path).size, 0);
  console.log("[2/3] txt -> png");
  console.log(`      страниц: ${total}`);
  console.log(`      выход:   ${path.relative(ROOT, PAGE_PATH)}`);
}

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});
```

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

```
ФАЙЛ 82/82: .github/workflows/rebuild-pdf.yml
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ
```

name: Rebuild code PDF

```
# Пайплайн пересборки PDF при каждом коммите:
#
#   исходный код → full_code_all_pages.txt   (script)
#   |
```



```
-----
git add -- public/export/full_code_all_pages.
if git diff --cached --quiet; then
    echo "Экспорт не изменился – коммит не нужен"
    exit 0
fi
git commit -m "chore(export): пересборка full"
git push origin "HEAD:${GITHUB_REF_NAME}"
```